

Important Information

Thank you for choosing Freenove products!

Getting Started

First, please read the **Start Here.pdf** document in the unzipped folder you created.

If you have not yet downloaded the zip file, associated with this kit, please do so now and unzip it.

Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be **used only when there is adult supervision present** as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. **Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.**
- When the product is turned ON, activated or tested, some parts will move or rotate. **To avoid injuries to hands and fingers keep them away from any moving parts!**
- It is possible that an improperly connected or shorted circuit may cause overheating. **Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it cools down!** When everything is safe and cool, review the product tutorial to identify the cause.
- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

<http://www.freenove.com>

sale@freenove.com

Copyright

All the files, materials and instructional guides provided are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](https://creativecommons.org/licenses/by-nc-sa/3.0/). A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.



Contents

Important Information.....	1
Contents.....	3
Preface.....	5
ESP32-S3 WROOM.....	6
CH343 (Importance)	8
Programming Software.....	19
Environment Configuration.....	22
Notes for GPIO.....	25
Chapter 1 LED	28
Project 1.1 Blink	28
Chapter 2 Serial Communication.....	37
Project 2.1 Serial Print	37
Project 2.2 Serial Read and Write.....	41
Chapter 3 Bluetooth.....	43
Project 3.1 Bluetooth Low Energy Data Passthrough.....	43
Chapter 4 Read and Write the SDcard	55
Project 4.1 SDMMC Test.....	55
Chapter 5 WiFi Working Modes.....	67
Project 5.1 Station mode	67
Project 5.2 AP mode	72
Project 5.3 AP+Station mode.....	77
Chapter 6 TCP/IP.....	81
Project 6.1 As Client.....	81
Project 6.2 As Server.....	93
Chapter 7 Camera Web Server.....	99
Project 7.1 Camera Web Server	99
Project 7.2 Video Web Server	109
Project 7.3 Camera and SDcard.....	116
Chapter 8 Camera Tcp Server.....	126
Project 8.1 Camera Tcp Server.....	126

Chapter 9 USB.....	144
Project 9.1 USB Serial Example	144
Project 9.2 USB Mouse Example.....	153
Project 9.3 USB Keypad Example.....	160
Project 9.4 USB Control Device Example.....	166
What's next?	172
End of the Tutorial.....	172

Preface

ESP32-S3 is a micro control unit with integrated Wi-Fi launched by Espressif, which features strong properties and integrates rich peripherals. It can be designed and studied as an ordinary Single Chip Microcontroller (SCM) chip, or connected to the Internet and used as an Internet of Things device.

ESP32-S3 can be developed using the Arduino platform, which will definitely make it easier for people who have learned Arduino to master. Moreover, the code of ESP32-S3 is completely open-source, so beginners can quickly learn how to develop and design IOT smart household products including smart curtains, fans, lamps and clocks.

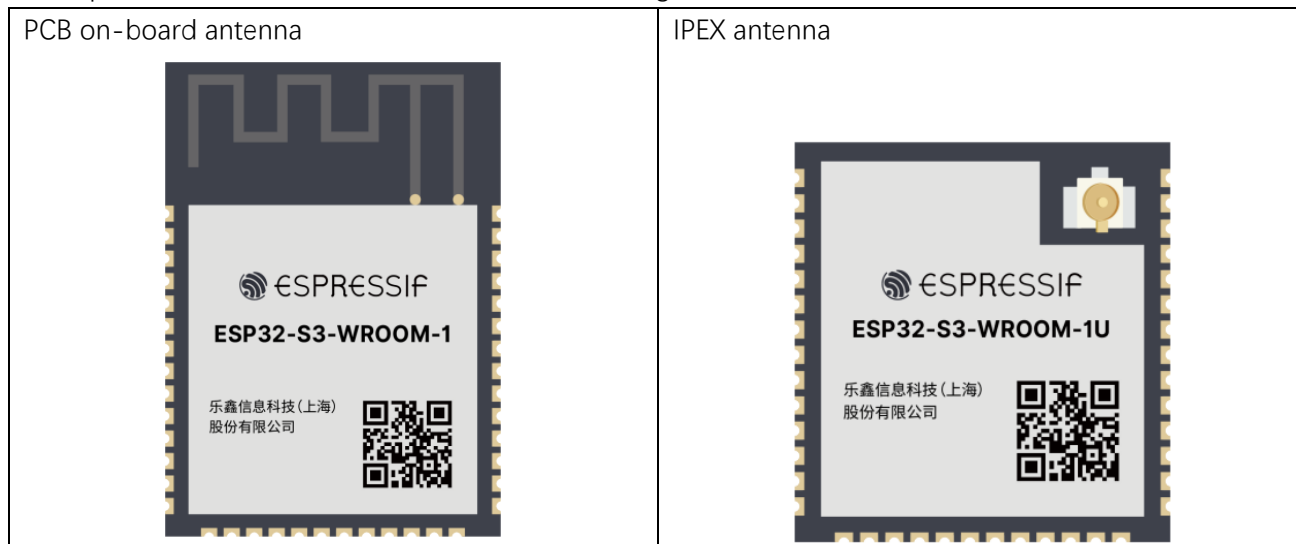
Generally, ESP32-S3 projects consist of code and circuits. Don't worry even if you've never learned code and circuits, because we will gradually introduce the basic knowledge of C programming language and electronic circuits, from easy to difficult. Our products contain all the electronic components and modules needed to complete these projects. It's especially suitable for beginners.

We divide each project into four parts, namely Component List, Component Knowledge, Circuit and Code. Component List helps you to prepare material for the experiment more quickly. Component Knowledge allows you to quickly understand new electronic modules or components, while Circuit helps you understand the operating principle of the circuit. And Code allows you to easily master the use of ESP32 and accessory kit. After finishing all the projects in this tutorial, you can also use these components and modules to make products such as smart household, smart cars and robots to transform your creative ideas into prototypes and new and innovative products.

In addition, if you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com

ESP32-S3 WROOM

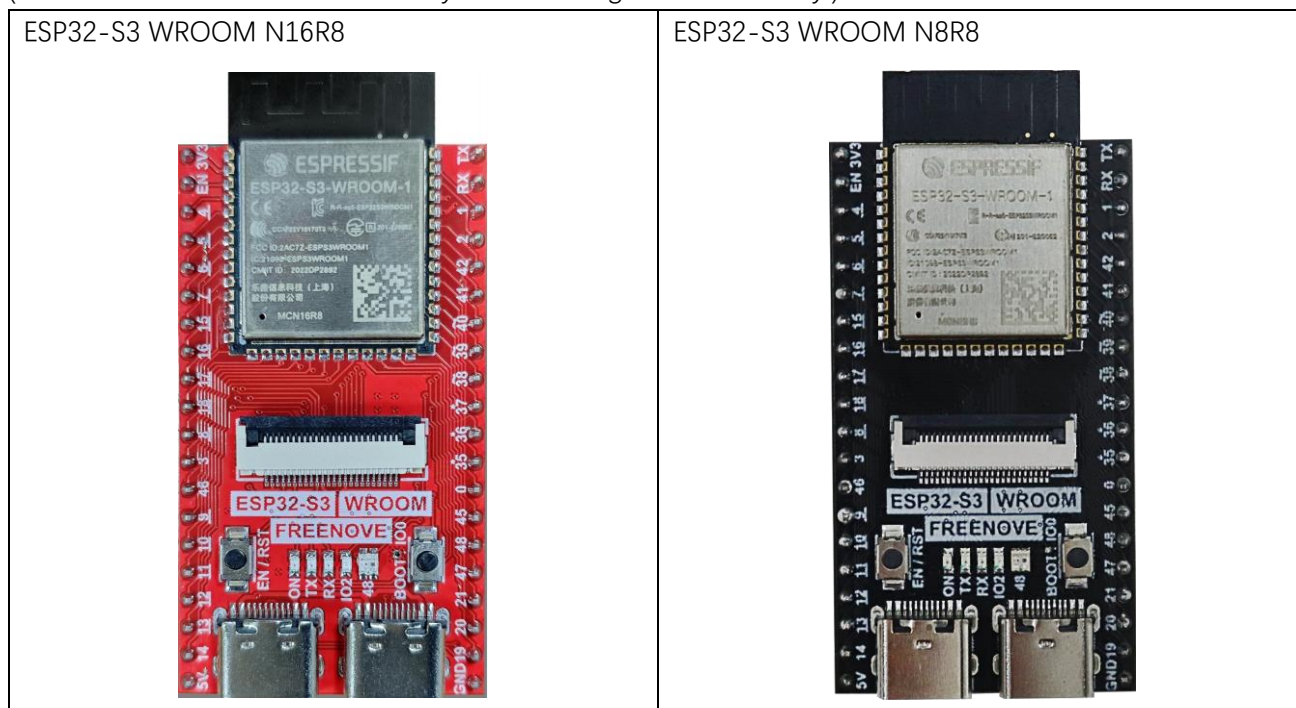
ESP32-S3-WROOM-1 has launched a total of two antenna packages, PCB on-board antenna and IPEX antenna respectively. The PCB on-board antenna is an integrated antenna in the chip module itself, so it is convenient to carry and design. The IPEX antenna is a metal antenna derived from the integrated antenna of the chip module itself, which is used to enhance the signal of the module.



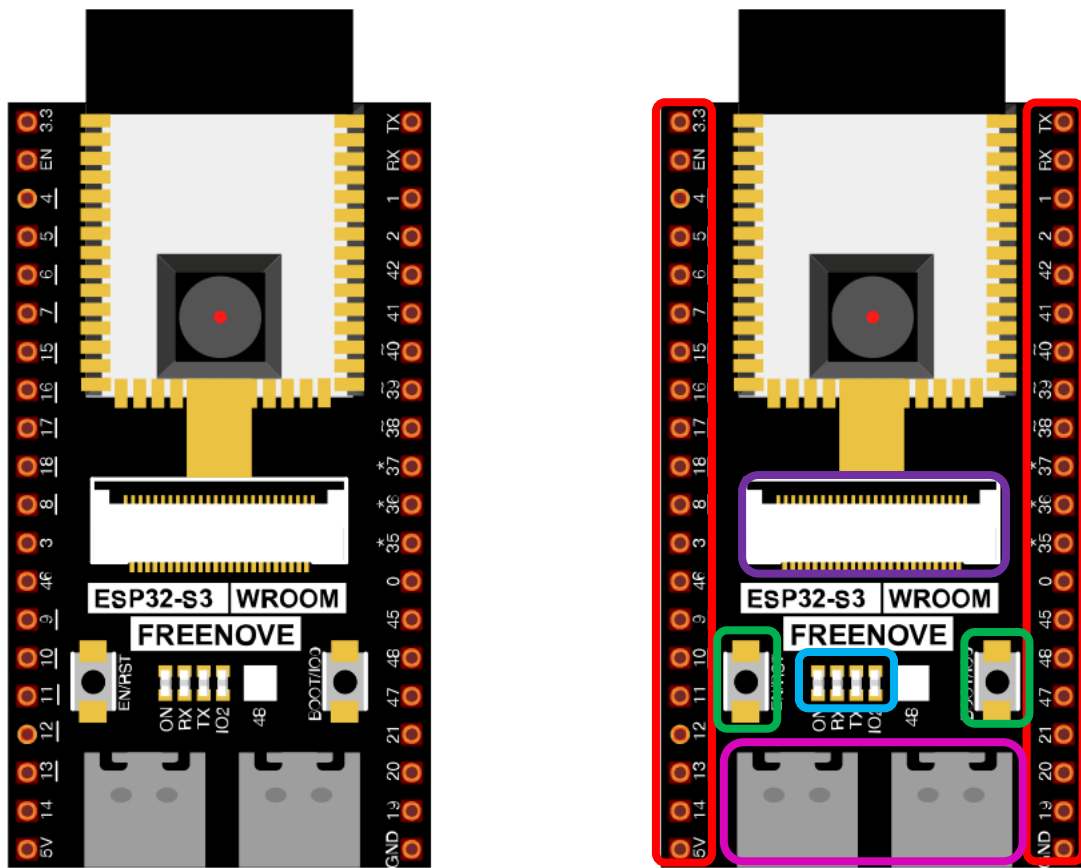
In this tutorial, the ESP32-S3 WROOM is designed based on the PCB on-board antenna-packaged ESP32-S3-WROOM-1 module.

Currently, we have two development board options available: the N8R8(**8MB Flash + 8MB PSRAM**) version with a black PCB and the N16R8(**16MB Flash + 8MB PSRAM**) version with a red PCB. The only difference between these two boards lies in their built-in Flash storage capacity, while all other features, functionalities, and peripheral circuits remain identical.

(Note: The N16R8 version is currently sold as a single-board kit only.)



The hardware interfaces of ESP32-S3 WROOM are distributed as follows:



Compare the left and right images. We've boxed off the resources on the ESP32-S3 WROOM in different colors to facilitate your understanding of the ESP32-S3 WROOM.

Box color	Corresponding resources introduction
	GPIO pin
	LED indicator
	Camera interface
	Reset button, Boot mode selection button
	USB port

For more information, please visit: https://www.espressif.com.cn/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf.

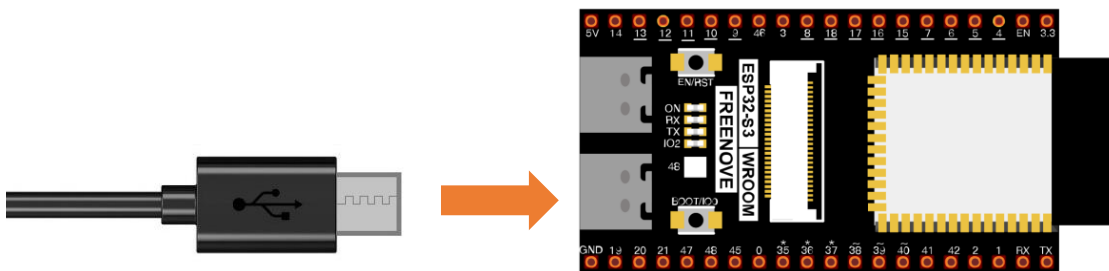
CH343 (Importance)

ESP32-S3 WROOM uses CH343 to download codes. So before using it, we need to install CH343 driver in our computers.

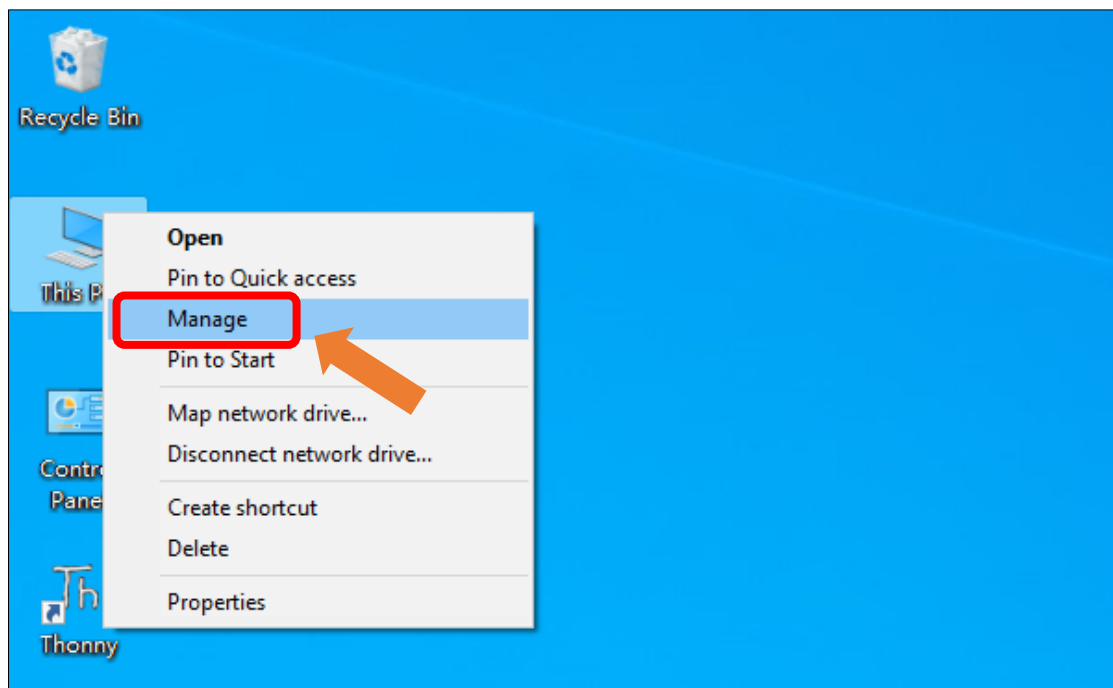
Windows

Check whether CH343 has been installed

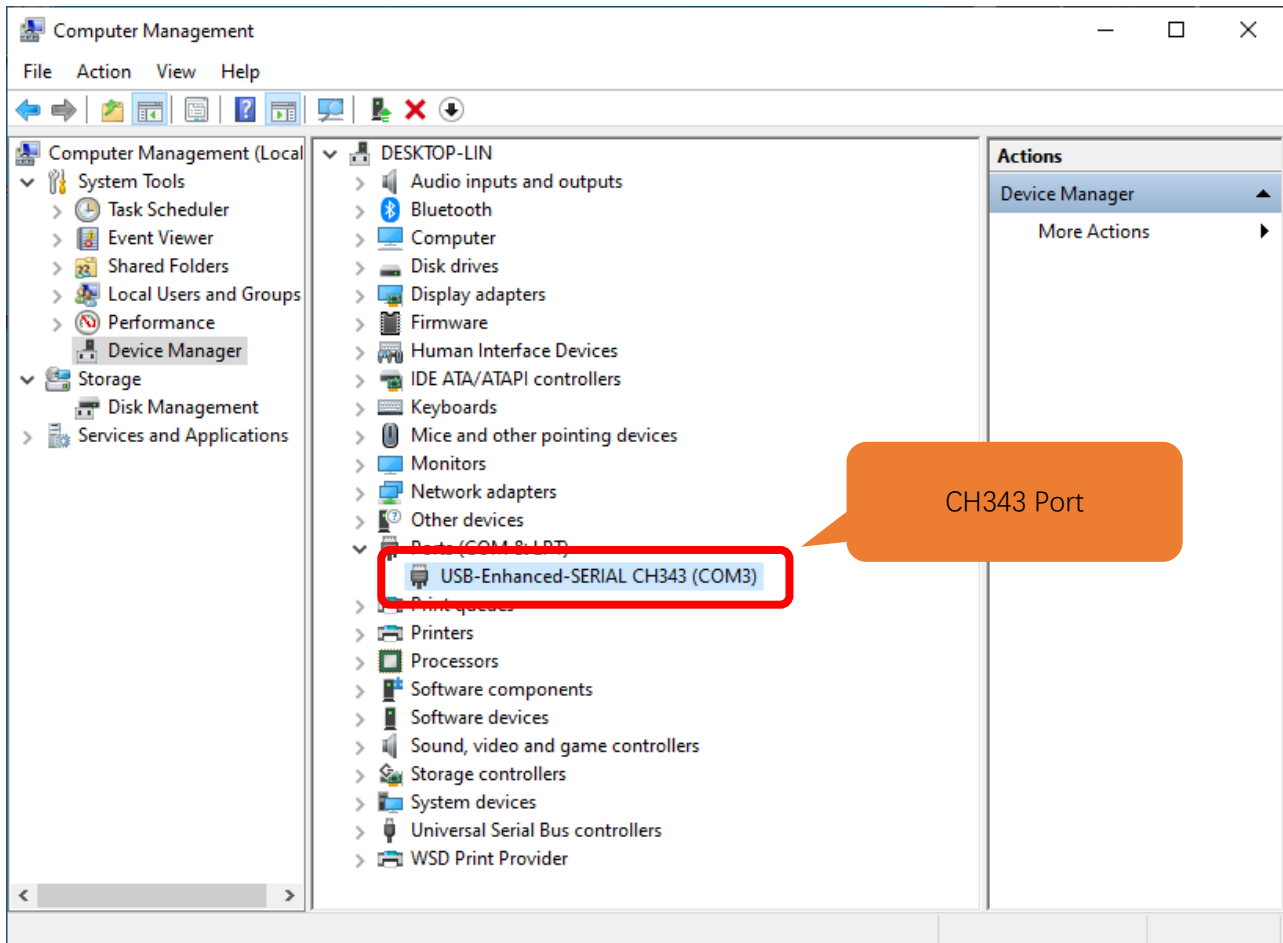
1. Connect your computer and ESP32-S3 WROOM with a USB cable.



2. Turn to the main interface of your computer, select "This PC" and right-click to select "Manage".



- Click “Device Manager”. If your computer has installed CH343, you can see “USB-Enhanced-SERIAL CH343 (COMx)”. And you can click [here](#) to move to the next step.



Installing CH343

- First, download CH343 driver, click <http://www.wch-ic.com/search?t=all&q=ch343> to download the appropriate one based on your operating system.

keyword ch343

Downloads(8)

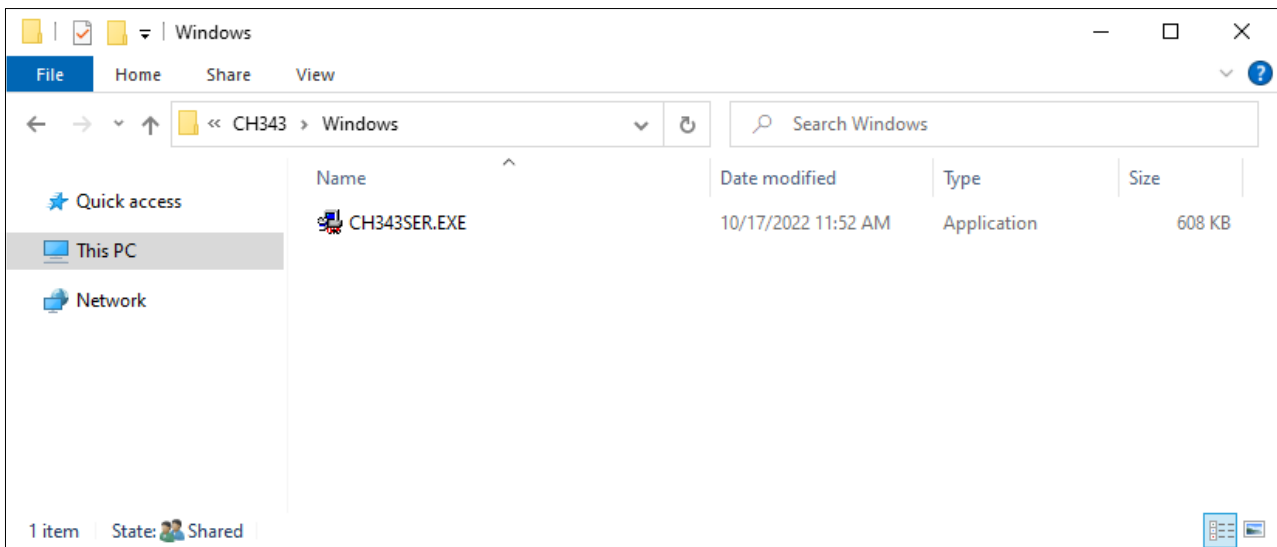
file category	file content	version	upload time
DataSheet			
CH343DS1.PDF	CH343 datasheet, USB to single serial port, supports up to 6M baud rate, serial port signals support 5V/3.3V/2.5V/1.8V, built-in crystal oscillator. CH343 supports built-in CDC driver in operating system or multi-functional high-speed VCP manufacture driver.	1.5	2021-11-18
Driver&Tools			
CH343SER.ZIP	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13
CH343CDC.ZIP	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13
CH343SER.EXE	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13
CH34XSER_MAC.ZIP	For CH340/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to serial port VCP vendor driver of macOS	1.7	2022-05-13
CH343CDC.EXE	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13
Application			
CH34xSerCfg.ZIP	USB configuration tool of Windows for CH340/CH342/CH343/CH344/CH347/CH348/CH9101/CH9102/CH9103. Via this tool, the chip's Vendor ID, product ID, maximum current value, BCD version	1.2	2022-05-24

If you would not like to download the installation package, you can open

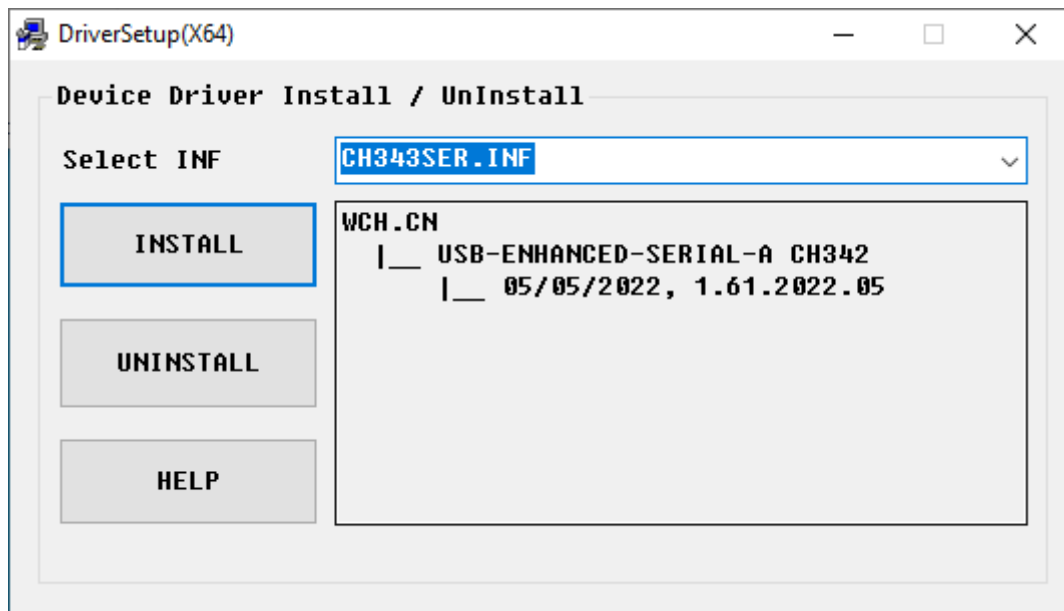
“Freenove_ESP32_S3_WROVER_Board/CH343”, we have prepared the installation package.

 Linux	10/17/2022 1:30 PM	File folder
 MAC	10/17/2022 1:30 PM	File folder
 Windows	10/17/2022 1:30 PM	File folder

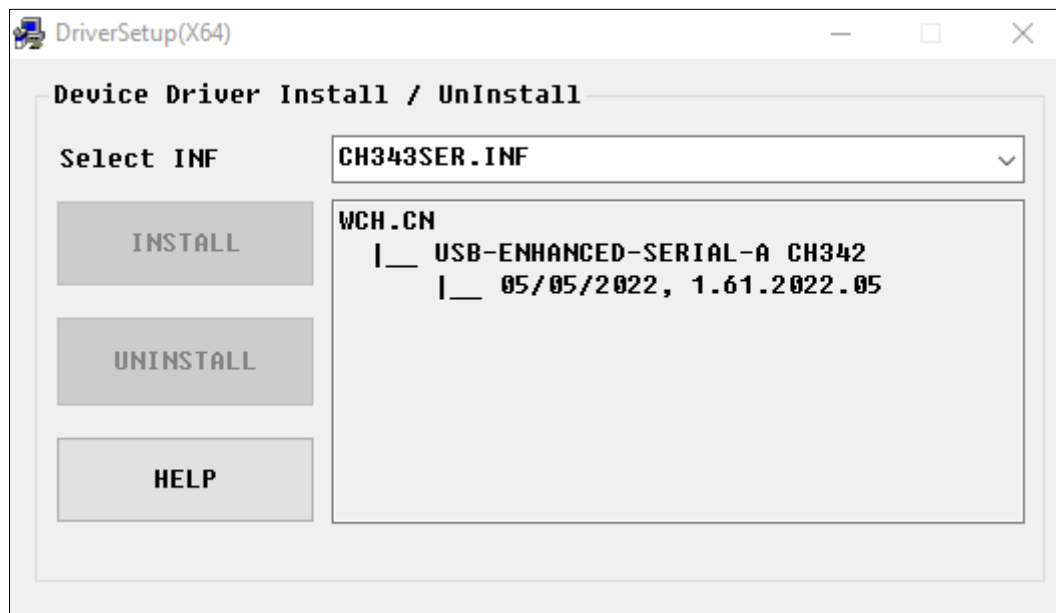
2. Open the folder "Freenove_ESP32_S3_WROVER_Board/CH343/Windows/"



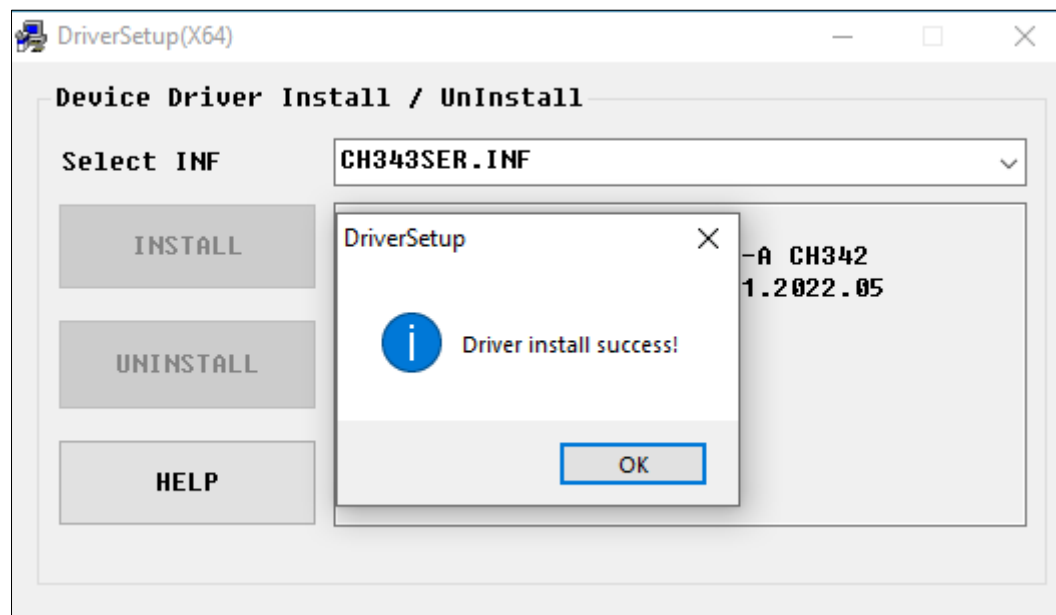
3. Double click "CH343SER.EXE".



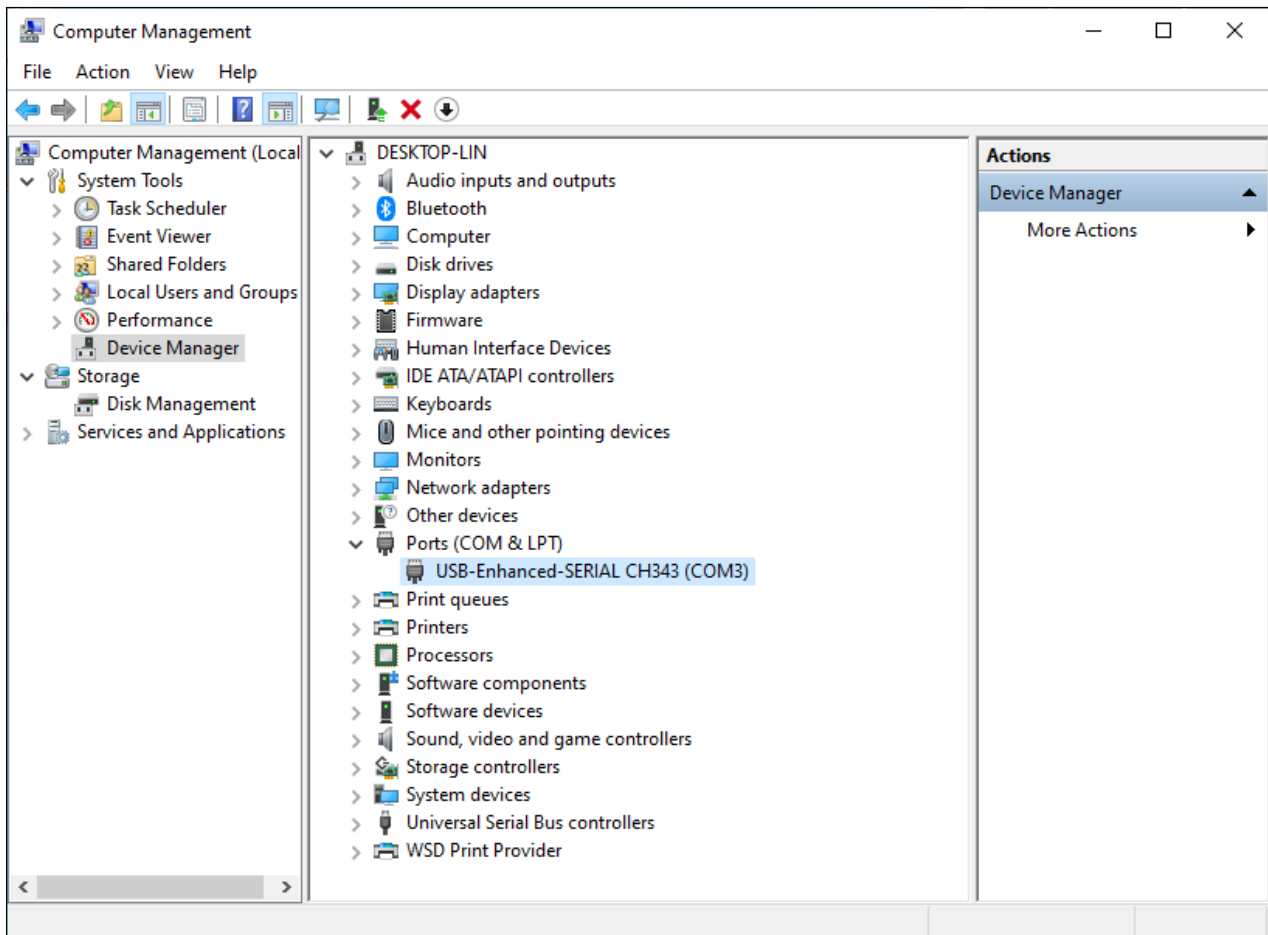
- Click "INSTALL" and wait for the installation to complete.



- Install successfully. Close all interfaces.



6. When ESP32-S3 WROOM is connected to computer, select “This PC”, right-click to select “Manage” and click “Device Manager” in the newly pop-up dialog box, and you can see the following interface.



7. So far, CH343 has been installed successfully. Close all dialog boxes.

MAC

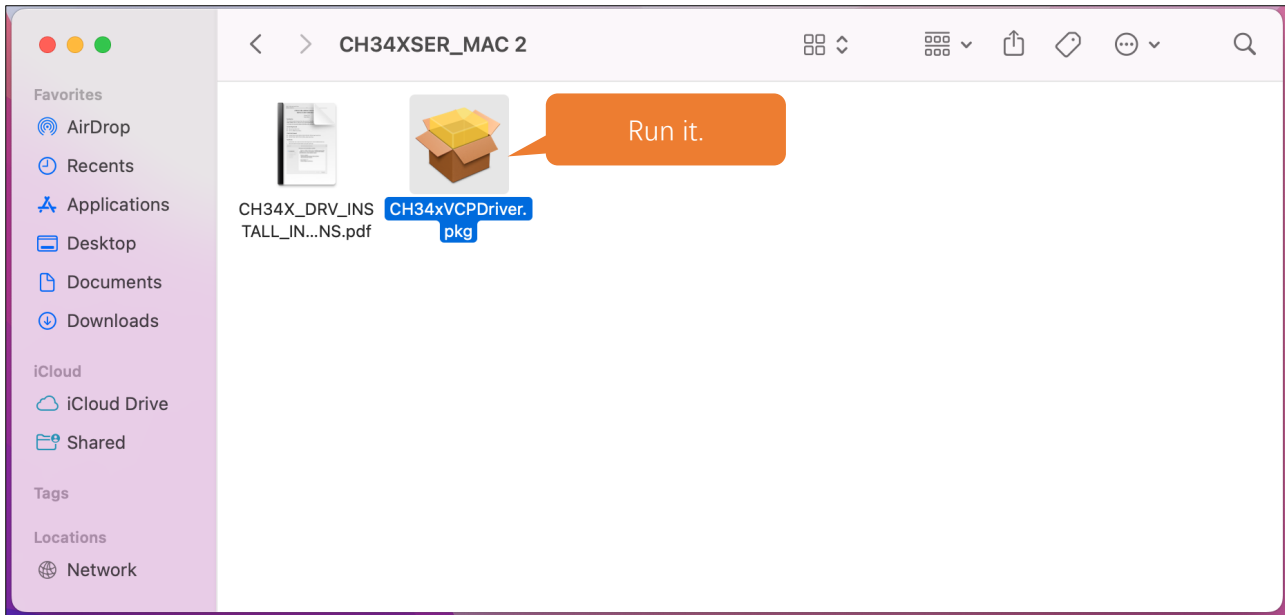
First, download CH343 driver, click <http://www.wch-ic.com/search?t=all&q=ch343> to download the appropriate one based on your operating system.

keyword ch343			
Downloads(8)			
file category	file content	version	upload time
DataSheet			
CH343DS1.PDF	CH343 datasheet, USB to single serial port, supports up to 6M baud rate, serial port signals support 5V/3.3V/2.5V/1.8V, built-in crystal oscillator. CH343 supports built-in CDC driver in operating system or multi-functional high-speed VCP manufacture driver.	1.5	2021-11-18
Driver&Tools			
CH343SER.ZIP	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13
CH343CDC.ZIP	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13
CH343SER.EXE	For CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to high-speed serial port VCP vendor driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.61	2022-05-13
CH34XSER_MAC.ZI...	For CH340/CH342/CH343/CH344/CH347/CH9101/CH9102/CH9103/CH9143, USB to serial port VCP vendor driver of macOS	1.7	2022-05-13
CH343CDC.EXE	For CH342/CH343/CH344/CH347/CH910X/CH9143/CH9340, USB to CDC serial port driver, supports Windows 11/10/8.1/8/7/VISTA/XP/2000	1.4	2022-05-13
Application			
CH34xSerCfg.ZIP	USB configuration tool of Windows for CH340/CH342/CH343/CH344/CH347/CH348/CH9101/CH9102/CH9103. Via this tool, the chip's Vendor ID, product ID, maximum current value, BCD version	1.2	2022-05-24

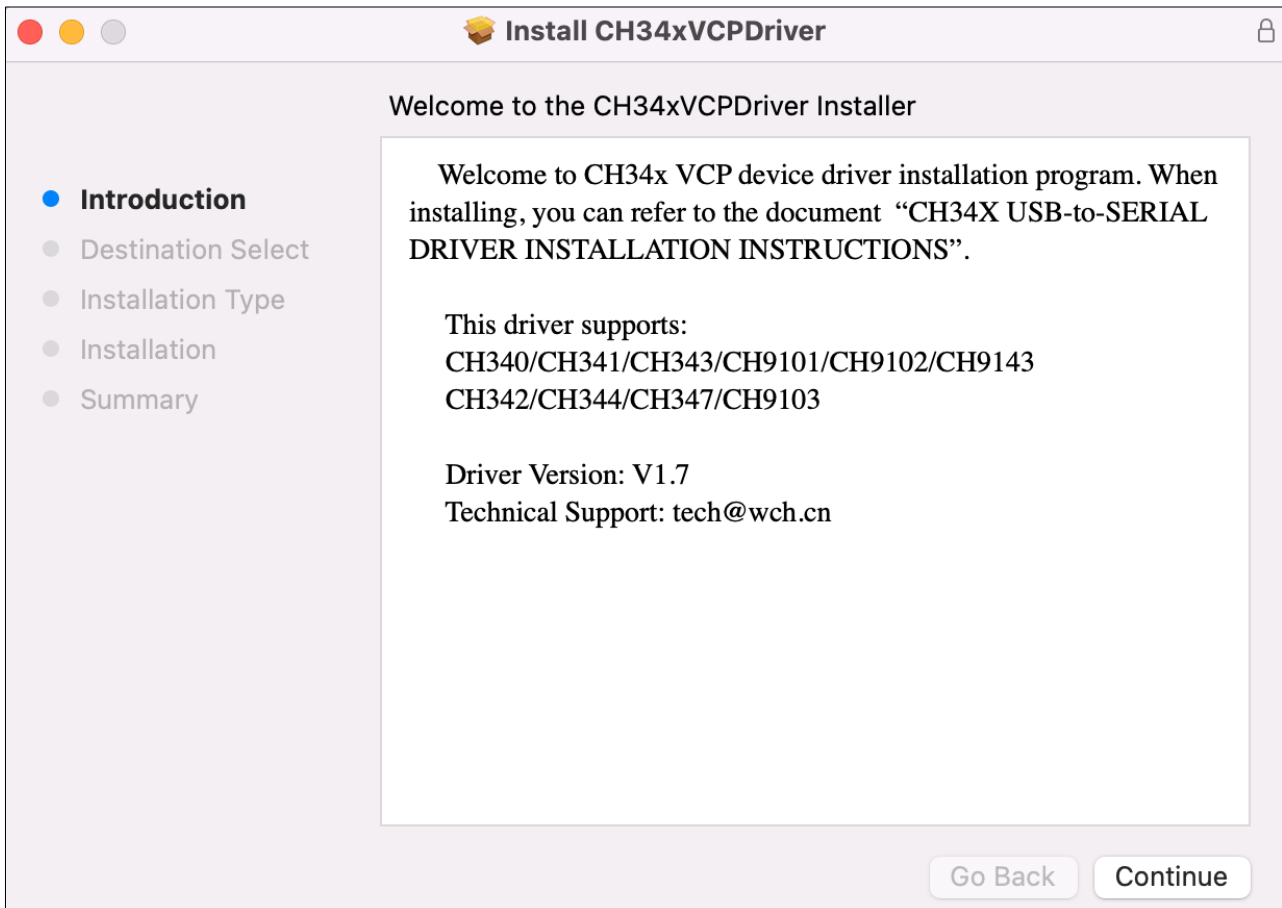
If you would not like to download the installation package, you can open

“**Freenove_ESP32_S3_WROVER_Board/CH343**”, we have prepared the installation package.

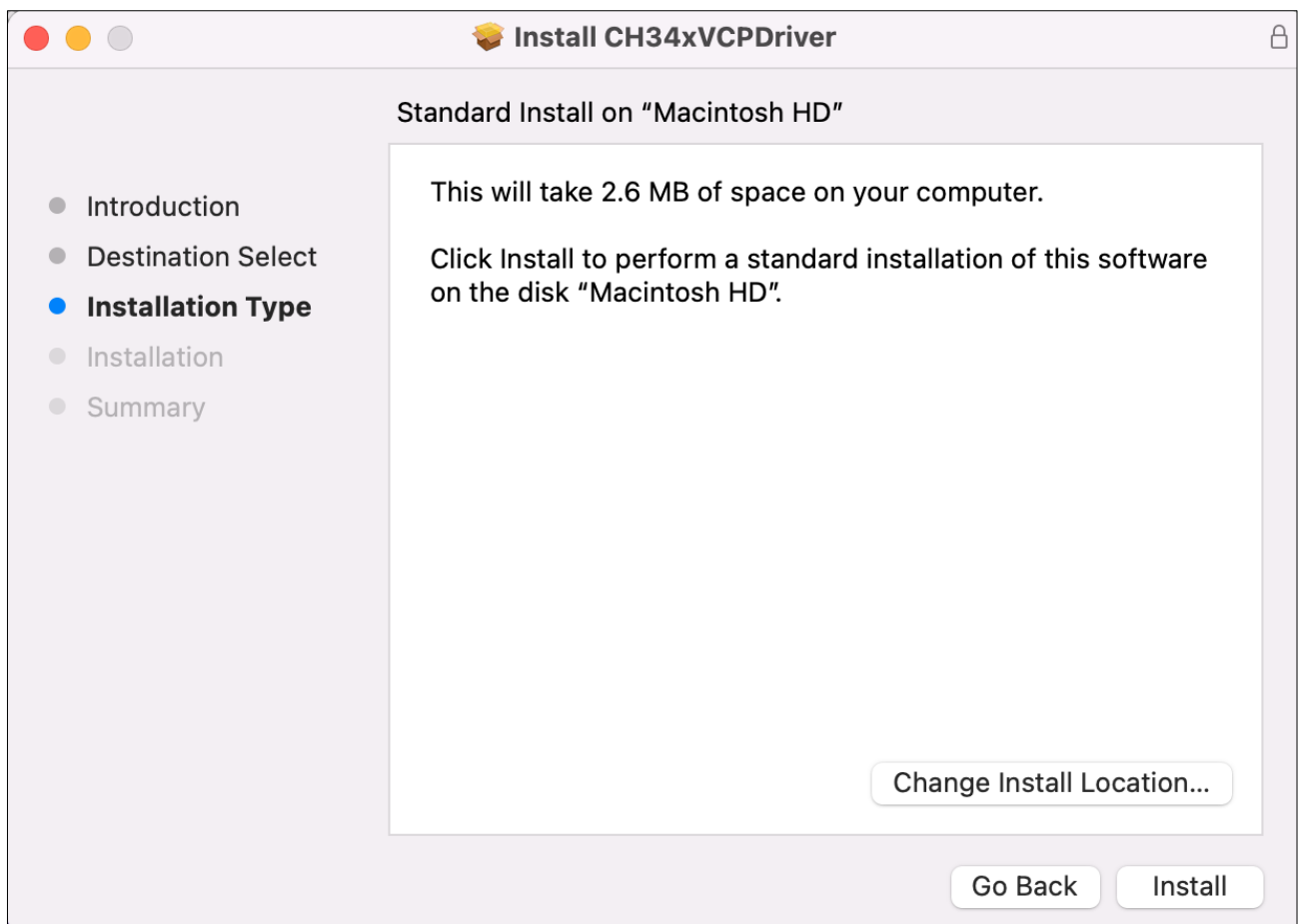
Second, open the folder “**Freenove_ESP32_S3_WROVER_Board/CH343/MAC/**”



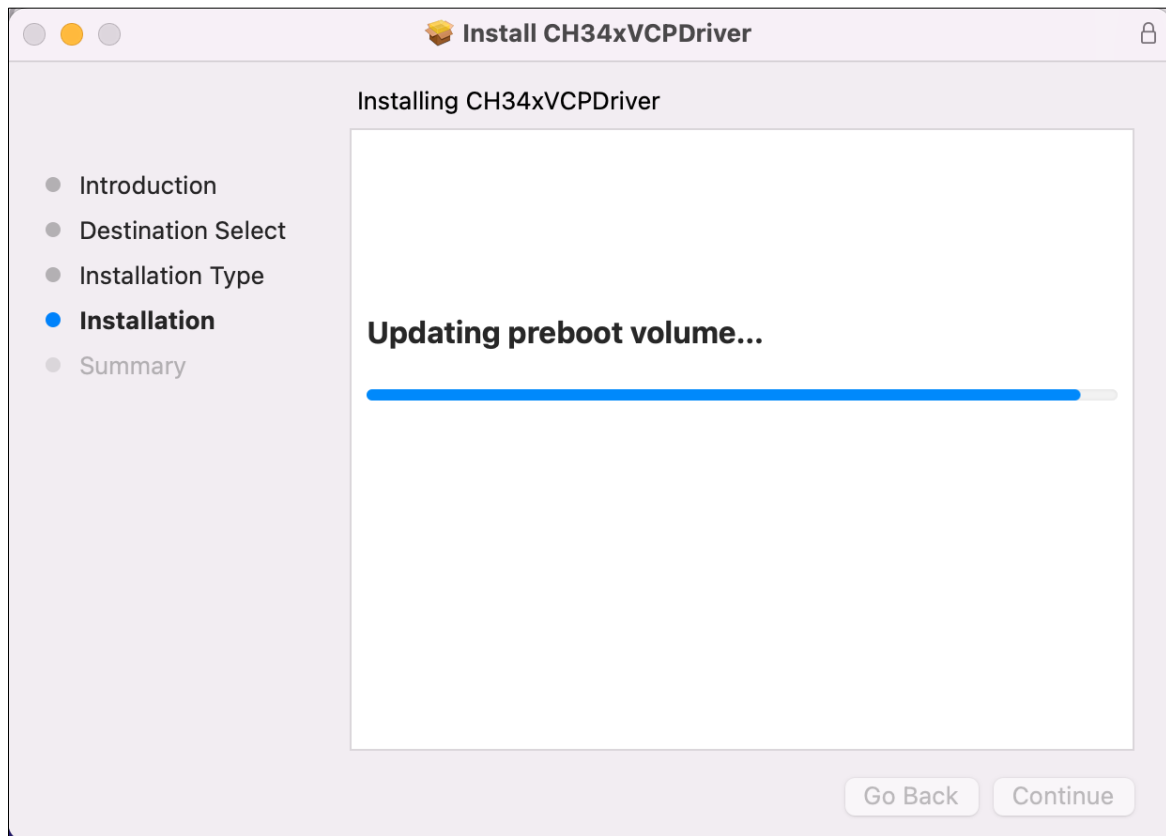
Third, click Continue.



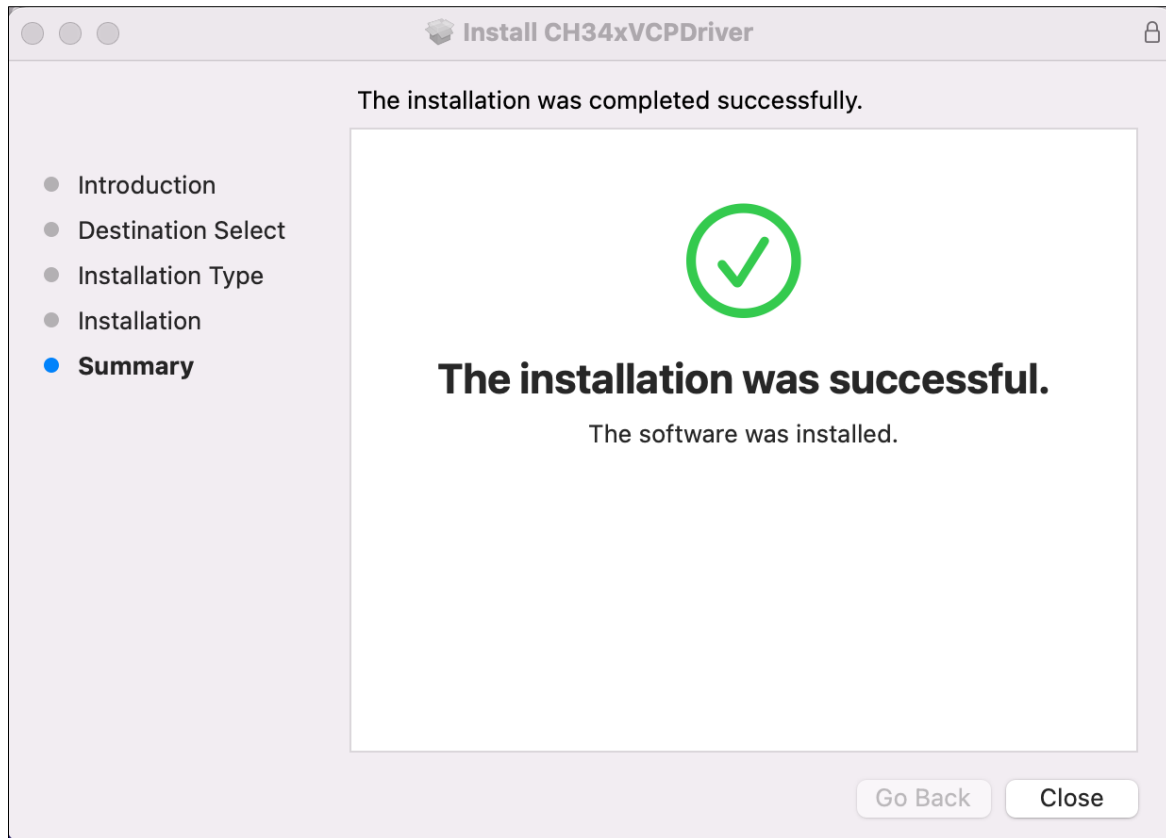
Fourth, click Install.



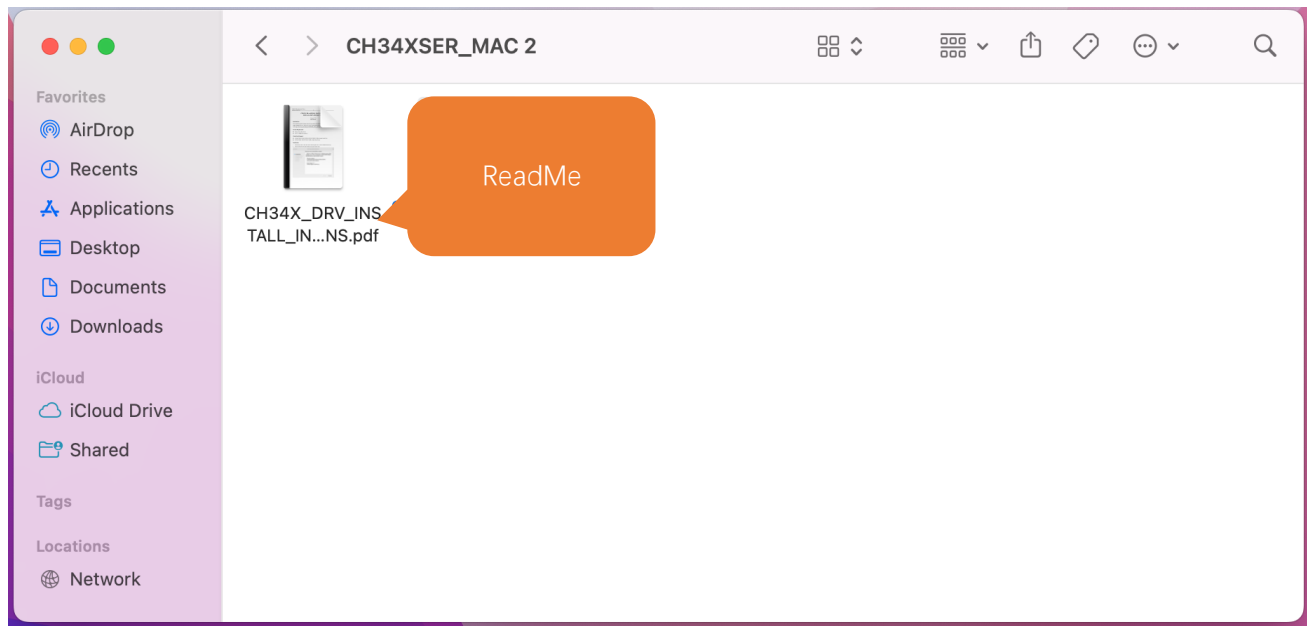
Then, waiting Finsh.



Finally, restart your PC.



If you still haven't installed the CH340 by following the steps above, you can view readme.pdf to install it.

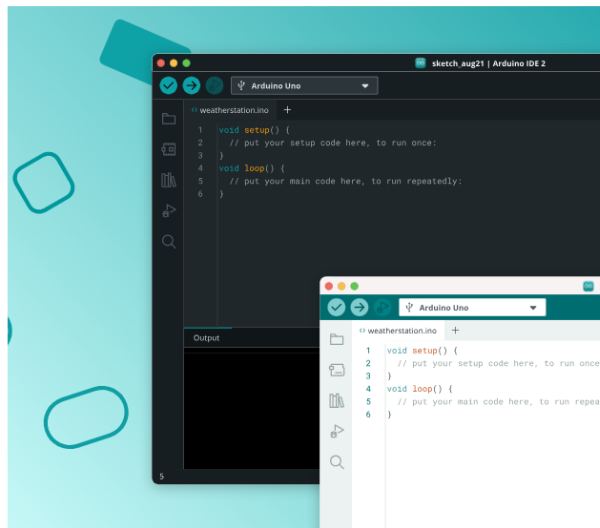


Programming Software

Arduino Software (IDE) is used to write and upload the code for Arduino Board.

First, install Arduino Software (IDE): visit <https://www.arduino.cc/en/software/>

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6 Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD



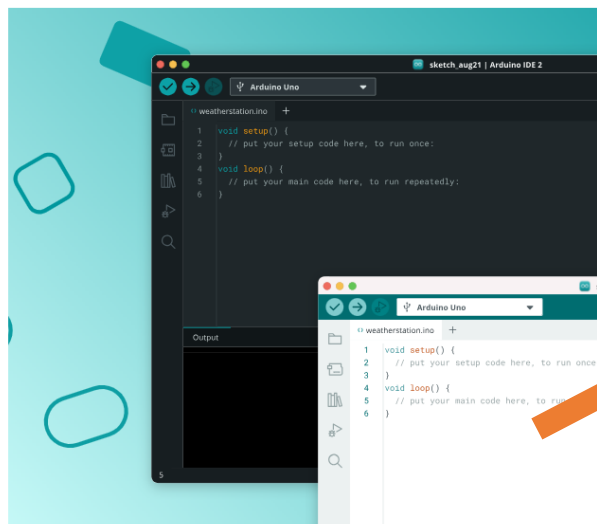
Nightly Builds

Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Select and download corresponding installer based on your operating system. If you are a Windows user, please select the "Windows" to download and install the driver correctly.

Bring Your Projects to Life with Arduino Software



Arduino IDE 2.3.6 Release notes

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit)

DOWNLOAD

Windows Win 10 or newer (64-bit)

Windows MSI installer

Windows ZIP file

Linux Appliance (64-bit X86-64)

Linux ZIP file (64-bit X86-64)

macOS Intel 10.15 Catalina or newer (64-bit)

macOS Apple Silicon 11 Big Sur or newer (64-bit)



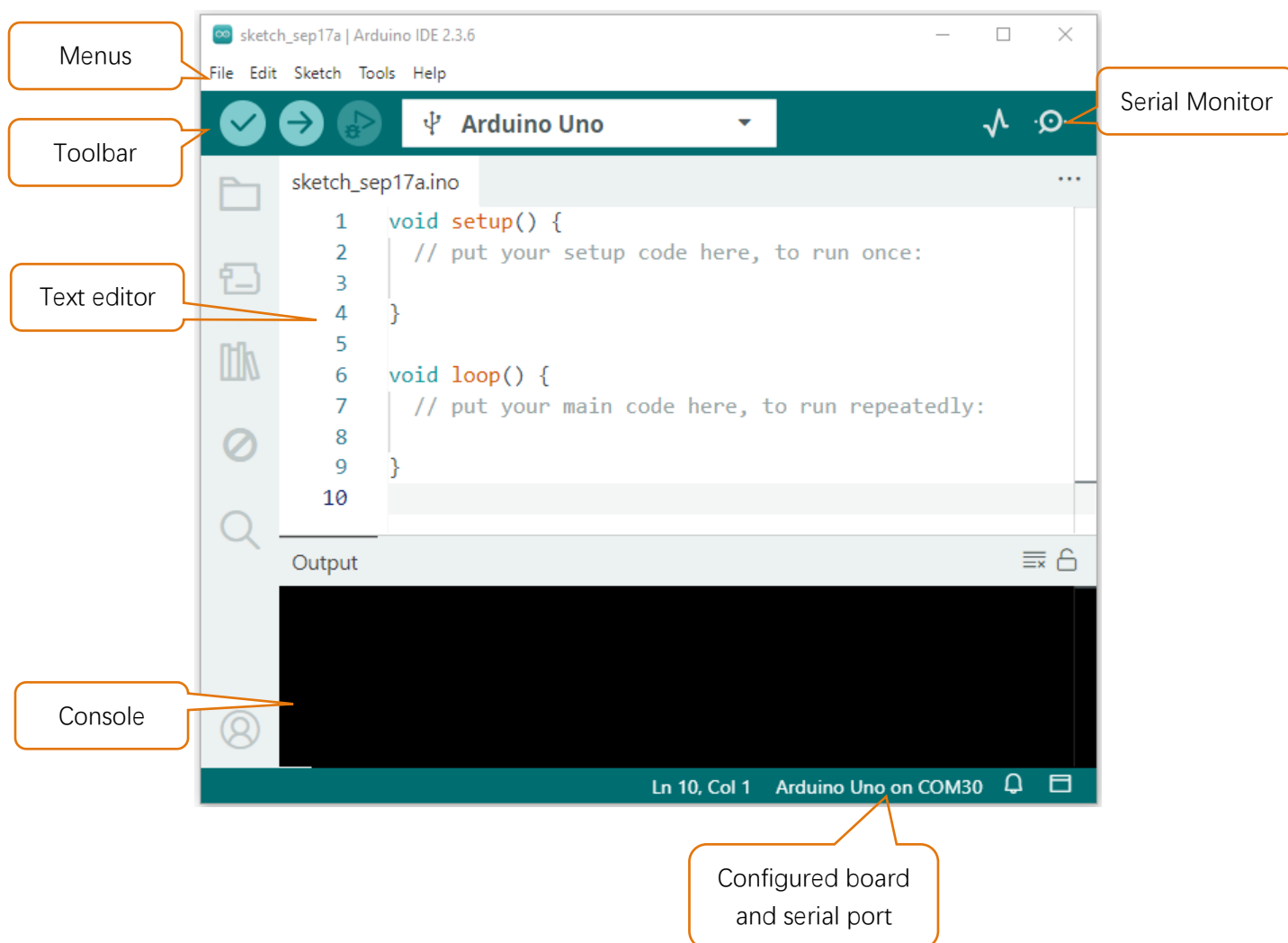
Legacy IDE (1.8.19)

Download a legacy version of the Arduino IDE.

After the downloading completes, run the installer. For Windows users, there may pop up an installation dialog box of driver during the installation process. When it is popped up, please allow the installation. After installation is completed, an shortcut will be generated in the desktop.



Run it. The interface of the software is as follows:



Programs written with Arduino IDE are called sketches. These sketches are written in a text editor and are saved with the file extension.ino. The editor has features for cutting/pasting and for searching/replacing text. The console displays text output by the Arduino IDE, including complete error messages and other information. The bottom right-hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, open the serial monitor, and access the serial plotter.

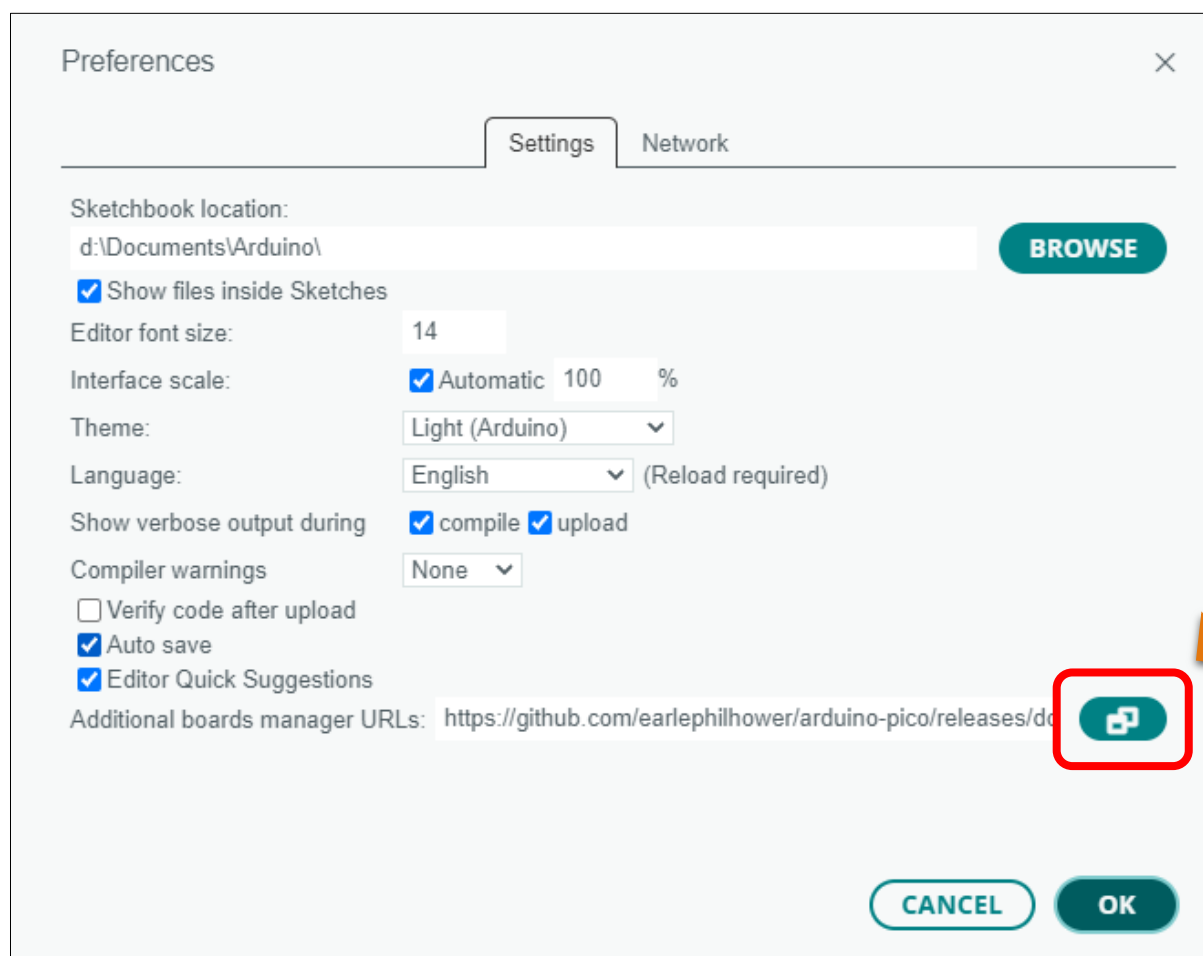
	Verify Checks your code for errors compiling it.
	Upload Compiles your code and uploads it to the configured board.
	Debug Troubleshoot code errors and monitor program running status.
	Serial Plotter Real-time plotting of serial port data charts.
	Serial Monitor Used for debugging and communication between devices and computers.

Environment Configuration

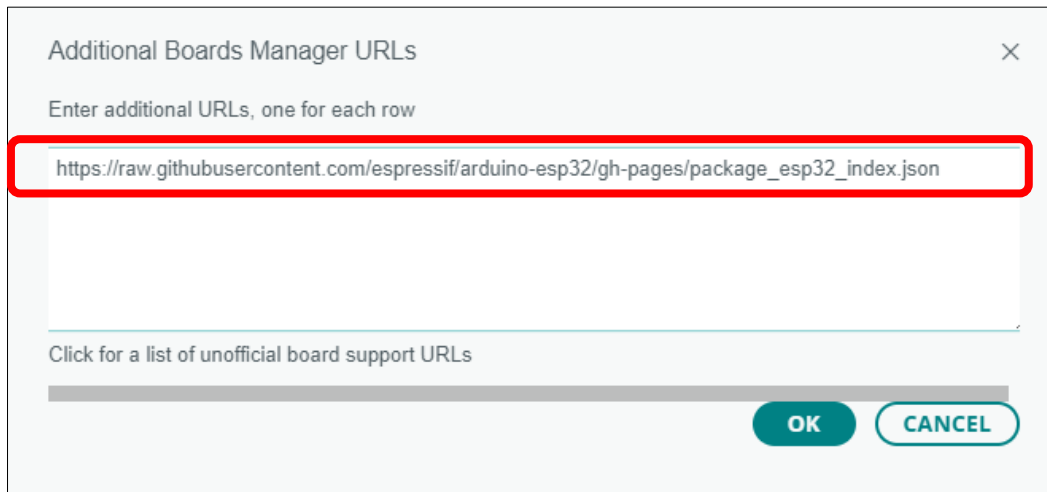
First, open the software platform arduino, and then click File in Menus and select Preferences.



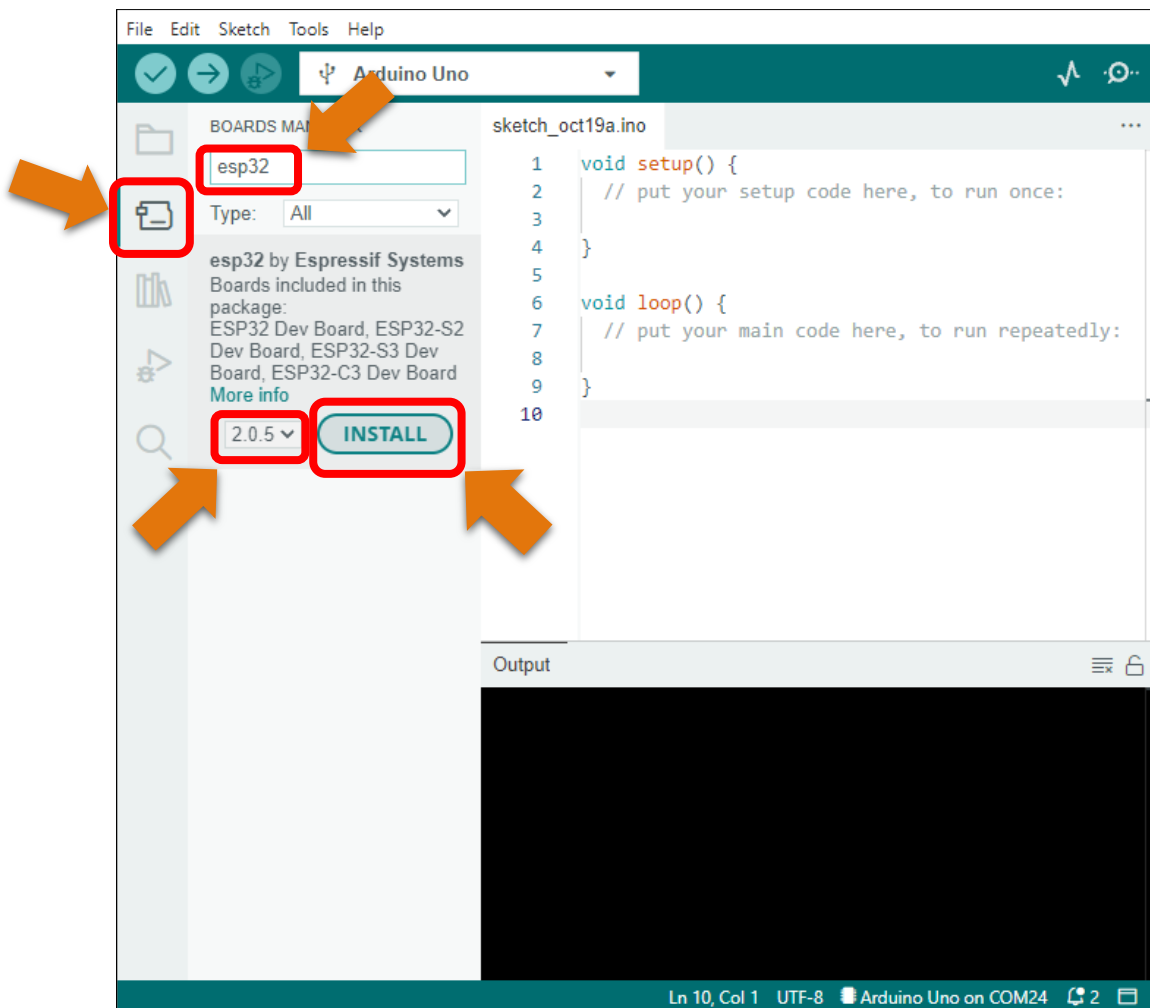
Second, click on the symbol behind "Additional Boards Manager URLs"



Third, fill in https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json in the new window, click OK, and click OK on the Preferences window again.



Fourth, click "Boards Manager". Enter "esp32" in Boards manager and select 2.0.5, Then click "INSTALL".

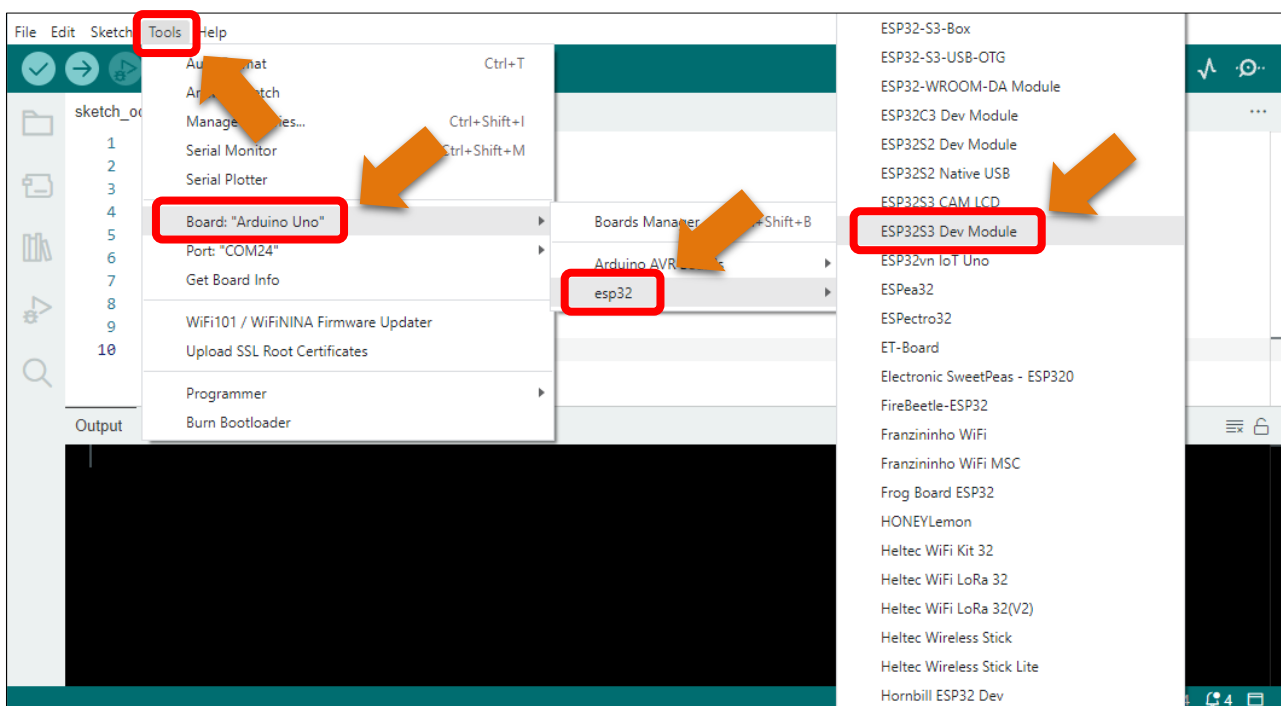


Arduinowill download these files automatically. Wait for the installation to complete.

```

Output
Downloading packages
esp32:riscv32-esp-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:xtensa-esp32-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:xtensa-esp32s2-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:xtensa-esp32s3-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:esptool_py@4.2.1
esp32:mkspliffs@0.2.3
esp32:mklittlefs@3.0.0-gnu12-dc7f933
esp32:esp32@2.0.5
Installing esp32:riscv32-esp-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:riscv32-esp-elf-gcc@gcc8_4_0-esp-2021r2-patch3 installed
Installing esp32:xtensa-esp32-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:xtensa-esp32-elf-gcc@gcc8_4_0-esp-2021r2-patch3 installed
Installing esp32:xtensa-esp32s2-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:xtensa-esp32s2-elf-gcc@gcc8_4_0-esp-2021r2-patch3 installed
Installing esp32:xtensa-esp32s3-elf-gcc@gcc8_4_0-esp-2021r2-patch3
esp32:xtensa-esp32s3-elf-gcc@gcc8_4_0-esp-2021r2-patch3 installed
Installing esp32:esptool_py@4.2.1
esp32:esptool_py@4.2.1 installed
Installing esp32:mkspliffs@0.2.3
esp32:mkspliffs@0.2.3 installed
Installing esp32:mklittlefs@3.0.0-gnu12-dc7f933
esp32:mklittlefs@3.0.0-gnu12-dc7f933 installed
Installing platform esp32:esp32@2.0.5
Configuring platform.
Platform esp32:esp32@2.0.5 installed
  
```

When finishing installation, click Tools in the Menus again and select Board: "Arduino Uno", and then you can see information of ESP32. click "ESP32-S3 Dev Module" so that the ESP32-S3 programming development environment is configured.



Notes for GPIO

Strapping Pin

There are four Strapping pins for ESP32-S3: GPIO0、GPIO45、GPIO46、GPIO3.

With the release of the chip's system reset (power-on reset, RTC watchdog reset, undervoltage reset), the strapping pins sample the level and store it in the latch as "0" or "1", and keep it until the chip is powered off or turned off.

Each Strapping pin is connecting to internal pull-up/pull-down. Connecting to high-impedance external circuit or without an external connection, a strapping pin's default value of input level will be determined by internal weak pull-up/pull-down. To change the value of the Strapping, users can apply an external pull-down/pull-up resistor, or use the GPIO of the host MCU to control the level of the strapping pin when the ESP32-S3's power on reset is released.

When releasing the reset, the strapping pin has the same function as a normal pin.

The followings are default configurations of these four strapping pins at power-on and their functions under the corresponding configuration.

VDD_SPI Voltage			
Pin	Default	3.3 V	1.8 V
GPIO45	Pull-down	0	1
Bootling Mode ¹			
Pin	Default	SPI Boot	Download Boot
GPIO0	Pull-up	1	0
GPIO46	Pull-down	Don't care	0
Enabling/Disabling ROM Messages Print During Bootling ²			
Pin	Default	Enabled	Disabled
GPIO46	Pull-down	See the 2nd note	See the 2nd note
JTAG Signal Selection			
Pin	Default	EFUSE_DIS_USB_JTAG = 0, EFUSE_DIS_PAD_JTAG = 0, EFUSE_STRAP_JTAG_SEL=1	
GPIO3	N/A	0: JTAG signal from on-chip JTAG pins 1: JTAG signal from USB Serial/JTAG controller	

Note:

1. The strapping combination of GPIO46 = 1 and GPIO0 = 0 is invalid and will trigger unexpected behavior.
2. By default, the ROM boot messages are printed over UART0 (U0TXD pin) and USB Serial/JTAG controller together. The ROM code printing can be disabled through configuration register and eFuse. For detailed information, please refer to Chapter [Chip Boot Control](#) in *ESP32-S3 Technical Reference Manual*.

If you have any difficulties or questions with this tutorial or toolkit, feel free to ask for our quick and free technical support through support@freenove.com at any time.

or check: https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf

PSRAM Pin

The module on the ESP32-S3-WROOM board uses the ESP32-S3R8 chip with 8MB of external Flash. When we use the OPI PSRAM, please note that the GPIO35-GPIO37 on the ESP32-S3-WROOM board will not be available for other purposes. When OPI PSRAM is not used, GPIO35-GPIO37 on the board can be used as normal GPIO.

ESP32-S3R8 / ESP32-S3R8V	In-package PSRAM (8 MB, Octal SPI)
SPICLK	CLK
SPICS1	CE#
SPID	DQ0
SPIQ	DQ1
SPIWP	DQ2
SPIHD	DQ3
GPIO33	DQ4
GPIO34	DQ5
GPIO35	DQ6
GPIO36	DQ7
GPIO37	DQS/DM

SDcard Pin

An SDcard slot is integrated on the back of the ESP32-S3-WROOM board. We can use GPIO38-GPIO40 of ESP32-S3-WROOM to drive SD card.

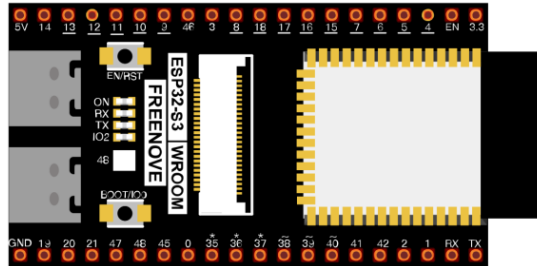
The SDcard of ESP32-S3-WROOM uses SDMMC, a 1-bit bus driving method, which has been integrated in the Arduino IDE, and we can call the "SD_MMC.h" library to drive it. For details, see the SDcard chapter in this tutorial.

USB Pin

In Micropython, GPIO19 and GPIO20 are used for the USB function of ESP32S3, so they cannot be used as other functions!

Cam Pin

When using the camera of our ESP32-S3 WROOM, please check the pins of it. Pins with underlined numbers are used by the camera function, if you want to use other functions besides it, please avoid using them.



CAM_Pin	GPIO_pin
<u>SIOD</u>	GPIO4
<u>SIOC</u>	GPIO5
<u>CSI_VYSNC</u>	GPIO6
<u>CSI_HREF</u>	GPIO7
<u>CSI_Y9</u>	GPIO16
<u>XCLK</u>	GPIO15
<u>CSI_Y8</u>	GPIO17
<u>CSI_Y7</u>	GPIO18
<u>CSI_PCLK</u>	GPIO13
<u>CSI_Y6</u>	GPIO12
<u>CSI_Y2</u>	GPIO11
<u>CSI_Y5</u>	GPIO10
<u>CSI_Y3</u>	GPIO9
<u>CSI_Y4</u>	GPIO8

If you have any questions about the information of GPIO, you can click [here](#) to go back to ESP32-S3 WROOM to view specific information about GPIO.

or check: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.

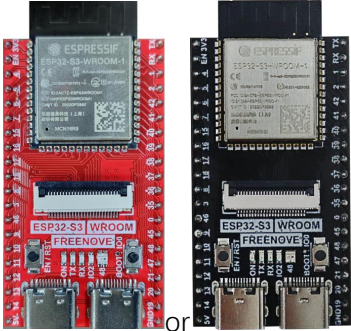
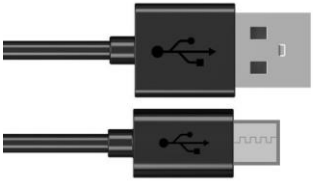
Chapter 1 LED

This chapter is the Start Point in the journey to build and explore ESP32-S3 WROOM electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

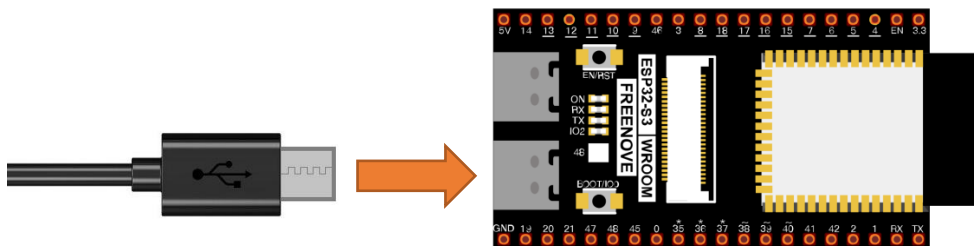
In this project, we will use ESP32-S3 WROOM to control blinking a common LED.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
--	---

Power

ESP32-S3 WROOM needs 5v power supply. In this tutorial, we need connect ESP32-S3 WROOM to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



In the following projects, we only use USB cable to power ESP32-S3 WROOM by default.

In the whole tutorial, we don't use T extension to power ESP32-S3 WROOM. So 5V and 3.3V (includeing EXT 3.3V) on the extension board are provided by ESP32-S3 WROOM.

We can also use DC jack of extension board to power ESP32-S3 WROOM. In this way, 5v and EXT 3.3v on extension board are provided by external power resource.

Sketch

According to the circuit, when the GPIO2 of ESP32-S3 WROOM output level is high, the LED turns ON. Conversely, when the GPIO2 ESP32-S3 WROOM output level is low, the LED turns OFF. Therefore, we can let GPIO2 circularly output high and low level to make the LED blink.

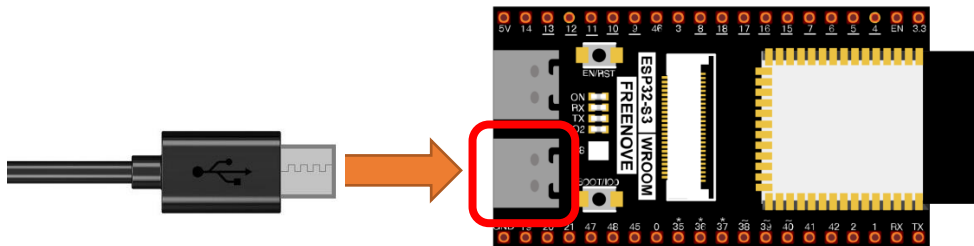
Upload the following Sketch:

Freenove_ESP32_S3_WROVER_Board\Sketches\Sketch_01.1_Blink.

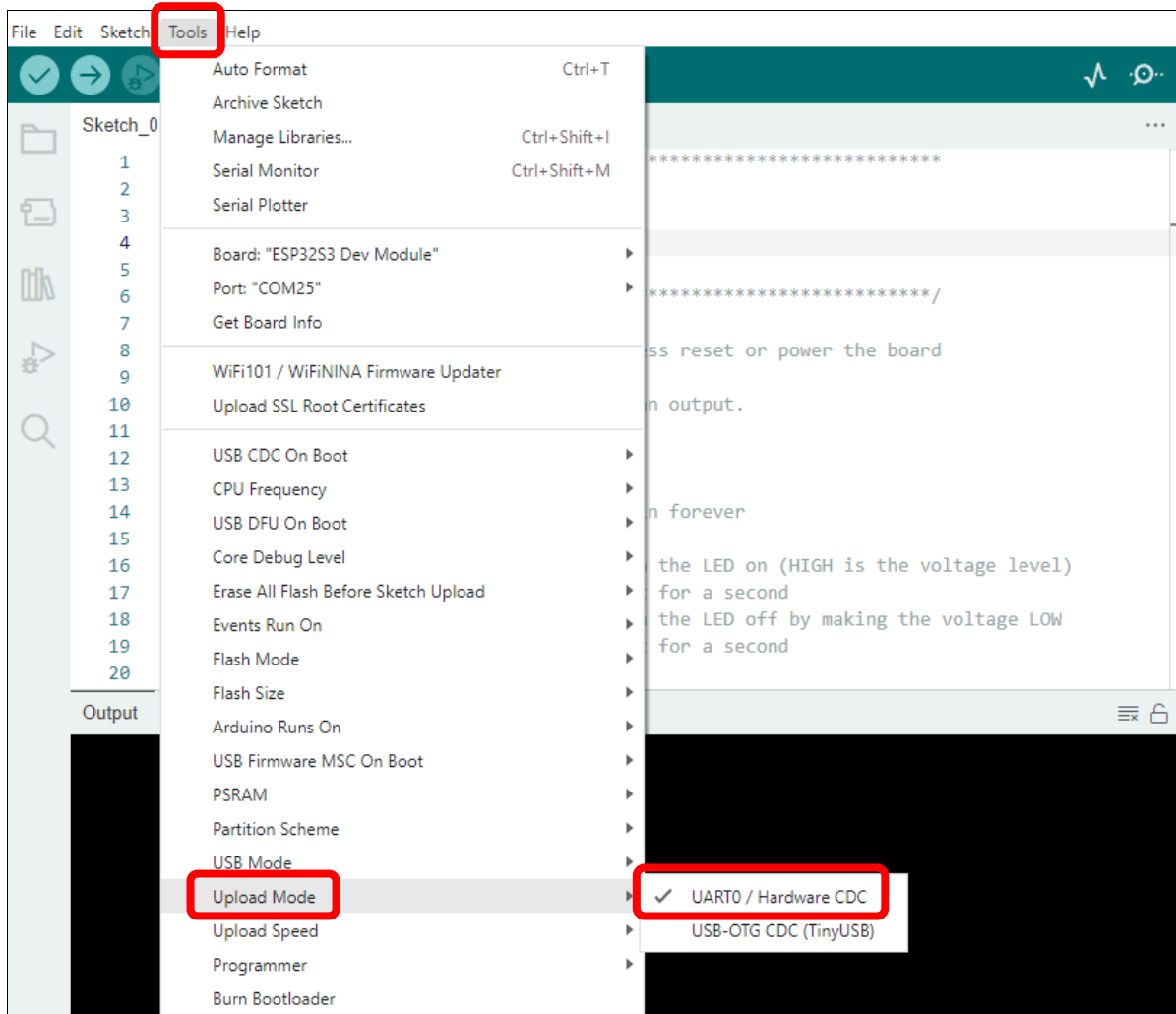
Next we will introduce two ways to upload code to ESP32-S3 WROOM.

Option 1:

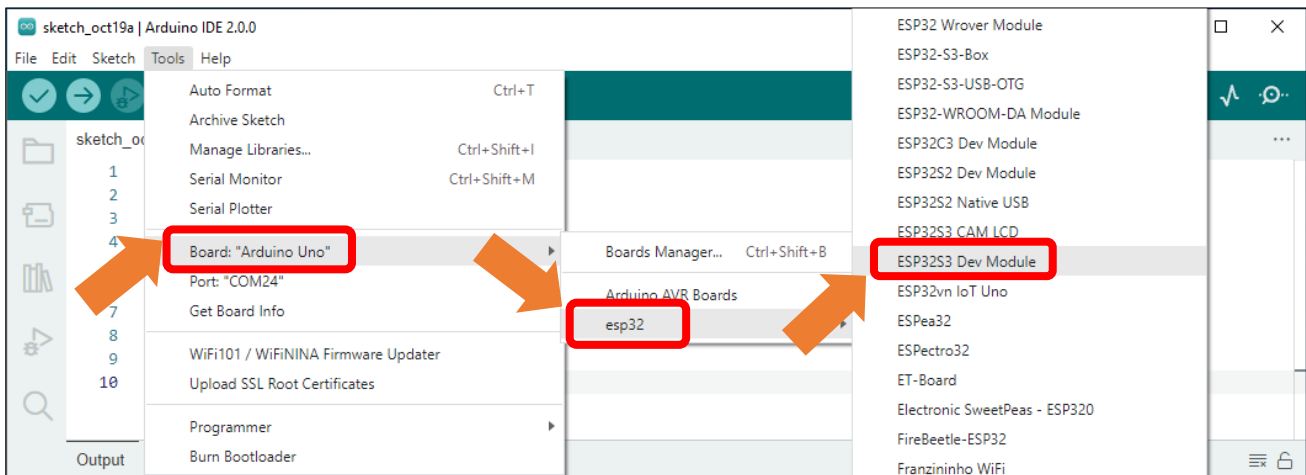
Connect ESP32-S3 WROOM to computer.



Open Arduino IDE 2.0.0. Click Tools->Upload Mode. Select UART0 / Hardware CDC.

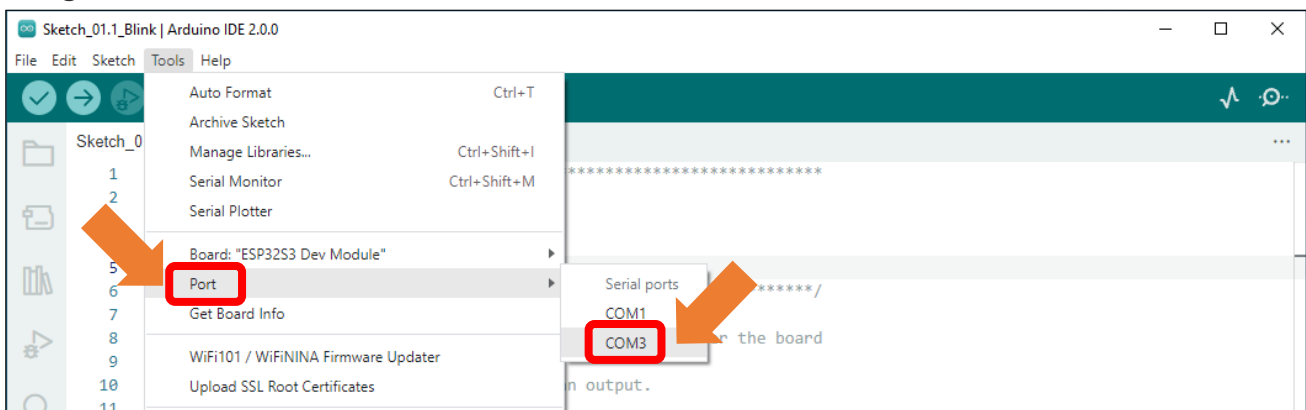


Before uploading the code, click "Tools", "Board" and select "ESP32S3 Dev Module".

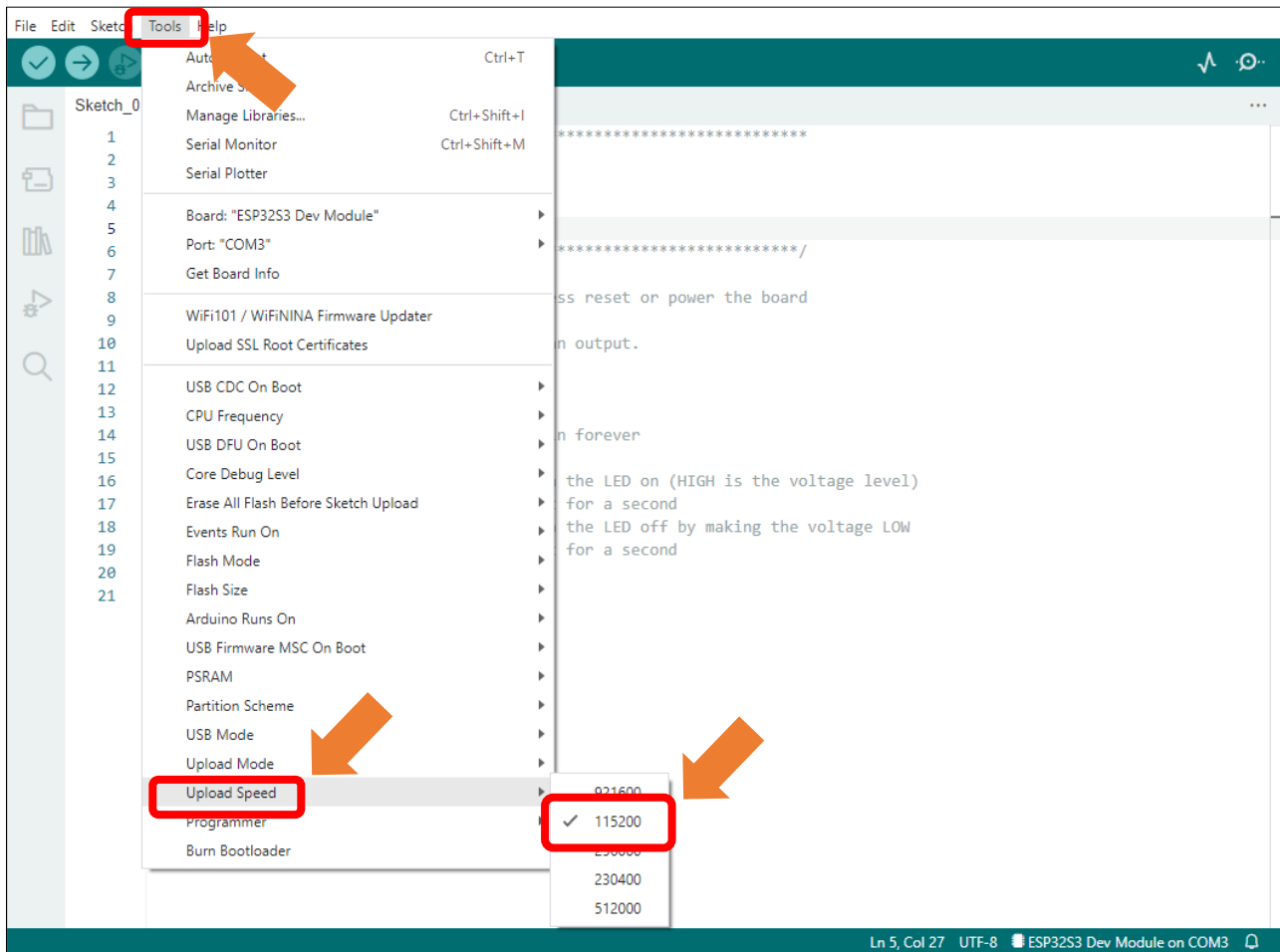


Select the serial port.

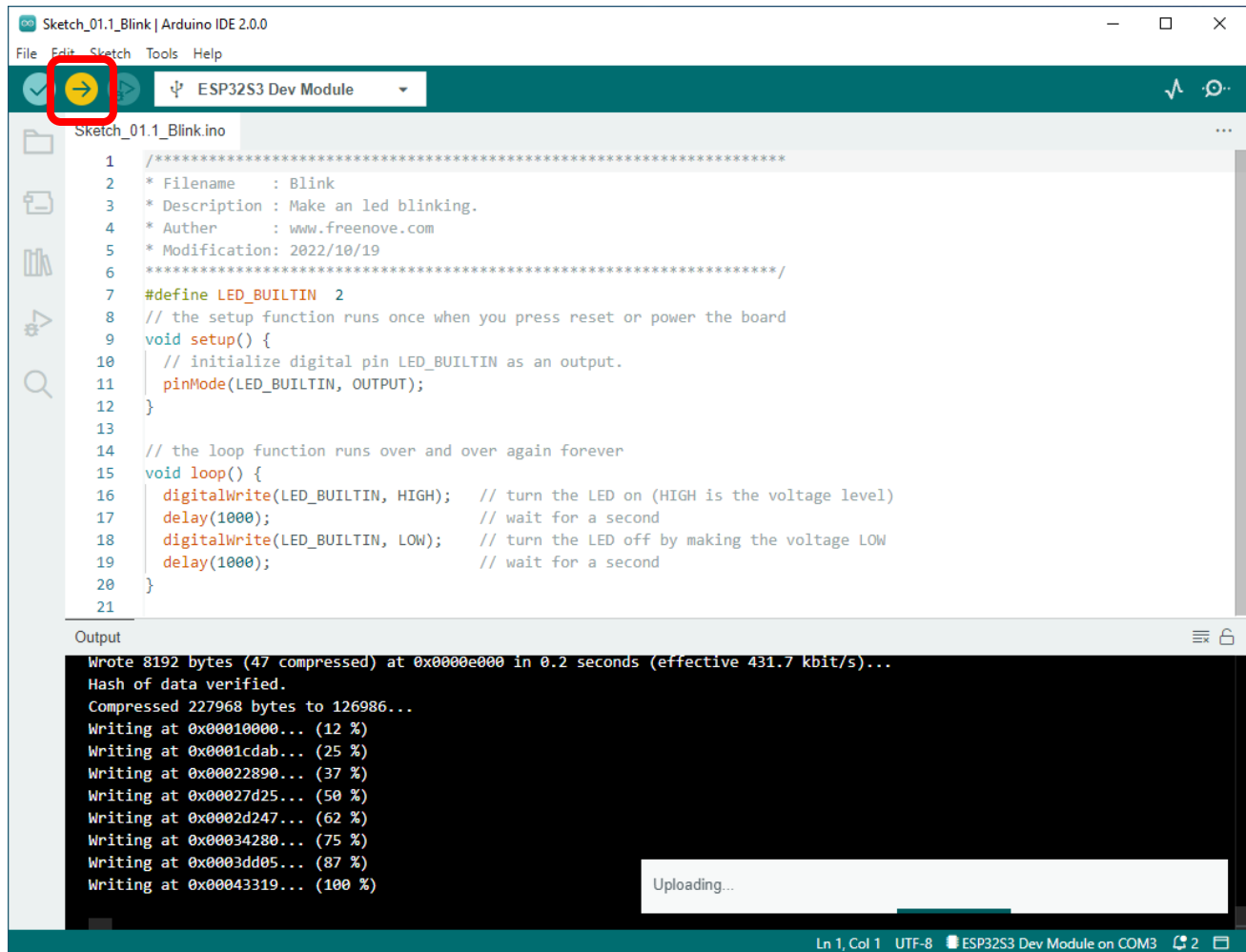
Note that the computer port number of each user may be different. Please select the correct serial port according to your computer. Taking the window system as an example, my computer recognizes that the communication interface of the ESP32-S3-WROOM is COM3, so I select COM3.



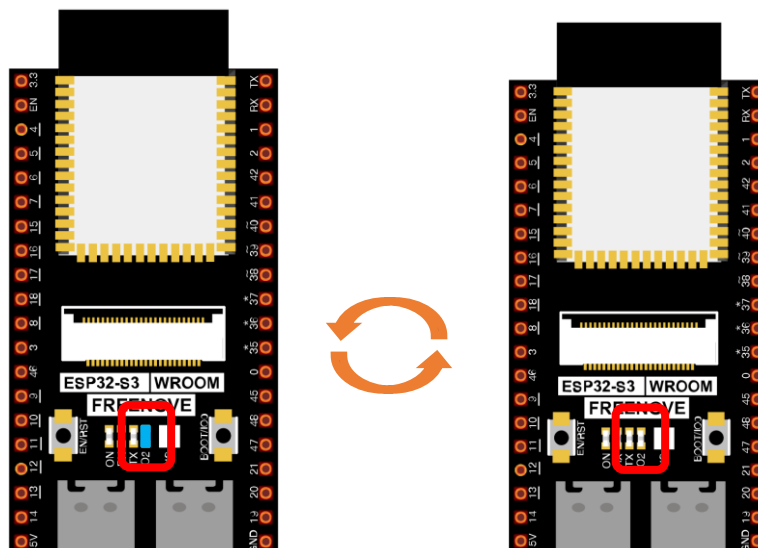
Note: For macOS users, if the uploading fails, please set the baud rate to 115200 before clicking “Upload Using Programmer”.



Click the Upload button and it will compile and upload the Sketch to the ESP32-S3-WROOM.



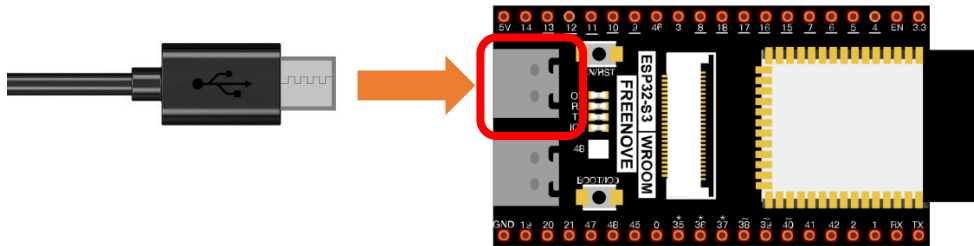
Wait for the Sketch upload to complete, and observe the ESP32-S3-WROOM. You can see that the blue LED (IO2) on the board flashes cyclically.



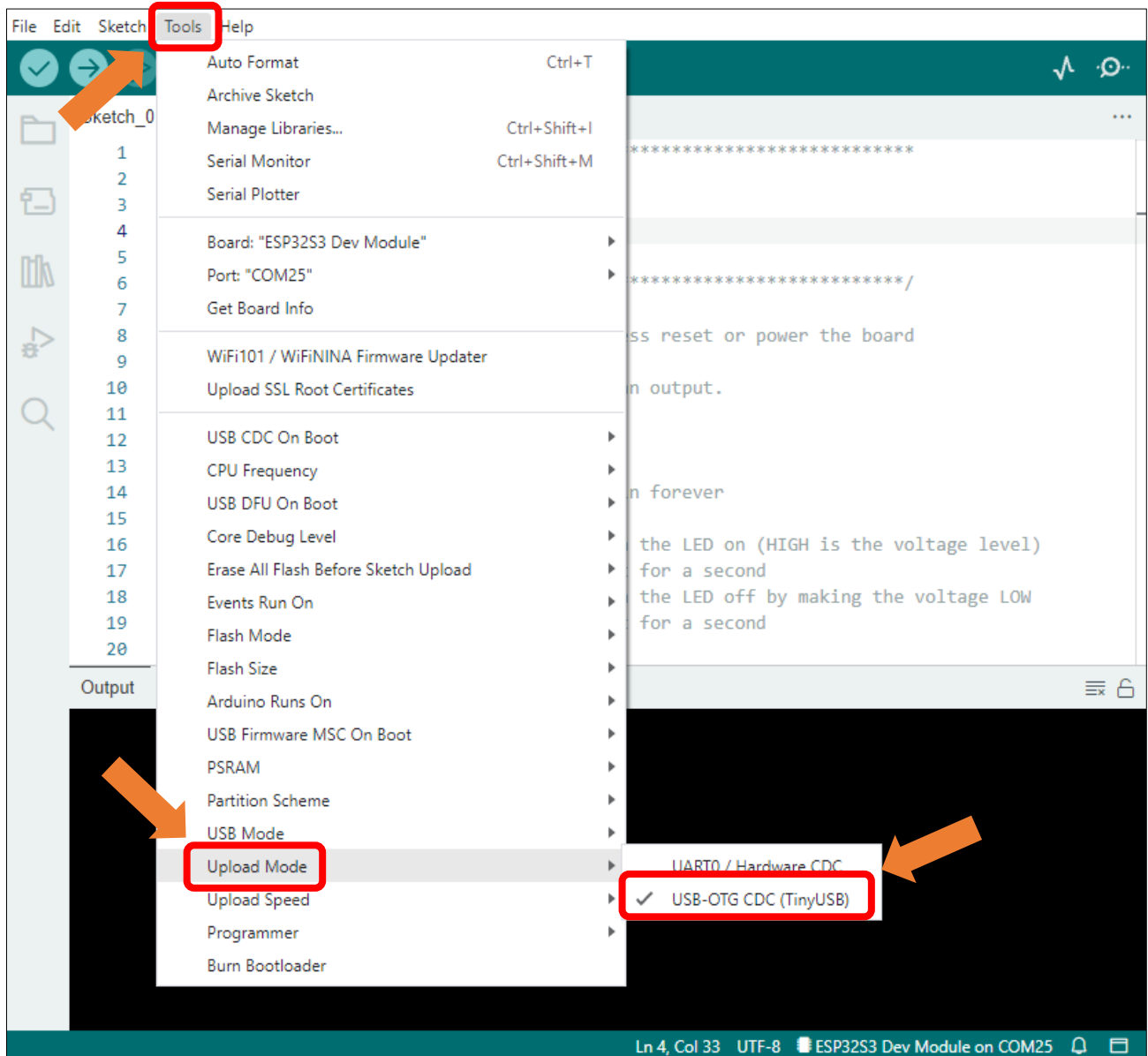
If you have any concerns, please contact us via: support@freenove.com.

Option 2:

Connect ESP32-S3 WROOM to computer.

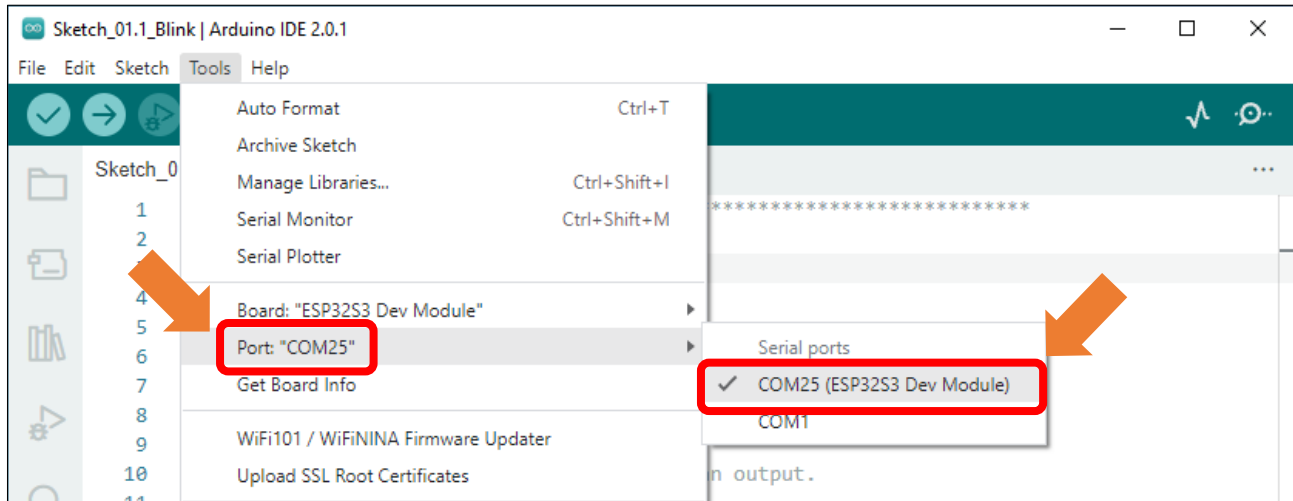


Open Arduino IDE 2.0.0. Click Tools->Upload Mode. Select USB-OTG CDC(TinyUSB).

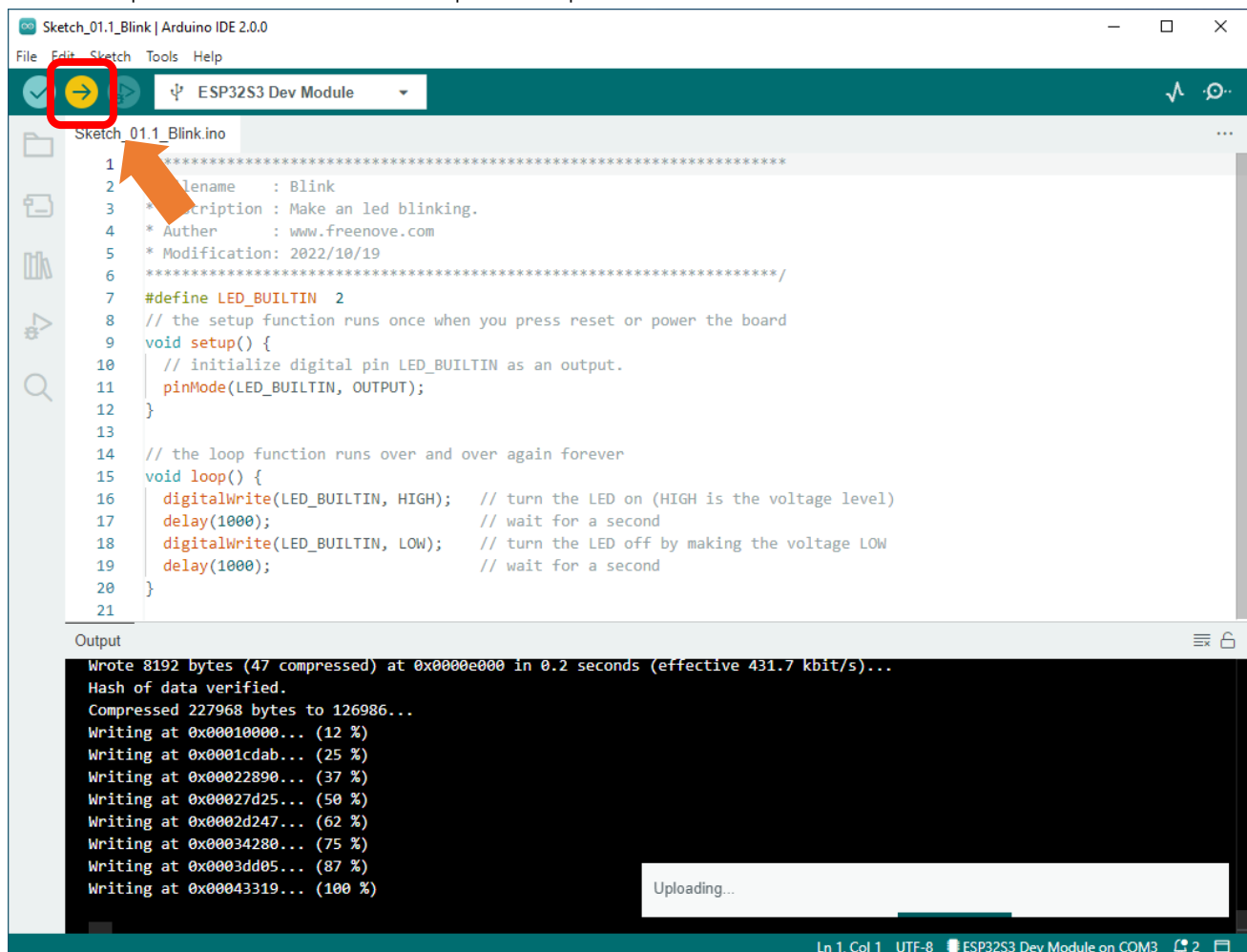


Select the serial port.

Note that the computer port number of each user may be different. Please select the correct serial port according to your computer. Taking the window system as an example, my computer recognizes that the communication interface of the ESP32-S3-WROOM is COM25, so I select COM25.

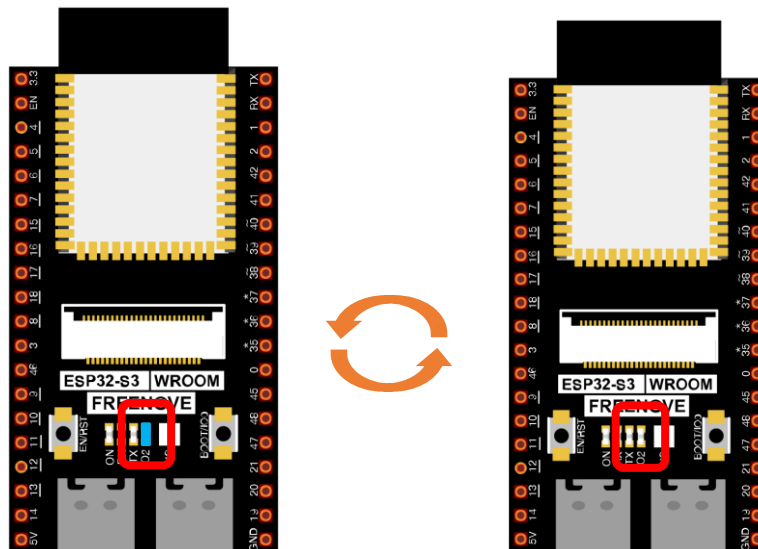


Click the Upload button and it will compile and upload the Sketch to the ESP32-S3-WROOM.



Wait for the Sketch upload to complete, and observe the ESP32-S3-WROOM. You can see that the blue

LED (IO2) on the board flashes cyclically.



Sketch_01.1 Blink

The following is the program code:

```

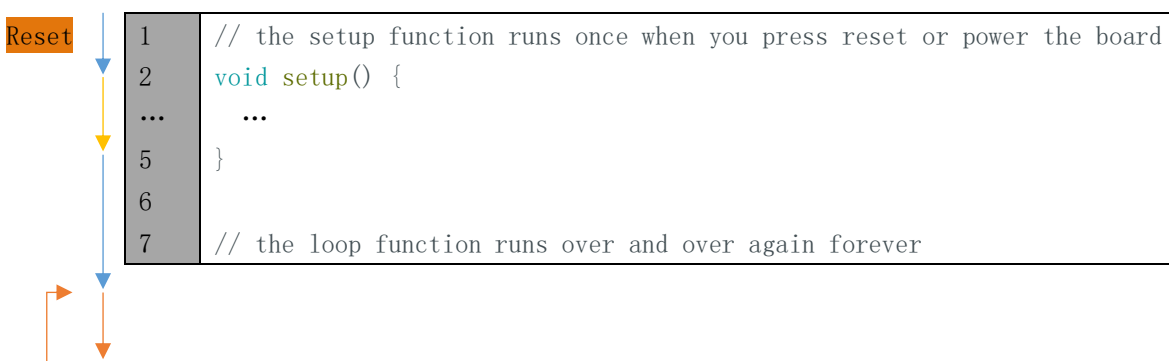
1  #define LED_BUILTIN 2
2  // the setup function runs once when you press reset or power the board
3  void setup() {
4      // initialize digital pin LED_BUILTIN as an output.
5      pinMode(LED_BUILTIN, OUTPUT);
6  }
7
8  // the loop function runs over and over again forever
9  void loop() {
10     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
11     delay(1000);                      // wait for a second
12     digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
13     delay(1000);                      // wait for a second
14 }

```

The Arduino IDE code usually contains two basic functions: void setup() and void loop().

After the board is reset, the setup() function will be executed firstly, and then the loop() function.

setup() function is generally used to write code to initialize the hardware. And loop() function is used to write code to achieve certain functions. loop() function is executed repeatedly. When the execution reaches the end of loop(), it will jump to the beginning of loop() to run again.



```

8   void loop() {
...       ...
13  }
```

Reset

Reset operation will lead the code to be executed from the beginning. Switching on the power, finishing uploading the code and pressing the reset button will trigger reset operation.

In the circuit, ESP32-S3 WROOM's GPIO2 is connected to the LED, so the LED pin is defined as 2.

```
1   #define LED_BUILTIN 2
```

This means that after this line of code, all LED_BUILTIN will be treated as 2.

In the setup () function, first, we set the LED_BUILTIN as output mode, which can make the port output high level or low level.

```

4   // initialize digital pin LED_BUILTIN as an output.
5   pinMode(LED_BUILTIN, OUTPUT);
```

Then, in the loop () function, set the LED_BUILTIN to output high level to make LED light up.

```
10  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
```

Wait for 1000ms, that is 1s. Delay () function is used to make control board wait for a moment before executing the next statement. The parameter indicates the number of milliseconds to wait for.

```
11  delay(1000); // wait for a second
```

Then set the LED_BUILTIN to output low level, and LED light off. One second later, the execution of loop () function will be completed.

```

12  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
13  delay(1000); // wait for a second
```

The loop() function is constantly being executed, so LED will keep blinking.

Reference

```
void pinMode(int pin, int mode);
```

Configures the specified pin to behave either as an input or an output.

Parameters

pin: the pin number to set the mode of.

mode: INPUT, OUTPUT, INPUT_PULLDOWN, or INPUT_PULLUP.

```
void digitalWrite (int pin, int value);
```

Writes the value HIGH or LOW (1 or 0) to the given pin which must have been previously set as an output.

For more related functions, please refer to <https://www.arduino.cc/reference/en/>

Chapter 2 Serial Communication

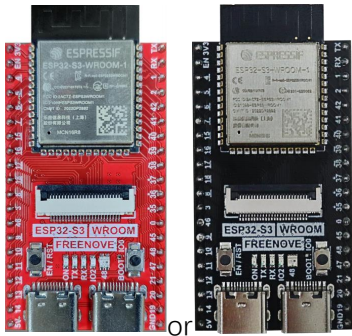
Serial Communication is a means of communication between different devices/devices. This section describes ESP32-S3's Serial Communication.

Project 2.1 Serial Print

This project uses ESP32-S3's serial communicator to send data to the computer and print it on the serial monitor.

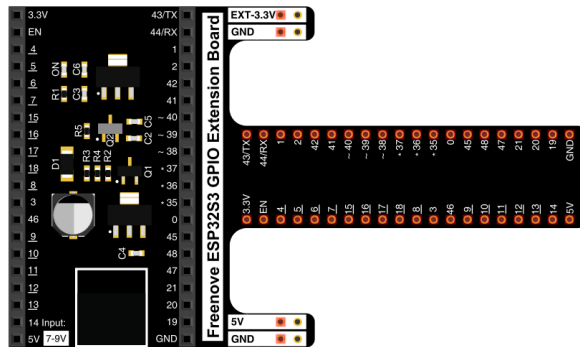
Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8)

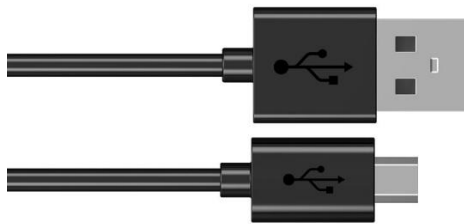


or

GPIO Extension Board x1



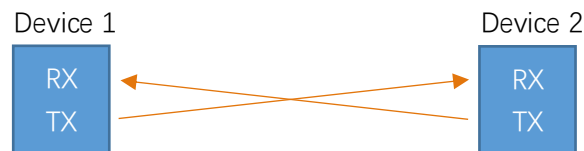
Micro USB Wire x1



Related knowledge

Serial communication

Serial communication generally refers to the Universal Asynchronous Receiver/Transmitter (UART), which is commonly used in electronic circuit communication. It has two communication lines, one is responsible for sending data (TX line) and the other for receiving data (RX line). The serial communication connections of two devices is as follows:



Before serial communication starts, the baud rate of both sides must be the same. Communication between devices can work only if the same baud rate is used. The baud rates commonly used is 9600 and 115200.

Serial port on ESP32-S3

Freenove ESP32-S3 has integrated USB to serial transfer, so it could communicate with computer connecting to USB cable.

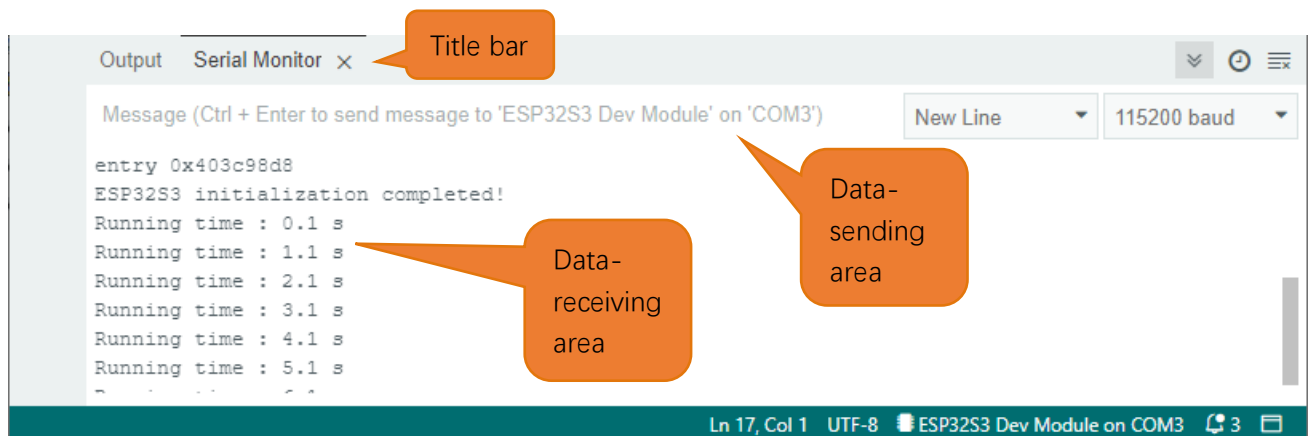


Arduino Software also uploads code to Freenove ESP32-S3 through the serial connection.

Your computer identifies serial devices connecting to it as COMx. We can use the Serial Monitor window of Arduino Software to communicate with Freenove ESP32-S3, connect Freenove ESP32-S3 to computer through the USB cable, choose the correct device, and then click the Serial Monitor icon to open the Serial Monitor window.

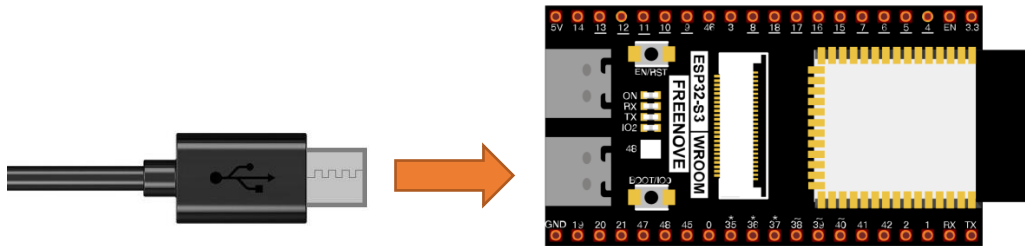


Interface of serial monitor window is as follows. If you can't open it, make sure Freenove ESP32-S3 has been connected to the computer, and choose the right serial port in the menu bar "Tools".



Circuit

Connect Freenove ESP32-S3 to the computer with USB cable.



Sketch

Sketch_02.1_SerialPrinter

```
File Edit Sketch Tools Help
ESP32S3 Dev Module

Sketch_08.1_SerialPrinter.ino
1  /*****
2   Filename   : SerialPrinter
3   Description : Use UART send some data to PC, and show them on serial monitor.
4   Author     : www.freenove.com
5   Modification: 2022/10/20
6   *****/
7
8  void setup() {
9      Serial.begin(115200);
10     Serial.println("ESP32S3 initialization completed!");
11 }
12
13 void loop() {
14     Serial.printf("Running time : %.1f s\n", millis() / 1000.0f);
15     delay(1000);
16 }
```

Download the code to ESP32-S3 WROOM, open the serial port monitor, set the baud rate to 115200, and press the reset button. As shown in the following figure:

```
Output Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3') New Line 115200 baud

load:0x403cc700,len:0x2a3c
entry 0x403c98d8
ESP32S3 initialization completed!
Running time : 0.1 s
Running time : 1.1 s
Running time : 2.1 s
Running time : 3.1 s
Running time : 4.1 s
Ln 17, Col 1 UTF-8 ESP32S3 Dev Module on COM3 3
```

As shown in the image above, "ESP32-S3 initialization completed! " The previous is the printing message

when the system is started. The user program is then printed at a baud rate of 115200.

The following is the program code:

```

1  void setup() {
2      Serial.begin(115200);
3      Serial.println("ESP32S3 initialization completed! ");
4  }
5
6  void loop() {
7      Serial.printf("Running time : %.1f s\n", millis() / 1000.0f);
8      delay(1000);
9  }

```

Reference

```
void begin(unsigned long baud, uint32_t config=SERIAL_8N1, int8_t rxPin=-1,
           int8_t txPin=-1, bool invert=false, unsigned long timeout_ms = 20000UL);
```

Initializes the serial port. Parameter baud is baud rate, other parameters generally use the default value.

```
size_t println( arg );
```

Print to the serial port and wrap. The parameter **arg** can be a number, a character, a string, an array of characters, etc.

```
size_t printf(const char * format, ...) __attribute__((format (printf, 2, 3)));
```

Print formatted content to the serial port in the same way as print in standard C.

```
unsigned long millis();
```

Returns the number of milliseconds since the current system was booted.

Project 2.2 Serial Read and Write

From last section, we use serial port on Freenove ESP32-S3 to send data to a computer, now we will use that to receive data from computer.

Component and circuit are the same as in the previous project.

Sketch

Sketch_02.2_SerialRW

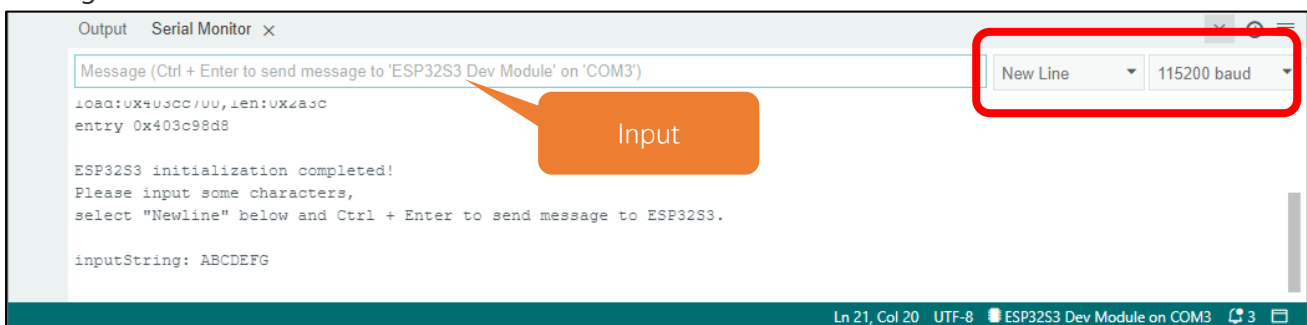


```
File Edit Sketch Tools Help
ESP32S3 Dev Module

Sketch_02.2_SerialRW.ino
7 String inputString = ""; //a String to hold incoming data
8 bool stringComplete = false; // whether the string is complete
9
10 void setup() {
11   Serial.begin(115200);
12   Serial.println(String("\nESP32S3 initialization completed!\r\n"));
13   + String("Please input some characters,\r\n")
14   + String("select \"Newline\" below and Ctrl + Enter to send message to ESP32S3. \r\n"));
15 }
16
17 void loop() {
18   if (Serial.available()) { // judge whether data has been received
19     char inChar = Serial.read(); // read one character
20     inputString += inChar;
21     if (inChar == '\n') {
22       stringComplete = true;
23     }
24   }
25   if (stringComplete) {
26     Serial.printf("inputString: %s \r\n", inputString);
27     inputString = "";
28     stringComplete = false;
29   }
30 }
```

Download the code to ESP32-S3 WROOM, open the serial monitor, and set the top right corner to **Newline**, **115200**. As shown in the following figure:

Then type characters like 'ABCDEFGH' into the data sent at the top, and press Ctrl+Enter to send the message.



The following is the program code:

```

1  String inputString = "";      //a String to hold incoming data
2  bool stringComplete = false; // whether the string is complete
3
4  void setup() {
5      Serial.begin(115200);
6      Serial.println(String("\nESP32S3 initialization completed! \r\n")
7          + String("Please input some characters, \r\n")
8          + String("select \"Newline\" below and Ctrl + Enter to send message to
ESP32S3. \r\n"));
9  }
10
11 void loop() {
12     if (Serial.available()) {      // judge whether data has been received
13         char inChar = Serial.read(); // read one character
14         inputString += inChar;
15         if (inChar == '\n') {
16             stringComplete = true;
17         }
18     }
19     if (stringComplete) {
20         Serial.printf("inputString: %s \n", inputString);
21         inputString = "";
22         stringComplete = false;
23     }
24 }

```

In loop(), determine whether the serial port has data, if so, read and save the data, and if the newline character is read, print out all the data that has been read.

Reference

String();

Constructs an instance of the String class.

For more information, please visit

<https://www.arduino.cc/reference/en/language/variables/data-types/stringobject/>

int available(void);

Get the number of bytes (characters) available for reading from the serial port. This is data that's already arrived and stored in the serial receive buffer.

Serial.read();

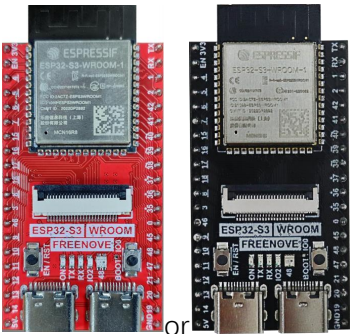
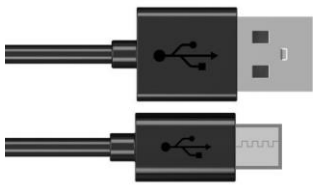
Reads incoming serial data.

Chapter 3 Bluetooth

This chapter mainly introduces how to make simple data transmission through Bluetooth of ESP32-S3 WROOM and mobile phones.

Project 3.1 Bluetooth Low Energy Data Passthrough

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
--	---

Component knowledge

ESP32-S3's integrated Bluetooth function Bluetooth is a short-distance communication system, which can be divided into two types, namely Bluetooth Low Energy(BLE) and Classic Bluetooth. There are two modes for simple data transmission: master mode and slave mode.

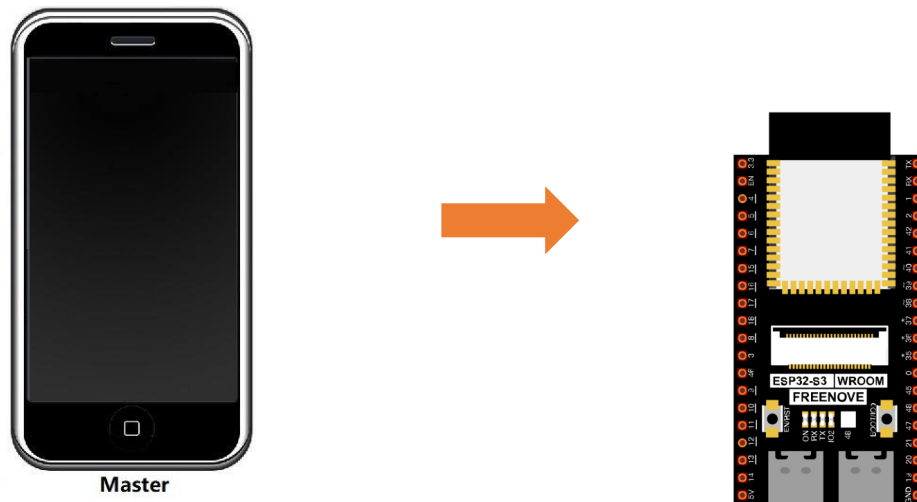
Master mode

In this mode, works are done in the master device and it can connect with a slave device. And we can search and select slave devices nearby to connect with. When a device initiates connection request in master mode, it requires information of the other Bluetooth devices including their address and pairing passkey. After finishing pairing, it can connect with them directly.

Slave mode

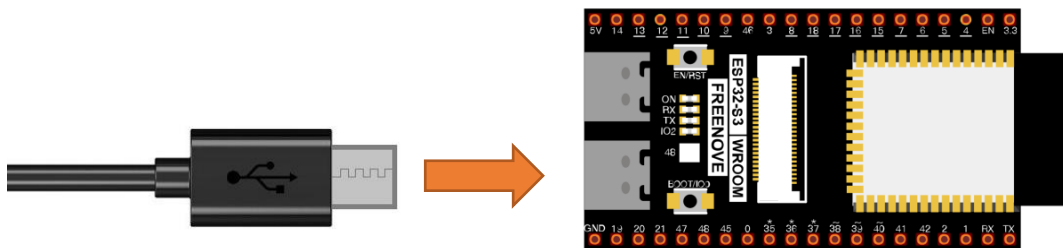
The Bluetooth module in slave mode can only accept connection request from a host computer, but cannot initiate a connection request. After connecting with a host device, it can send data to or receive from the host device.

Bluetooth devices can make data interaction with each other, as one is in master mode and the other in slave mode. When they are making data interaction, the Bluetooth device in master mode searches and selects devices nearby to connect to. When establishing connection, they can exchange data. When mobile phones exchange data with ESP32-S3, they are usually in master mode and ESP32-S3 in slave mode.



Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.

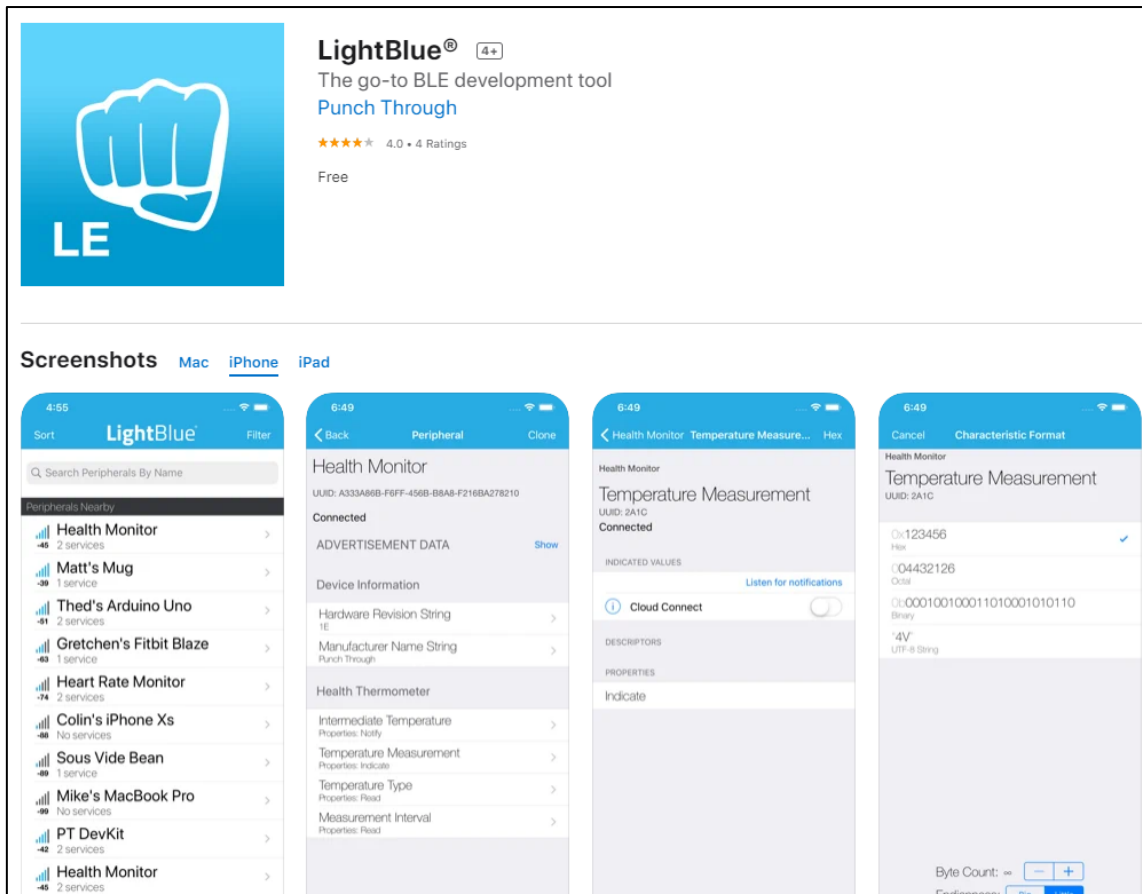


Sketch

Lightblue

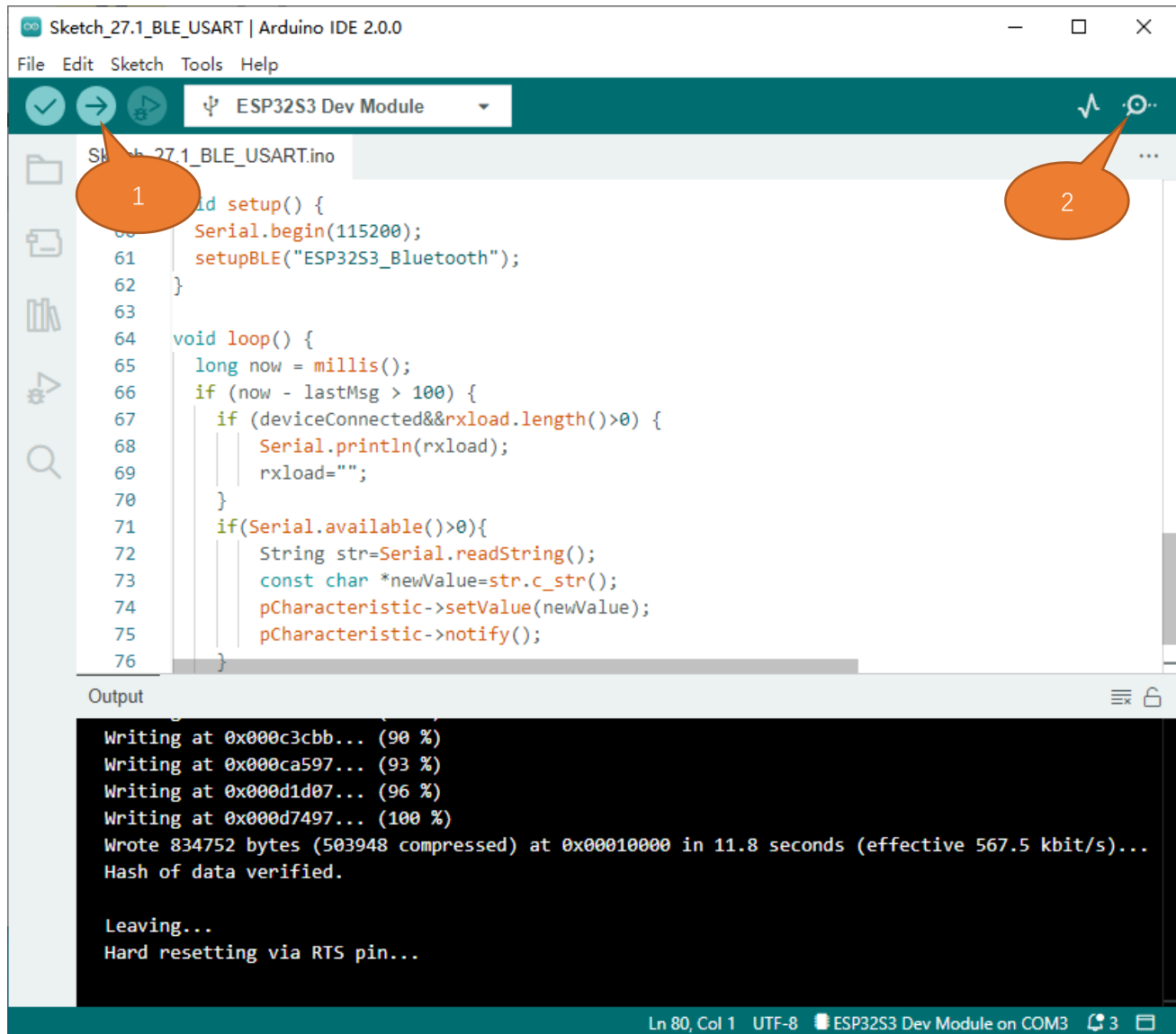
If you can't install Serial Bluetooth on your phone, try LightBlue. If you do not have this software installed on your phone, you can refer to this link:

<https://apps.apple.com/us/app/lightblue/id557428110/?platform=iphone>.

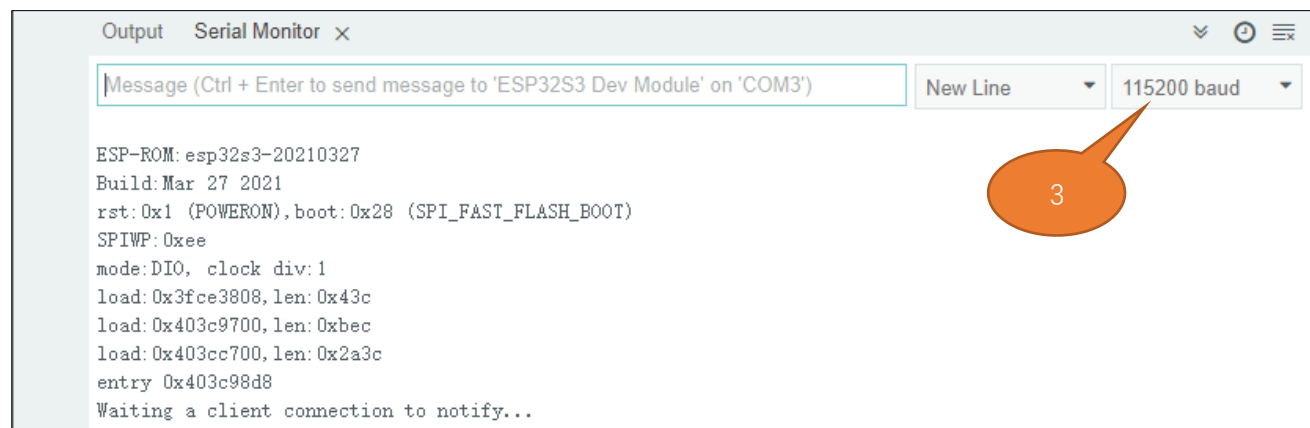


Step1. Upload the code of Project 3.1 to ESP32-S3.

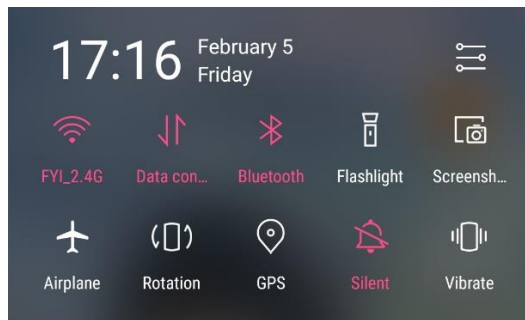
Step2. Click on serial monitor.



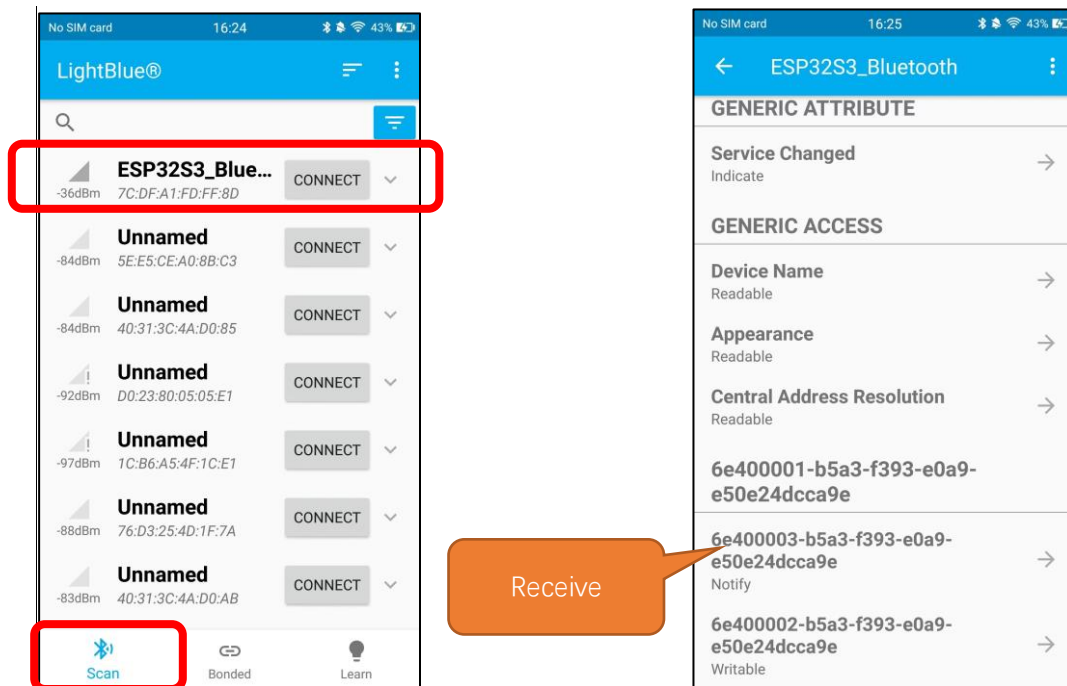
Step3. Set baud rate to 115200.



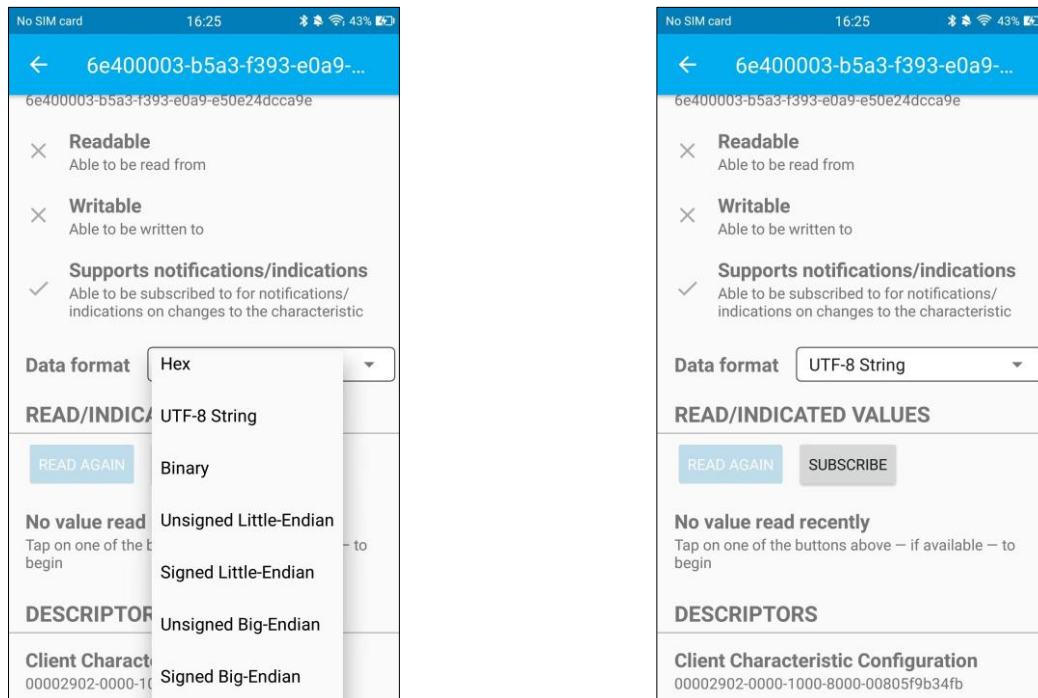
Turn ON Bluetooth on your phone, and open the Lightblue APP.



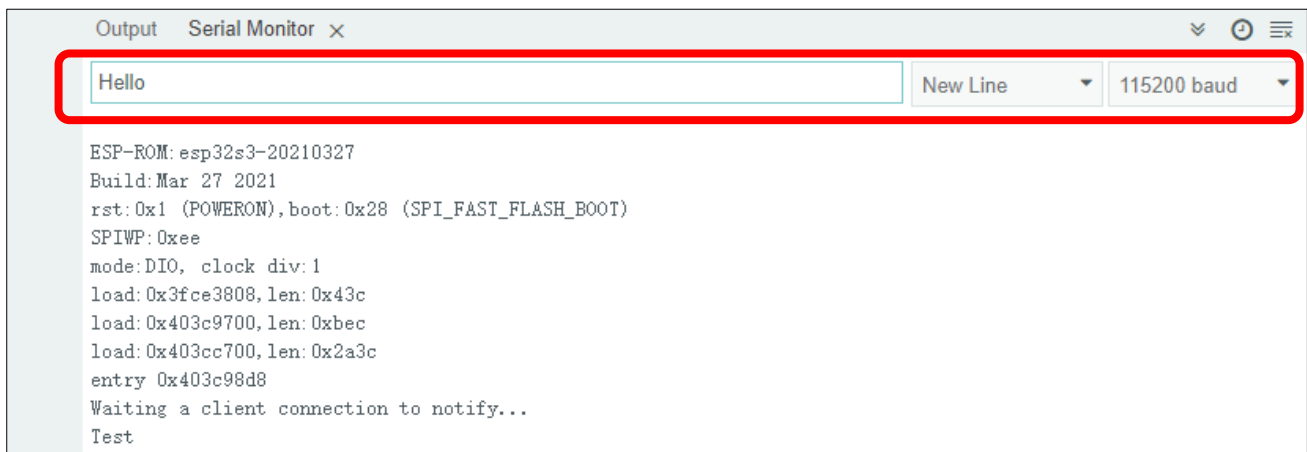
In the Scan page, swipe down to refresh the name of Bluetooth that the phone searches for. Click ESP32S3_Bluetooth.



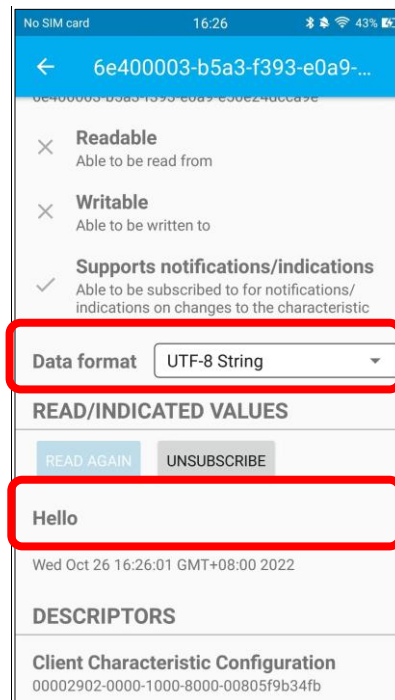
Click "Receive". Select the appropriate Data format in the box to the right of Data Format. For example, HEX for hexadecimal, utf-string for character, Binary for Binary, etc. Then click SUBSCRIBE.



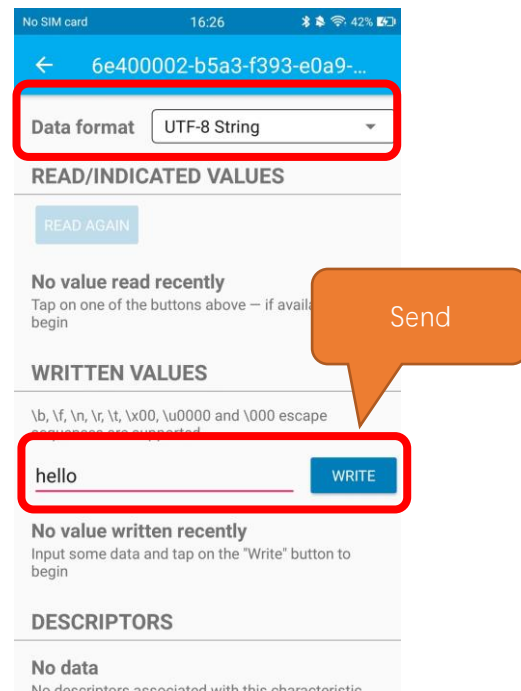
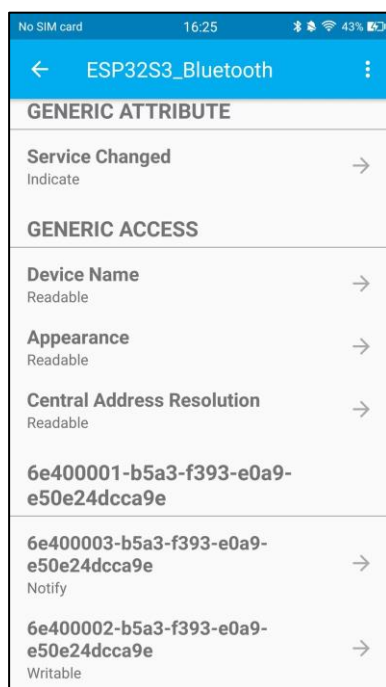
Back to the serial monitor on your computer. You can type anything in the left border of Send, and then click Send.



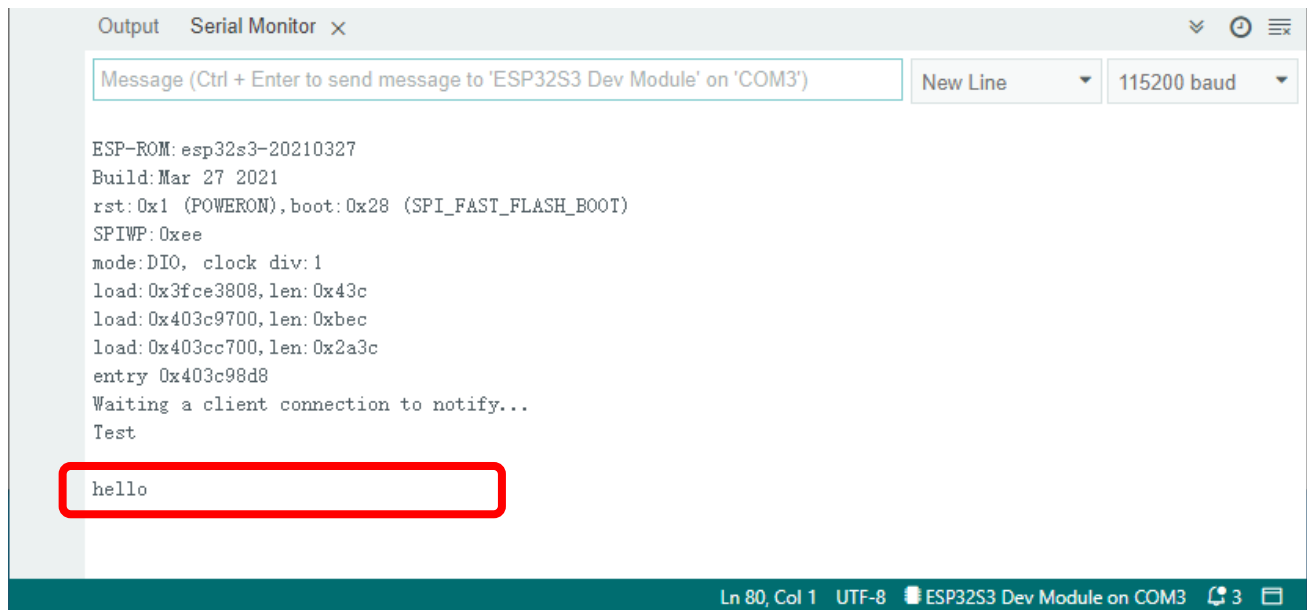
And then you can see the mobile Bluetooth has received the message.



Similarly, you can select “Send” on your phone. Set Data format, and then enter anything in the sending box and click Write to send.



And the computer will receive the message from the mobile Bluetooth.



And now data can be transferred between your mobile phone and computer via ESP32-S3 WROOM.

The following is the program code:

```
1  #include <BLEDevice.h>
2  #include <BLEServer.h>
3  #include <BLEUtils.h>
4  #include <BLE2902.h>
5
6  BLECharacteristic *pCharacteristic;
7  bool deviceConnected = false;
8  uint8_t txValue = 0;
9  long lastMsg = 0;
10 String rxload="Test\n";
11
12 #define SERVICE_UUID          "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
13 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
14 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"
15
16 class MyServerCallbacks: public BLEServerCallbacks {
17     void onConnect(BLEServer* pServer) {
18         deviceConnected = true;
19     };
20     void onDisconnect(BLEServer* pServer) {
21         deviceConnected = false;
22     }
23 };
24
25 class MyCallbacks: public BLECharacteristicCallbacks {
26     void onWrite(BLECharacteristic *pCharacteristic) {
27         String rxValue = pCharacteristic->getValue();
28         if (rxValue.length() > 0) {
29             rxload="";
30             for (int i = 0; i < rxValue.length(); i++) {
31                 rxload +=(char)rxValue[i];
32             }
33         }
34     }
35 };
36
37 void setupBLE(String BLEName){
38     const char *ble_name=BLEName.c_str();
39     BLEDevice::init(ble_name);
40     BLEServer *pServer = BLEDevice::createServer();
41     pServer->setCallbacks(new MyServerCallbacks());
42     BLEService *pService = pServer->createService(SERVICE_UUID);
```

```

43     pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX,BLECharacteristic::PROPERTY_NOTIFY);
44     pCharacteristic->addDescriptor(new BLE2902());
45     BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX,BLECharacteristic::PROPERTY_WRITE);
46     pCharacteristic->setCallbacks(new MyCallbacks());
47     pService->start();
48     pServer->getAdvertising()->start();
49     Serial.println("Waiting a client connection to notify...");
50 }
51
52 void setup() {
53     Serial.begin(115200);
54     setupBLE("ESP32S3_Bluetooth");
55 }
56
57 void loop() {
58     long now = millis();
59     if (now - lastMsg > 1000) {
60         if (deviceConnected&&rxload.length()>0) {
61             Serial.println(rxload);
62             rxload="";
63         }
64         if(Serial.available()>0){
65             String str=Serial.readString();
66             const char *newValue=str.c_str();
67             pCharacteristic->setValue(newValue);
68             pCharacteristic->notify();
69         }
70         lastMsg = now;
71     }
72 }

```

Define the specified UUID number for BLE vendor.

```

12 #define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
13 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
14 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

```

Write a Callback function for BLE server to manage connection of BLE.

```

16 class MyServerCallbacks: public BLEServerCallbacks {
17     void onConnect(BLEServer* pServer) {
18         deviceConnected = true;
19     };
20     void onDisconnect(BLEServer* pServer) {
21         deviceConnected = false;
22     }
23 };

```

Write Callback function with BLE features. When it is called, as the mobile terminal send data to ESP32-S3, it will store them into reload.

```

25 class MyCallbacks: public BLECharacteristicCallbacks {
26     void onWrite(BLECharacteristic *pCharacteristic) {
27         String rxValue = pCharacteristic->getValue();
28         if (rxValue.length() > 0) {
29             rxload="";
30             for (int i = 0; i < rxValue.length(); i++) {
31                 rxload +=(char)rxValue[i];
32             }
33         }
34     }
35 };

```

Initialize the BLE function and name it.

```

54 setupBLE("ESP32S3_Bluetooth");

```

When the mobile phone send data to ESP32-S3 via BLE Bluetooth, it will print them out with serial port; When the serial port of ESP32-S3 receive data, it will send them to mobile via BLE Bluetooth.

```

58 long now = millis();
59 if (now - lastMsg > 1000) {
60     if (deviceConnected&&rxload.length()>0) {
61         Serial.println(rxload);
62         rxload="";
63     }
64     if(Serial.available()>0){
65         String str=Serial.readString();
66         const char *newValue=str.c_str();
67         pCharacteristic->setValue(newValue);
68         pCharacteristic->notify();
69     }
70     lastMsg = now;
71 }

```

The design for creating the BLE server is:

1. Create a BLE Server
2. Create a BLE Service
3. Create a BLE Characteristic on the Service
4. Create a BLE Descriptor on the characteristic
5. Start the service.
6. Start advertising.

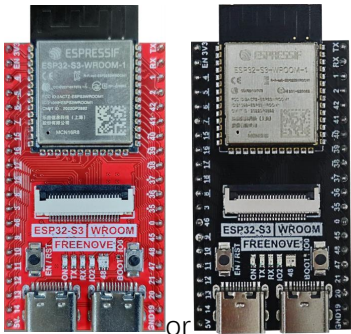
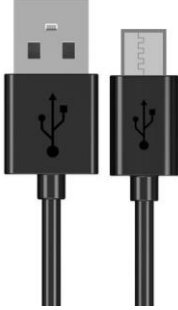
```
37 void setupBLE(String BLEName) {
38     const char *ble_name=BLEName.c_str();
39     BLEDevice::init(ble_name);
40     BLEServer *pServer = BLEDevice::createServer();
41     pServer->setCallbacks(new MyServerCallbacks());
42     BLEService *pService = pServer->createService(SERVICE_UUID);
43     pCharacteristic=
pService->createCharacteristic(CHARACTERISTIC_UUID_TX,BLECharacteristic::PROPERTY_NOTIFY);
44     pCharacteristic->addDescriptor(new BLE2902());
45     BLECharacteristic *pCharacteristic =
pService->createCharacteristic(CHARACTERISTIC_UUID_RX,BLECharacteristic::PROPERTY_WRITE);
46     pCharacteristic->setCallbacks(new MyCallbacks());
pService->start();
47     pServer->getAdvertising()->start();
48     Serial.println("Waiting a client connection to notify...");
49 }
```

Chapter 4 Read and Write the SDcard

An SDcard slot is integrated on the back of the ESP32-S3 WROOM. In this chapter we learn how to use ESP32-S3 to read and write SDcard.

Project 4.1 SDMMC Test

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
SDcard reader x1 (random color)  (Not a USB flash drive.)	SDcard x1 

Component knowledge

SD card read and write method

ESP32-S3 has two ways to use SD card, one is to use the SPI interface to access the SD card, and the other is to use the SDMMC interface to access the SD card. SPI mode uses 4 IOs to access SD card. The SDMMC has one-bit bus mode and four-bit bus mode. In one-bit bus mode, SDMMC use 3 IOs to access SD card. In four-bit bus mode, SDMMC uses 6 IOs to access the SD card.

The above three methods can all be used to access the SD card, the difference is that the access speed is different.

In the four-bit bus mode of SDMMC, the reading and writing speed of accessing the SD card is the fastest. In the one-bit bus mode of SDMMC, the access speed is about 80% of the four-bit bus mode. The access speed of SPI is the slowest, which is about 50% of the four-bit bus mode of SDMMC.

Usually, we recommend using the one-bit bus mode to access the SD card, because in this mode, we only need to use the least pin IO to access the SD card with good performance and speed.

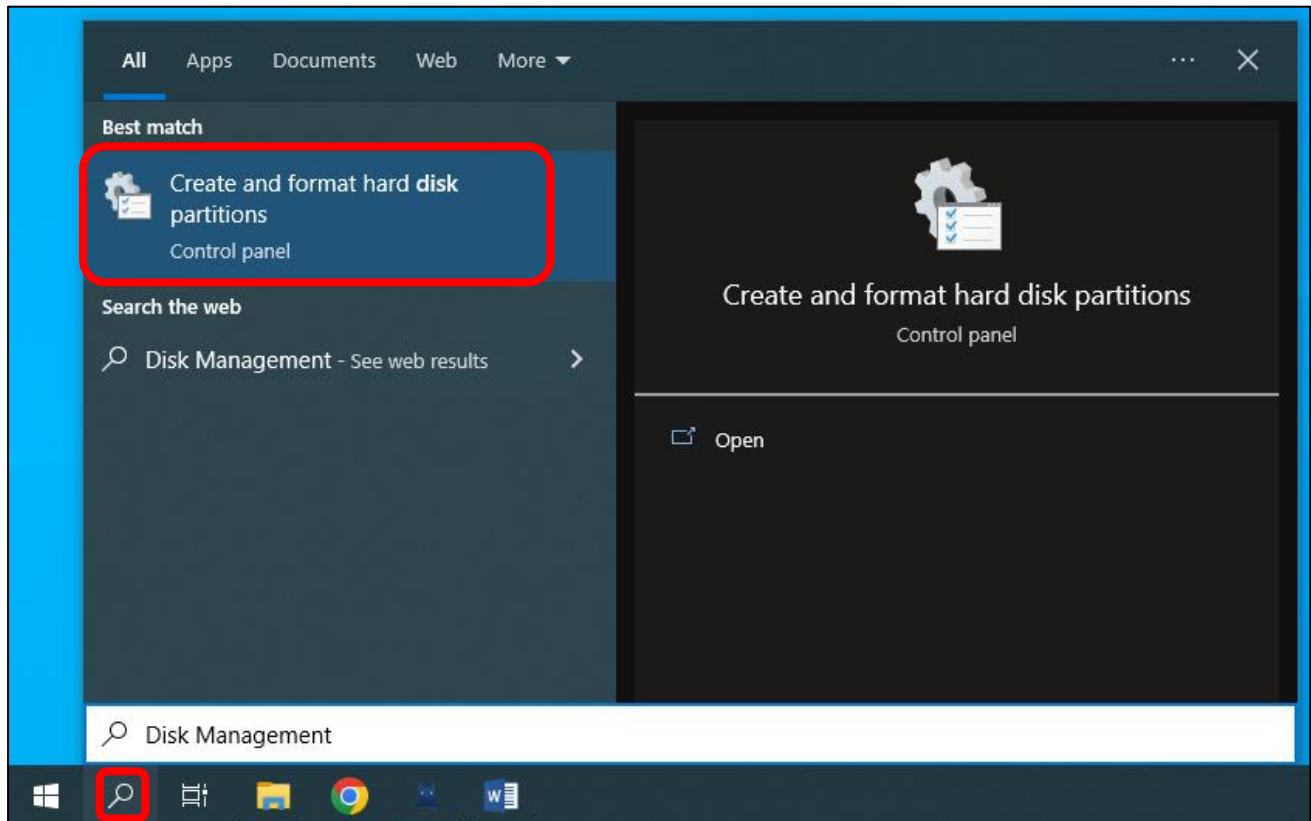
Format SD card

Before starting the tutorial, we need to create a drive letter for the blank SD card and format it. This step requires a card reader and SD card. Please prepare them in advance. Below we will guide you to do it on different computer systems. You can choose the guide that matches your computer.

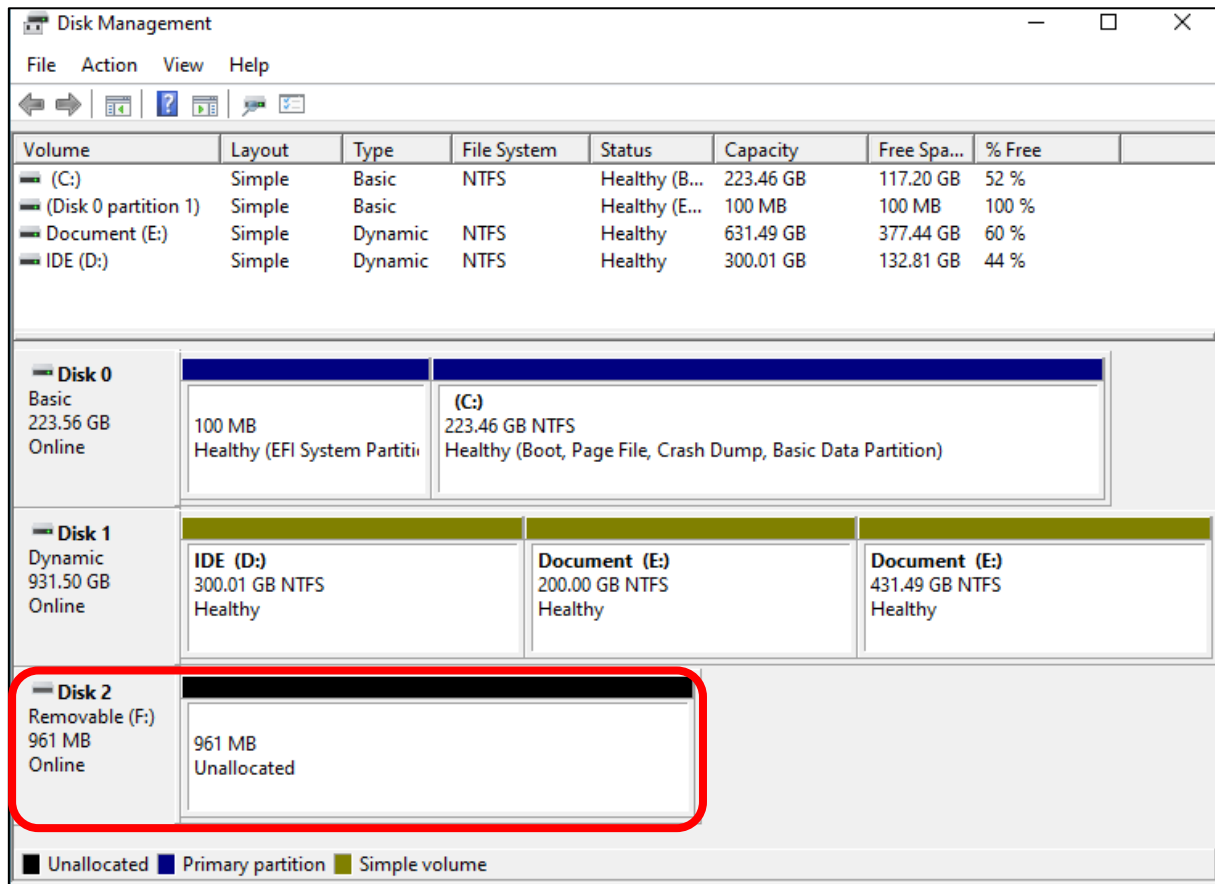
Windows

Insert the SD card into the card reader, then insert the card reader into the computer.

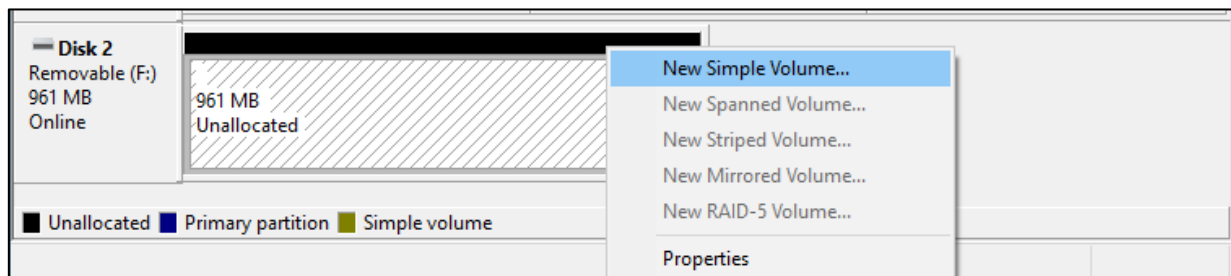
In the Windows search box, enter "Disk Management" and select "Create and format hard disk partitions".



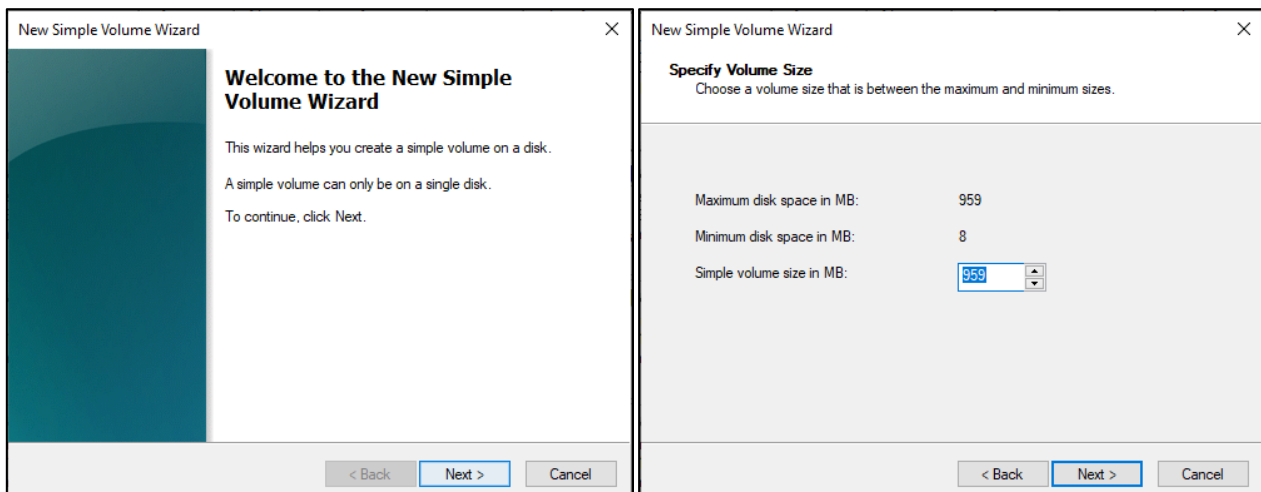
In the new pop-up window, find an unallocated volume close to 1G in size.



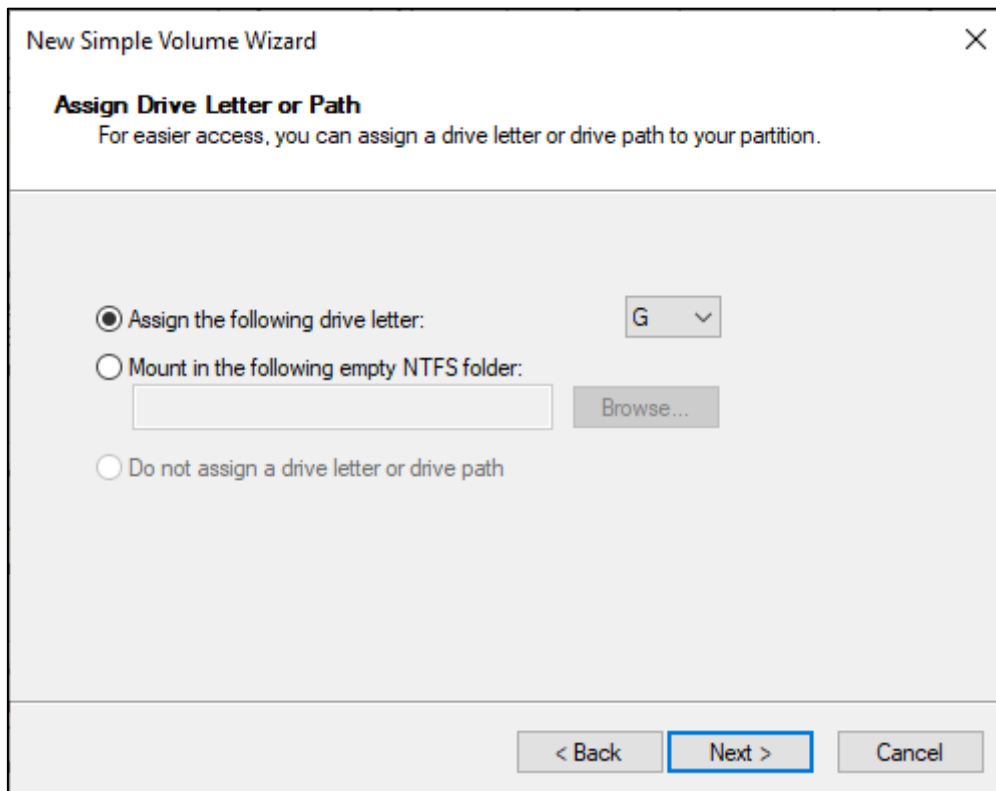
Click to select the volume, right-click and select "New Simple Volume".



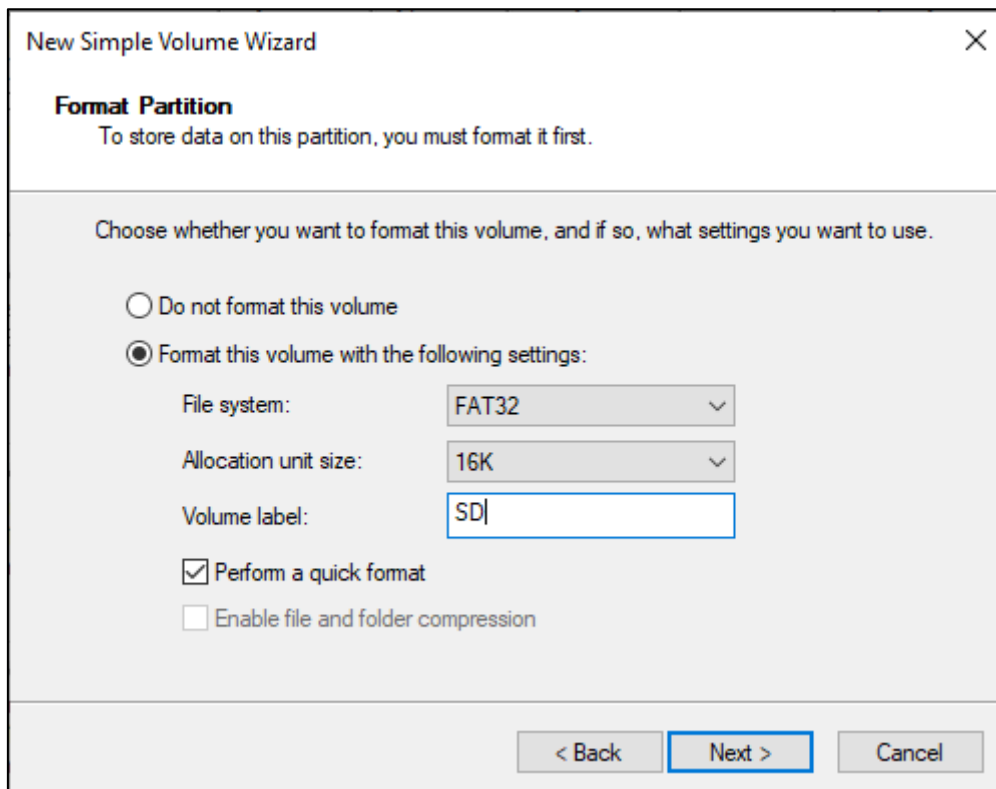
Click Next.



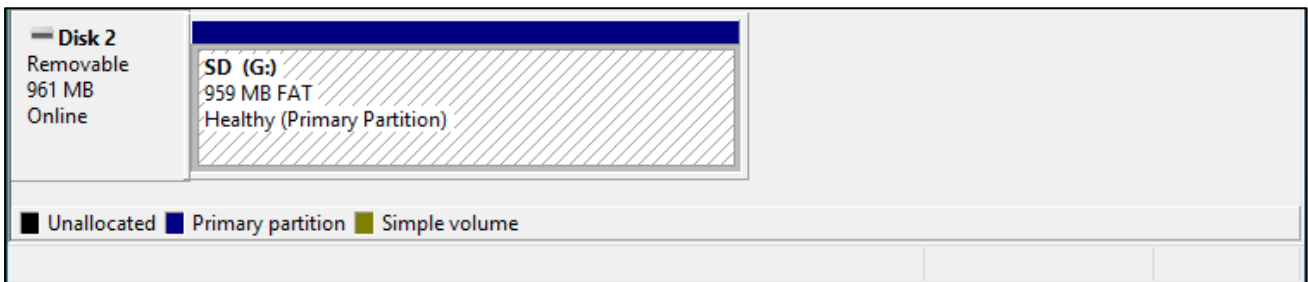
You can choose the drive letter on the right, or you can choose the default. By default, just click Next.



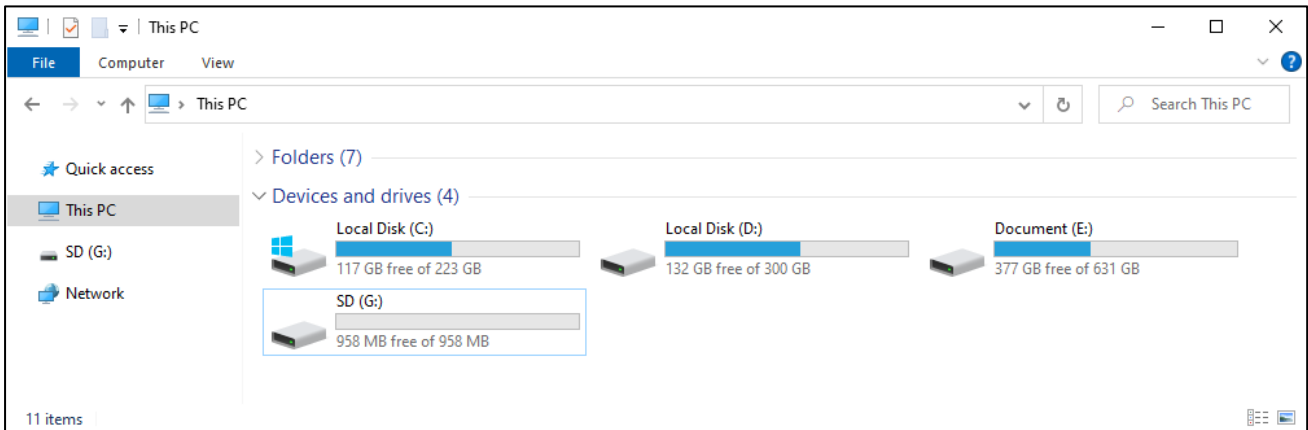
File system is FAT(or FAT32). The Allocation unit size is 16K, and the Volume label can be set to any name. After setting, click Next.



Click Finish. Wait for the SD card initialization to complete.



At this point, you can see the SD card in This PC.

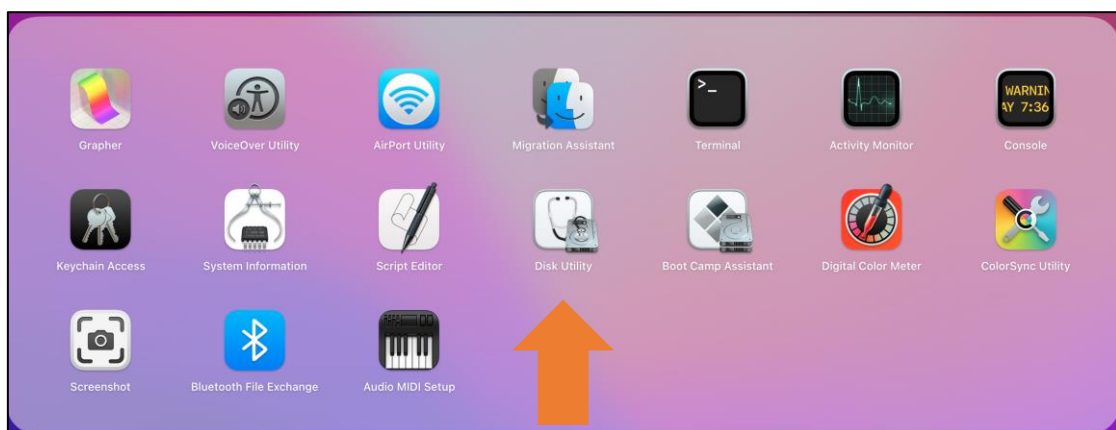


MAC

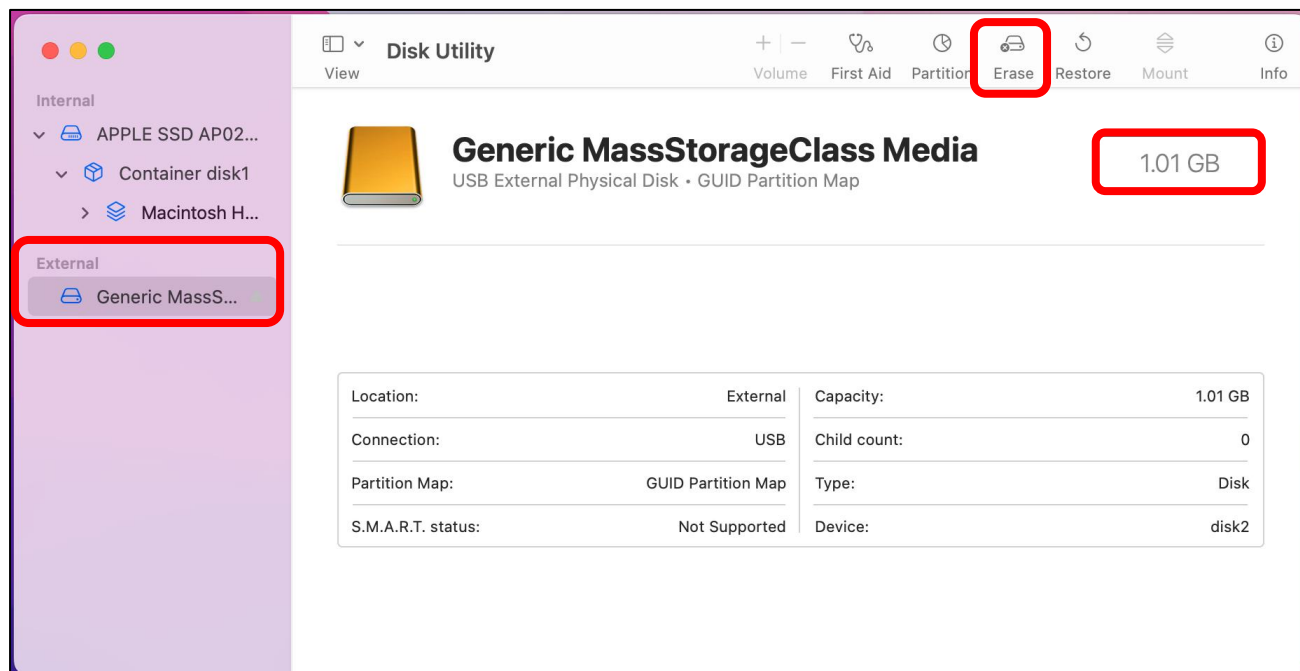
Insert the SD card into the card reader, then insert the card reader into the computer. Some computers will prompt the following information, please click to ignore it.



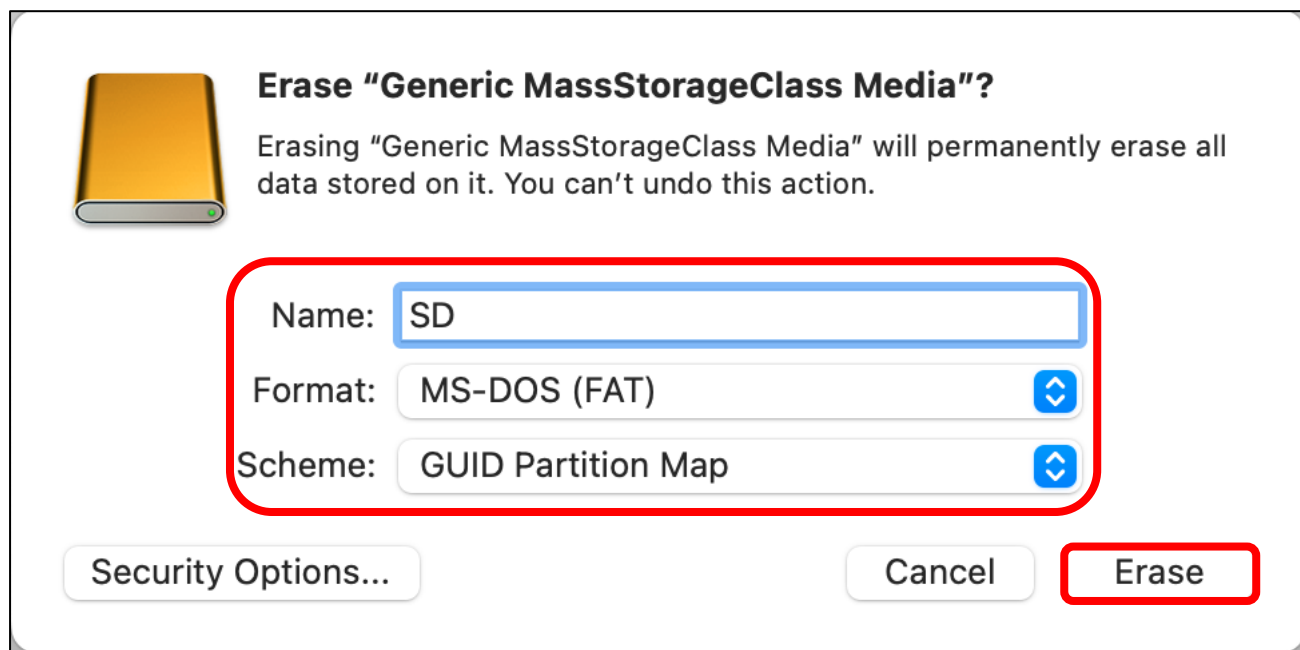
Find "Disk Utility" in the MAC system and click to open it.



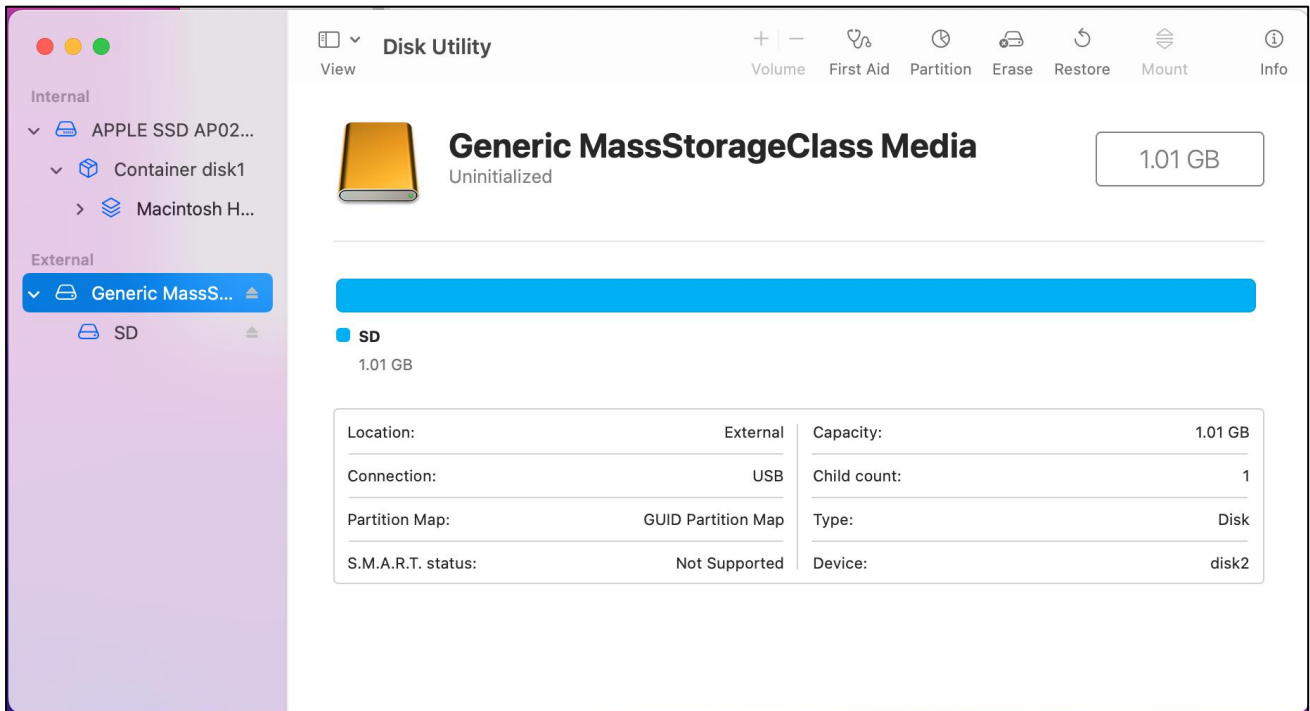
Select "Generic MassStorageClass Media", note that its size is about 1G. Please do not choose wrong item. Click "Erase".



Select the configuration as shown in the figure below, and then click "Erase".

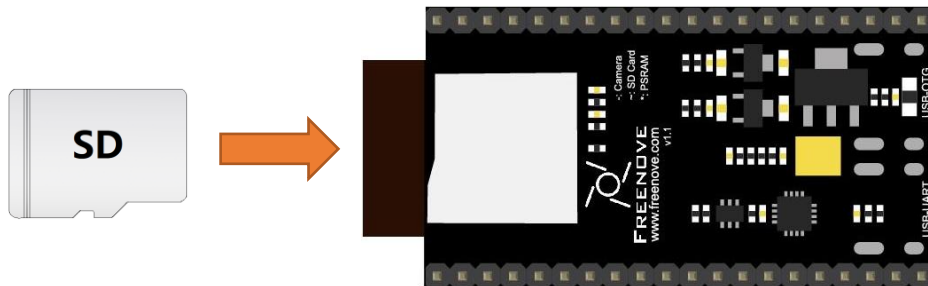


Wait for the formatting to complete. When finished, it will look like the picture below. At this point, you can see a new disk on the desktop named "SD".

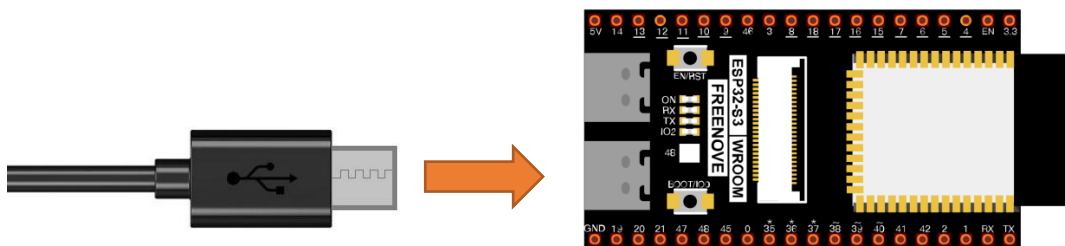


Circuit

Before connecting the USB cable, insert the SD card into the SD card slot on the back of the ESP32-S3.



Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Sketch_04.1_SDMMC_Test



```
File Edit Sketch Tools Help
ESP32S3 Dev Module

Sketch_28.1_SDMMC_Test.ino
168 void setup(){
169     Serial.begin(115200);
170     SD_MMC.setPins(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_D0);
171     if (!SD_MMC.begin("/sdcard", true, true, SDMMC_FREQ_DEFAULT, 5)) {
172         Serial.println("Card Mount Failed");
173         return;
174     }
175     uint8_t cardType = SD_MMC.cardType();
176
177     if(cardType == CARD_NONE){
178         Serial.println("No SD_MMC card attached");
179         return;
180     }
181
182     Serial.print("SD_MMC Card Type: ");
183     if(cardType == CARD_MMC){
184         Serial.println("MMC");
185     } else if(cardType == CARD_SD){
186         Serial.println("SDSC");
187     } else if(cardType == CARD_SDHC){
188         Serial.println("SDHC");
189     } else {
190         Serial.println("UNKNOWN");
191     }
192
193     uint64_t cardSize = SD_MMC.cardSize() / (1024 * 1024);
194     Serial.printf("SD_MMC Card Size: %lluMB\n", cardSize);
```

Compile and upload the code to ESP32-S3-WROOM, open the serial monitor, and press the RST button on the board.

You can see the printout as shown below.



The screenshot shows a Serial Monitor window titled "Output Serial Monitor x". It has a "Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM24')" input field, a "New Line" dropdown, and a "115200 baud" dropdown. The main area displays the following text:

```
Listing directory: /  
  DIR : System Volume Information  
Listing directory: /System Volume Information  
  FILE: IndexerVolumeGuid  SIZE: 76  
  FILE: WPSettings.dat     SIZE: 12  
  FILE: test.txt           SIZE: 1048576  
  FILE: foo.txt            SIZE: 13  
  FILE: 1.py               SIZE: 22  
Writing file: /hello.txt  
File written  
Appending to file: /hello.txt  
Message appended  
Reading file: /hello.txt  
Read from file: Hello World!  
Deleting file: /foo.txt  
File deleted  
Renaming file /hello.txt to /foo.txt  
File renamed  
Reading file: /foo.txt  
Read from file: Hello World!  
1048576 bytes read for 549 ms  
1048576 bytes written for 809 ms  
Total space: 960MB  
Used space: 1MB
```

The status bar at the bottom indicates "Ln 41, Col 12 UTF-8" and "ESP32S3 Dev Module on COM24" with a connection icon and the number "2".

The following is the program code:

```
1  #include "sd_read_write.h"
2  #include "SD_MMC.h"
3
4  #define SD_MMC_CMD 38 //Please do not modify it.
5  #define SD_MMC_CLK 39 //Please do not modify it.
6  #define SD_MMC_DO 40 //Please do not modify it.
7
8  void setup() {
9      Serial.begin(115200);
10     SD_MMC.setPins(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_DO);
11     if (!SD_MMC.begin("/sdcard", true, true, SDMMC_FREQ_DEFAULT, 5)) {
12         Serial.println("Card Mount Failed");
13         return;
14     }
15     uint8_t cardType = SD_MMC.cardType();
16     if(cardType == CARD_NONE) {
17         Serial.println("No SD_MMC card attached");
18         return;
19     }
20     Serial.print("SD_MMC Card Type: ");
21     if(cardType == CARD_MMC) {
22         Serial.println("MMC");
23     } else if(cardType == CARD_SD) {
24         Serial.println("SDSC");
25     } else if(cardType == CARD_SDHC) {
26         Serial.println("SDHC");
27     } else {
28         Serial.println("UNKNOWN");
29     }
30
31     uint64_t cardSize = SD_MMC.cardSize() / (1024 * 1024);
32     Serial.printf("SD_MMC Card Size: %lluMB\n", cardSize);
33
34     listDir(SD_MMC, "/", 0);
35
36     createDir(SD_MMC, "/mydir");
37     listDir(SD_MMC, "/", 0);
38
39     removeDir(SD_MMC, "/mydir");
40     listDir(SD_MMC, "/", 2);
41
42     writeFile(SD_MMC, "/hello.txt", "Hello ");
43     appendFile(SD_MMC, "/hello.txt", "World!\n");
```

```

44     readFile(SD_MMC, "/hello.txt");
45
46     deleteFile(SD_MMC, "/foo.txt");
47     renameFile(SD_MMC, "/hello.txt", "/foo.txt");
48     readFile(SD_MMC, "/foo.txt");
49
50     testFileIO(SD_MMC, "/test.txt");
51
52     Serial.printf("Total space: %lluMB\r\n", SD_MMC.totalBytes() / (1024 * 1024));
53     Serial.printf("Used space: %lluMB\r\n", SD_MMC.usedBytes() / (1024 * 1024));
54 }
55
56 void loop() {
57     delay(10000);
58 }

```

Add the SD card drive header file.

```

1  #include "sd_read_write.h"
2  #include "SD_MMC.h"

```

Defines the drive pins of the SD card. Please do not modify it. Because these pins are fixed.

```

4  #define SD_MMC_CMD 38 //Please do not modify it.
5  #define SD_MMC_CLK 39 //Please do not modify it.
6  #define SD_MMC_DO 40 //Please do not modify it.

```

Initialize the serial port function. Sets the drive pin for SDMMC one-bit bus mode.

```

9     Serial.begin(115200);
10    SD_MMC.setPins(SD_MMC_CLK, SD_MMC_CMD, SD_MMC_DO);

```

Set the mount point of the SD card, set SDMMC to one-bit bus mode, and set the read and write speed to 20MHz.

```

11    if (!SD_MMC.begin("/sdcard", true, true, SDMMC_FREQ_DEFAULT, 5)) {
12        Serial.println("Card Mount Failed");
13        return;
14    }

```

Get the type of SD card and print it out through the serial port.

```

15    uint8_t cardType = SD_MMC.cardType();
16    if(cardType == CARD_NONE) {
17        Serial.println("No SD_MMC card attached");
18        return;
19    }
20    Serial.print("SD_MMC Card Type: ");
21    if(cardType == CARD_MMC) {
22        Serial.println("MMC");
23    } else if(cardType == CARD_SD) {
24        Serial.println("SDSC");
25    } else if(cardType == CARD_SDHC) {
26        Serial.println("SDHC");

```

```
27     } else {  
28         Serial.println("UNKNOWN");  
29     }
```

Call the `listDir()` function to read the folder and file names in the SD card, and print them out through the serial port. This function can be found in "sd_read_write.cpp".

```
34     listDir(SD_MMC, "/", 0);
```

Call `createDir()` to create a folder, and call `removeDir()` to delete a folder.

```
36     createDir(SD_MMC, "/mydir");  
39     removeDir(SD_MMC, "/mydir");
```

Call `writeFile()` to write any content to the txt file. If there is no such file, create this file first.

Call `appendFile()` to append any content to txt.

Call `readFile()` to read the content in txt and print it via the serial port.

```
42     writeFile(SD_MMC, "/hello.txt", "Hello ");  
43     appendFile(SD_MMC, "/hello.txt", "World!\n");  
44     readFile(SD_MMC, "/hello.txt");
```

Call `deleteFile()` to delete a specified file.

Call `renameFile()` to copy a file and rename it.

```
46     deleteFile(SD_MMC, "/foo.txt");  
47     renameFile(SD_MMC, "/hello.txt", "/foo.txt");
```

Call the `testFileIO()` function to test the time it takes to read 512 bytes and the time it takes to write 2048*512 bytes of data.

```
50     testFileIO(SD_MMC, "/test.txt");
```

Print the total size and used size of the SD card via the serial port.

```
52     Serial.printf("Total space: %lluMB\r\n", SD_MMC.totalBytes() / (1024 * 1024));  
53     Serial.printf("Used space: %lluMB\r\n", SD_MMC.usedBytes() / (1024 * 1024));
```

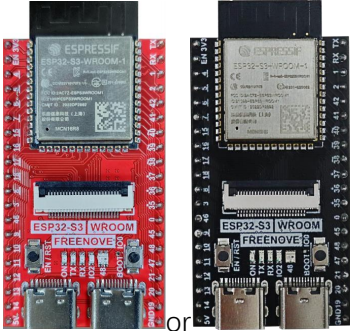
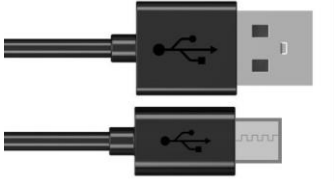
Chapter 5 WiFi Working Modes

In this chapter, we'll focus on the WiFi infrastructure for ESP32-S3 WROOM.

ESP32-S3 WROOM has 3 different WiFi operating modes: station mode, AP mode and AP+station mode. All WiFi programming projects must be configured with WiFi operating mode before using WiFi, otherwise WiFi cannot be used.

Project 5.1 Station mode

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
--	---

Component knowledge

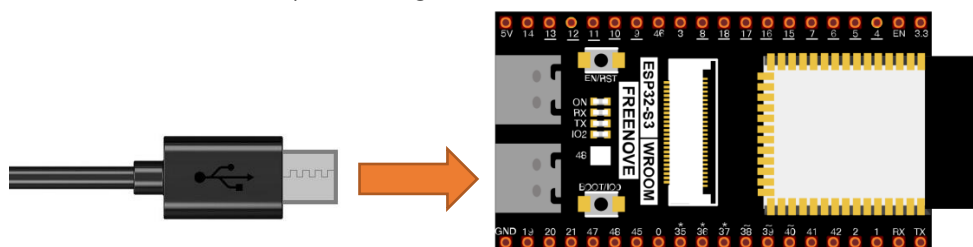
Station mode

When ESP32-S3 selects Station mode, it acts as a WiFi client. It can connect to the router network and communicate with other devices on the router via WiFi connection. As shown below, the PC is connected to the router, and if ESP32-S3 wants to communicate with the PC, it needs to be connected to the router.



Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



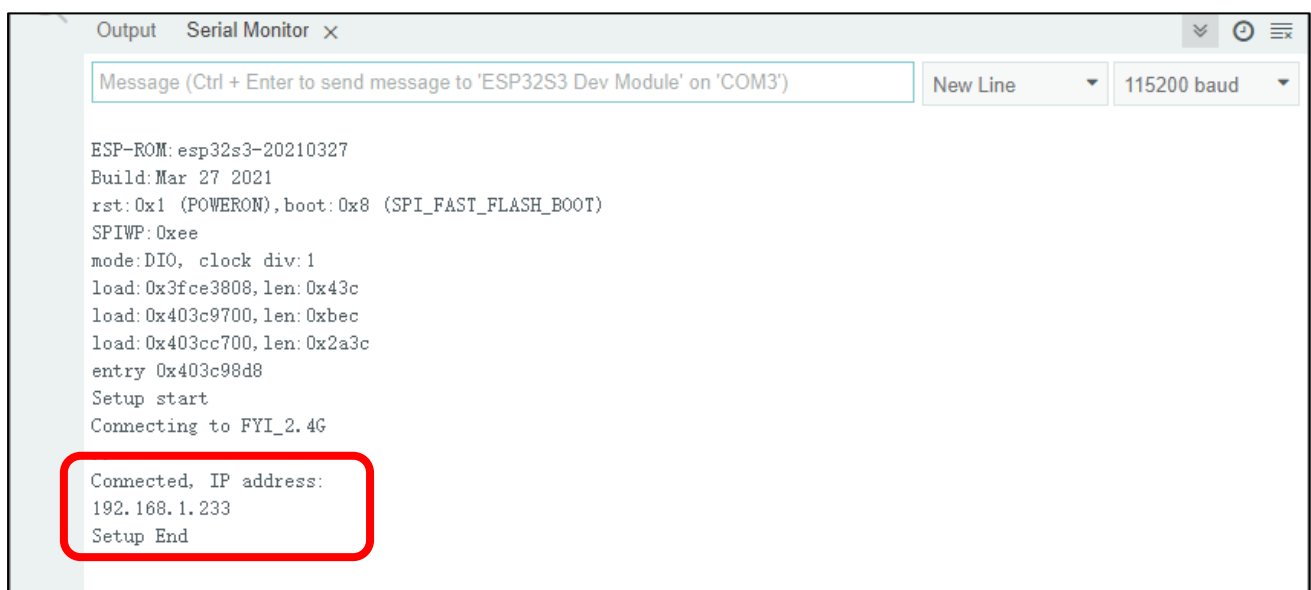
Sketch

Sketch_05.1_Station_mode



Because the names and passwords of routers in various places are different, before the Sketch runs, users need to enter the correct router's name and password in the box as shown in the illustration above.

After making sure the router name and password are entered correctly, compile and upload codes to ESP32-S3 WROOM, open serial monitor and set baud rate to 115200. And then it will display as follows:



When ESP32-S3 WROOM successfully connects to "ssid_Router", serial monitor will print out the IP address assigned to ESP32-S3 WROOM by the router.

The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5
6  void setup() {
7      Serial.begin(115200);
8      delay(2000);
9      Serial.println("Setup start");
10     WiFi.begin(ssid_Router, password_Router);
11     Serial.println(String("Connecting to ") + ssid_Router);
12     while (WiFi.status() != WL_CONNECTED) {
13         delay(500);
14         Serial.print(".");
15     }
16     Serial.println("\nConnected, IP address: ");
17     Serial.println(WiFi.localIP());
18     Serial.println("Setup End");
19 }
20
21 void loop() {
22 }

```

Include the WiFi Library header file of ESP32-S3.

```
1  #include <WiFi.h>
```

Enter correct router name and password.

```

3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password

```

Set ESP32-S3 in Station mode and connect it to your router.

```
10  WiFi.begin(ssid_Router, password_Router);
```

Check whether ESP32-S3 has connected to router successfully every 0.5s.

```

12  while (WiFi.status() != WL_CONNECTED) {
13      delay(500);
14      Serial.print(".");
15  }

```

Serial monitor prints out the IP address assigned to ESP32-S3 WROOM

```
17  Serial.println(WiFi.localIP());
```

Reference

Class Station

Every time when using WiFi, you need to include header file "WiFi.h".

begin(ssid, password, channel, bssid, connect): ESP32-S3 is used as Station to connect hotspot.

ssid: WiFi hotspot name

password: WiFi hotspot password

channel: WiFi hotspot channel number; communicating through specified channel; optional parameter

bssid: mac address of WiFi hotspot, optional parameter

connect: boolean optional parameter, defaulting to true. If set as false, then ESP32-S3 won't connect WiFi.

config(local_ip, gateway, subnet, dns1, dns2): set static local IP address.

local_ip: station fixed IP address.

subnet: subnet mask

dns1,dns2: optional parameter. define IP address of domain name server

status: obtain the connection status of WiFi

local IP(): obtain IP address in Station mode

disconnect(): disconnect wifi

setAutoConnect(boolean): set automatic connection Every time ESP32-S3 is power on, it will connect WiFi automatically.

setAutoReconnect(boolean): set automatic reconnection Every time ESP32-S3 disconnects WiFi, it will reconnect to WiFi automatically.

Project 5.2 AP mode

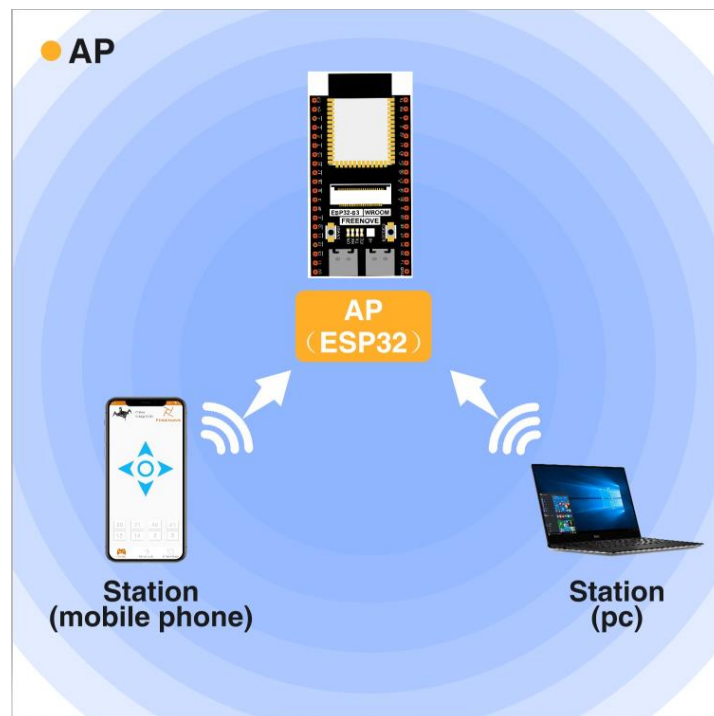
Component List & Circuit

Component List & Circuit are the same as in Project 5.1.

Component knowledge

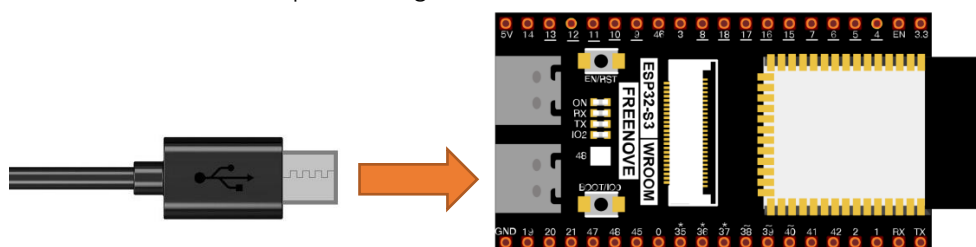
AP mode

When ESP32-S3 selects AP mode, it creates a hotspot network that is separate from the Internet and waits for other WiFi devices to connect. As shown in the figure below, ESP32-S3 is used as a hotspot. If a mobile phone or PC wants to communicate with ESP32-S3, it must be connected to the hotspot of ESP32-S3. Only after a connection is established with ESP32-S3 can they communicate.

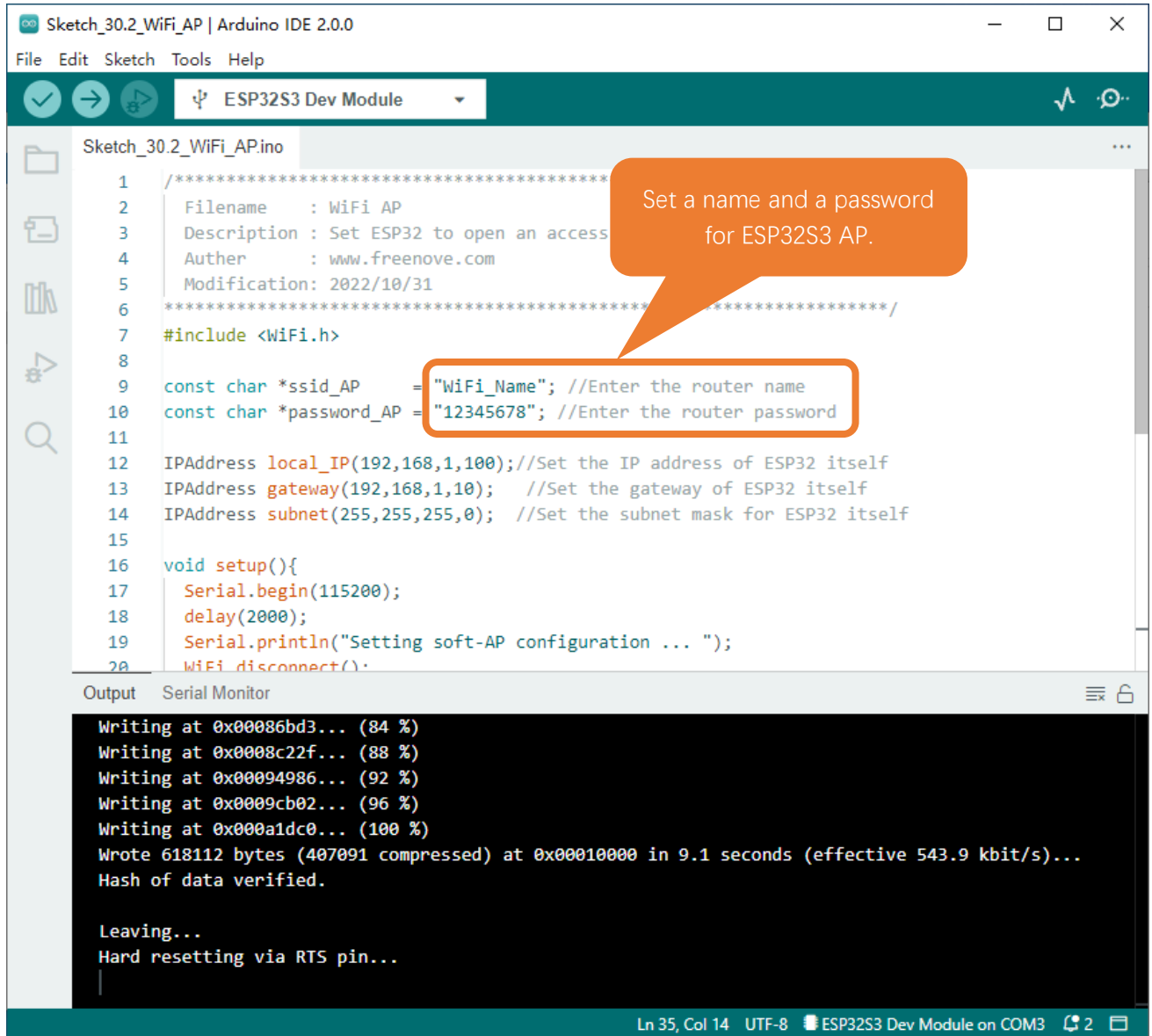


Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch



```
Sketch_30.2_WiFi_AP.ino
1  /*****
2  Filename   : WiFi AP
3  Description: Set ESP32 to open an access point
4  Author    : www.freenove.com
5  Modification: 2022/10/31
6  *****/
7  #include <WiFi.h>
8
9  const char *ssid_AP = "WiFi_Name"; //Enter the router name
10 const char *password_AP = "12345678"; //Enter the router password
11
12 IPAddress local_IP(192,168,1,100); //Set the IP address of ESP32 itself
13 IPAddress gateway(192,168,1,10); //Set the gateway of ESP32 itself
14 IPAddress subnet(255,255,255,0); //Set the subnet mask for ESP32 itself
15
16 void setup(){
17   Serial.begin(115200);
18   delay(2000);
19   Serial.println("Setting soft-AP configuration ... ");
20   WiFi.disconnect();
21 }
```

Set a name and a password for ESP32S3 AP.

Writing at 0x00086bd3... (84 %)
Writing at 0x0008c22f... (88 %)
Writing at 0x00094986... (92 %)
Writing at 0x0009cb02... (96 %)
Writing at 0x000a1dc0... (100 %)
Wrote 618112 bytes (407091 compressed) at 0x00010000 in 9.1 seconds (effective 543.9 kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...

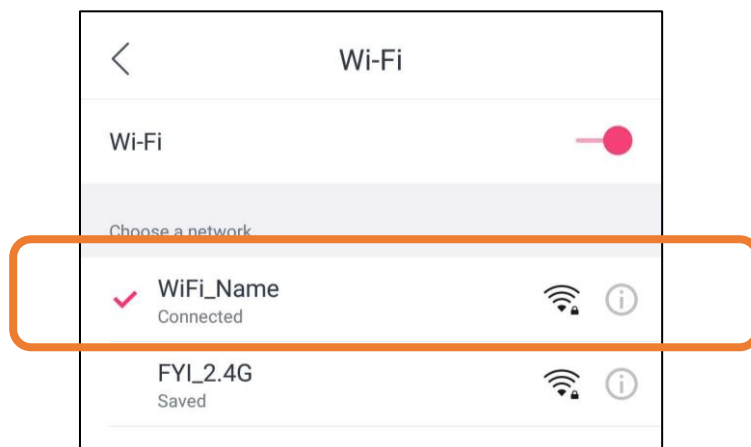
Before the Sketch runs, you can make any changes to the AP name and password for ESP32-S3 in the box as shown in the illustration above. Of course, you can leave it alone by default.

Compile and upload codes to ESP32-S3 WROOM, open the serial monitor and set the baud rate to 115200. And then it will display as follows.



```
Output Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3')
load: 0x403c9100, len: 0x2a3c
load: 0x403cc700, len: 0x2a3c
entry 0x403c98d8
Setting soft-AP configuration ...
Ready
Setting soft-AP ...
Ready
Soft-AP IP address = 192.168.1.100
MAC address = 7E:DF:A1:FD:FF:8C
Setup End
Ln 35, Col 14 UTF-8 ESP32S3 Dev Module on COM3 2
```

When observing the print information of the serial monitor, turn on the WiFi scanning function of your phone, and you can see the ssid_AP on ESP32-S3, which is called "WiFi_Name" in this Sketch. You can enter the password "12345678" to connect it or change its AP name and password by modifying Sketch.



Sketch_05.2_AP_mode

The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_AP    = "WiFi_Name"; //Enter the router name
4  const char *password_AP = "12345678"; //Enter the router password
5
6  IPAddress local_IP(192,168,1,100); //Set the IP address of ESP32-S3 itself
7  IPAddress gateway(192,168,1,10);   //Set the gateway of ESP32-S3 itself
8  IPAddress subnet(255,255,255,0);  //Set the subnet mask for ESP32-S3 itself
9
10 void setup() {
11     Serial.begin(115200);
12     delay(2000);
13     Serial.println("Setting soft-AP configuration ... ");
14     WiFi.disconnect();
15     WiFi.mode(WIFI_AP);
16     Serial.println(WiFi.softAPConfig(local_IP, gateway, subnet) ? "Ready" : "Failed!");
17     Serial.println("Setting soft-AP ... ");
18     boolean result = WiFi.softAP(ssid_AP, password_AP);
19     if(result) {
20         Serial.println("Ready");
21         Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
22         Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
23     } else {
24         Serial.println("Failed!");
25     }
26     Serial.println("Setup End");
27 }
28
29 void loop() {
30 }

```

Include WiFi Library header file of ESP32-S3.

```
1  #include <WiFi.h>
```

Enter correct AP name and password.

```

3  const char *ssid_AP    = "WiFi_Name"; //Enter the router name
4  const char *password_AP = "12345678"; //Enter the router password

```

Set ESP32-S3 in AP mode.

```
15  WiFi.mode(WIFI_AP);
```

Configure IP address, gateway and subnet mask for ESP32-S3.

```
16  WiFi.softAPConfig(local_IP, gateway, subnet)
```

Turn on an AP in ESP32-S3, whose name is set by ssid_AP and password is set by password_AP.

```
18  WiFi.softAP(ssid_AP, password_AP);
```

Check whether the AP is turned on successfully. If yes, print out IP and MAC address of AP established by ESP32-S3. If no, print out the failure prompt.

```

19  if(result) {
20      Serial.println("Ready");
21      Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
22      Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
23  }else{
24      Serial.println("Failed!");
25  }
26  Serial.println("Setup End");

```

Reference

Class AP

Every time when using WiFi, you need to include header file "WiFi.h".

softAP(ssid, password, channel, ssid_hidden, max_connection):

ssid: WiFi hotspot name

password: WiFi hotspot password

channel: Number of WiFi connection channels, range 1-13. The default is 1.

ssid_hidden: Whether to hide WiFi name from scanning by other devices. The default is not hide.

max_connection: Maximum number of WiFi connected devices. The range is 1-4. The default is 4.

softAPConfig(local_ip, gateway, subnet): set static local IP address.

local_ip: station fixed IP address.

Gateway: gateway IP address

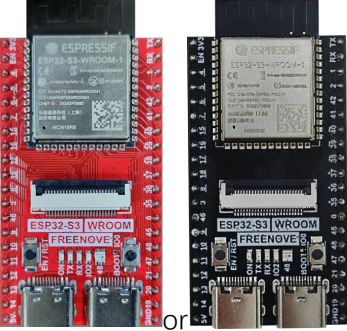
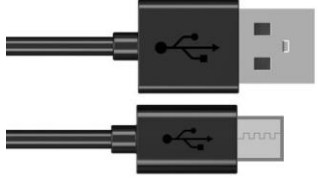
subnet: subnet mask

softAP(): obtain IP address in AP mode

softAPdisconnect (): disconnect AP mode.

Project 5.3 AP+Station mode

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
---	--

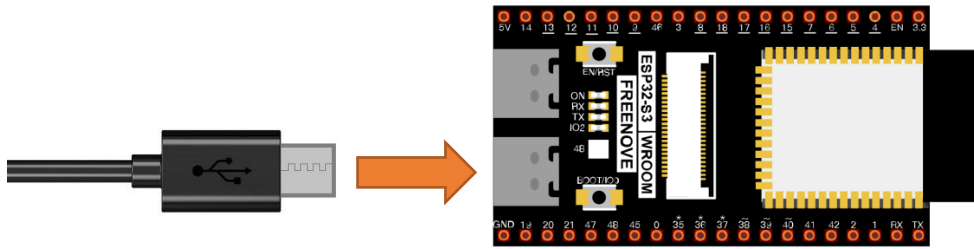
Component knowledge

AP+Station mode

In addition to AP mode and station mode, ESP32-S3 can also use AP mode and station mode at the same time. This mode contains the functions of the previous two modes. Turn on ESP32-S3's station mode, connect it to the router network, and it can communicate with the Internet via the router. At the same time, turn on its AP mode to create a hotspot network. Other WiFi devices can choose to connect to the router network or the hotspot network to communicate with ESP32-S3.

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Sketch_05.3_AP_Station_mode

```
File Edit Sketch Tools Help
Select Board

Sketch_30.3_AP_Station.ino
1  /*****
2  Filename   : WiFi AP+Station
3  Description : ESP32 connects to the user's
4  Author    : www.freenove.com
5  Modification: 2022/10/31
6  *****/
7  #include <WiFi.h>
8
9  const char *ssid_Router   = "*****"; //Enter the router name
10 const char *password_Router = "*****"; //Enter the router password
11 const char *ssid_AP       = "WiFi_Name"; //Enter the router name
12 const char *password_AP   = "12345678"; //Enter the router password
13
14 void setup(){
15   Serial.begin(115200);
16   Serial.println("Setting soft-AP configuration ... ");
17   WiFi.disconnect();
18   WiFi.mode(WIFI_AP);
19   Serial.println("Setting soft-AP ... ");
20   boolean result = WiFi.softAP(ssid_AP, password_AP);
21   if(result){
22     Serial.println("Ready");
23     Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
24     Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
25   }else{
26     Serial.println("Failed!");
27   }
}
```

Please enter the correct names and passwords of Router and AP.

It is analogous to Project 5.1 and Project 5.2. Before running the Sketch, you need to modify `ssid_Router`, `password_Router`, `ssid_AP` and `password_AP` shown in the box of the illustration above.

After making sure that Sketch is modified correctly, compile and upload codes to ESP32-S3 WROOM, open serial monitor and set baud rate to 115200. And then it will display as follows:

```

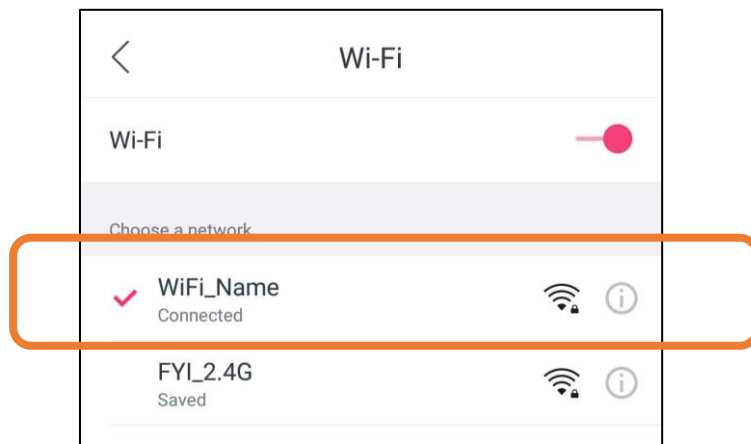
Output  Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3')
New Line 115200 baud

rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce3808,len:0x43c
load:0x403c9700,len:0xbec
load:0x403cc700,len:0x2a3c
entry 0x403c98d8
Setting soft-AP configuration ...
Setting soft-AP ...
Ready
Soft-AP IP address = 192.168.4.1
MAC address = 7E:DF:A1:FD:FF:8C

Setting Station configuration ...
Connecting to FYI_2.4G
...
Connected, IP address:
192.168.1.233
Setup End
Ln 10, Col 44 UTF-8 ESP32S3 Dev Module on COM3 2

```

When observing the print information of the serial monitor, turn on the WiFi scanning function of your phone, and you can see the ssid_AP on ESP32-S3.



The following is the program code:

```

1  #include <WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  const char *ssid_AP        = "WiFi_Name"; //Enter the AP name
6  const char *password_AP    = "12345678"; //Enter the AP password
7
8  void setup() {
9      Serial.begin(115200);
10     Serial.println("Setting soft-AP configuration ... ");

```

```
11  WiFi.disconnect();
12  WiFi.mode(WIFI_AP);
13  Serial.println("Setting soft-AP ... ");
14  boolean result = WiFi.softAP(ssid_AP, password_AP);
15  if(result) {
16      Serial.println("Ready");
17      Serial.println(String("Soft-AP IP address = ") + WiFi.softAPIP().toString());
18      Serial.println(String("MAC address = ") + WiFi.softAPmacAddress().c_str());
19  } else {
20      Serial.println("Failed!");
21  }
22
23  Serial.println("\nSetting Station configuration ... ");
24  WiFi.begin(ssid_Router, password_Router);
25  Serial.println(String("Connecting to ") + ssid_Router);
26  while (WiFi.status() != WL_CONNECTED) {
27      delay(500);
28      Serial.print(".");
29  }
30  Serial.println("\nConnected, IP address: ");
31  Serial.println(WiFi.localIP());
32  Serial.println("Setup End");
33  }
34
35  void loop() {
36  }
```

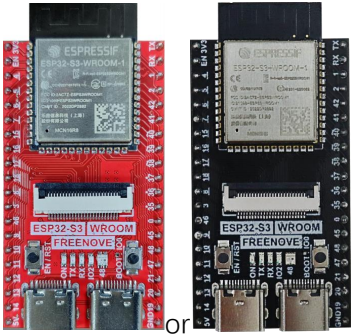
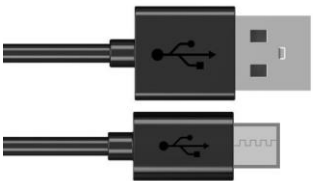
Chapter 6 TCP/IP

In this chapter, we will introduce how ESP32-S3 implements network communications based on TCP/IP protocol. There are two roles in TCP/IP communication, namely Server and Client, which will be implemented respectively with two projects in this chapter.

Project 6.1 As Client

In this section, ESP32-S3 is used as Client to connect Server on the same LAN and communicate with it.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8)  or	USB cable x1 
--	---

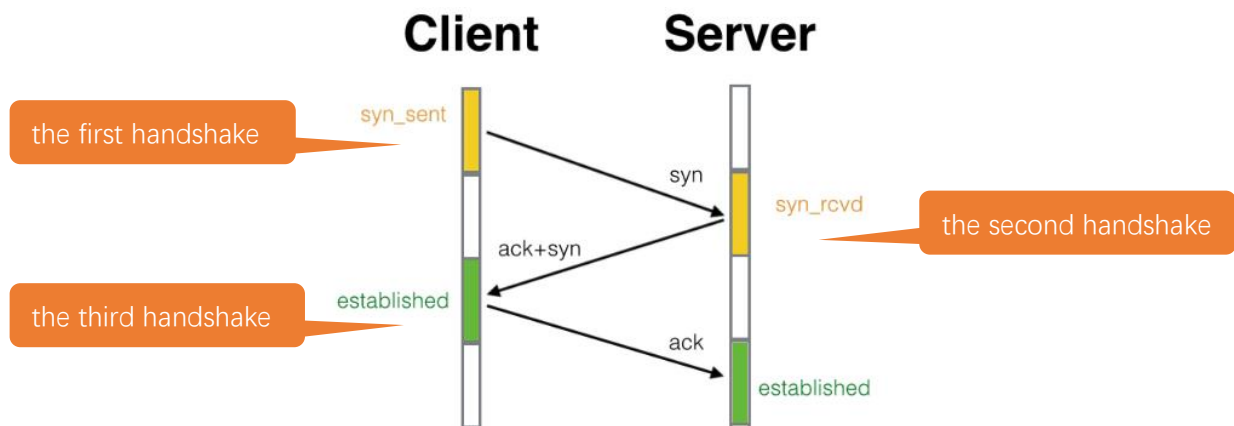
Component knowledge

TCP connection

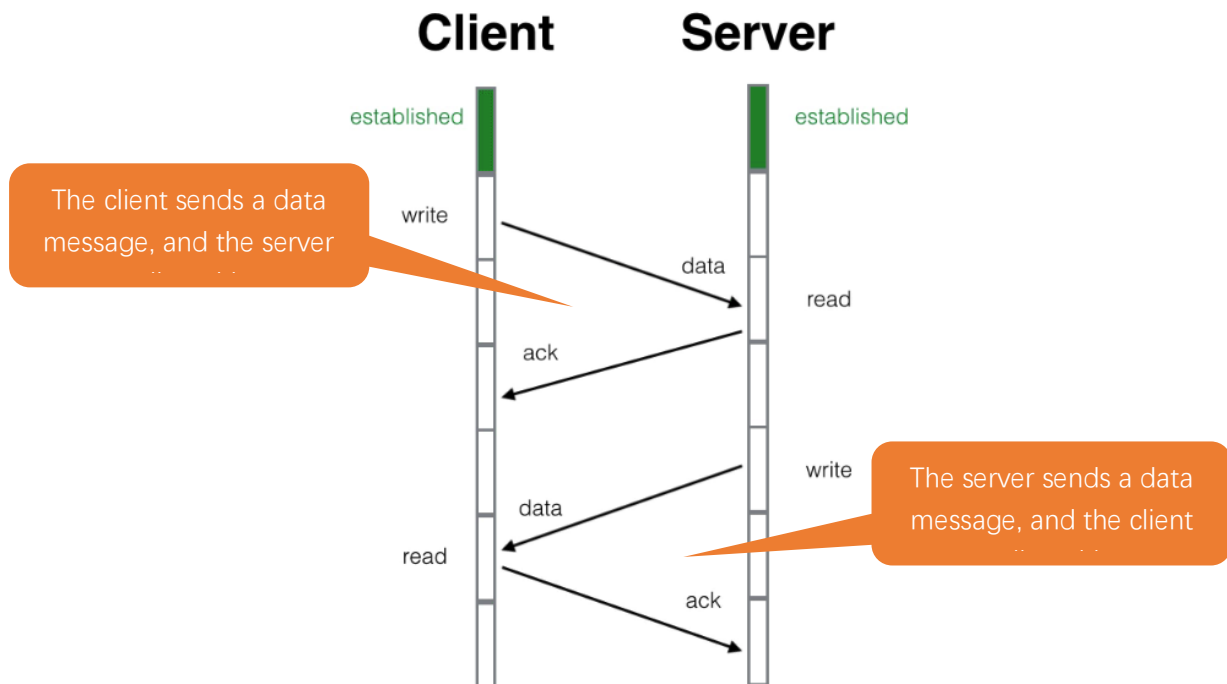
Before transmitting data, TCP needs to establish a logical connection between the sending end and the receiving end. It provides reliable and error-free data transmission between the two computers. In the TCP connection, the client and the server must be clarified. The client sends a connection request to the server, and each time such a request is proposed, a "three-times handshake" is required.

Three-times handshake: In the TCP protocol, during the preparation phase of sending data, the client and the server interact three times to ensure the reliability of the connection, which is called "three-times handshake". The first handshake, the client sends a connection request to the server and waits for the server to confirm. The second handshake, the server sends a response back to the client informing that it has received the connection request.

The third handshake, the client sends a confirmation message to the server again to confirm the connection.



TCP is a connection-oriented, low-level transmission control protocol. After TCP establishes a connection, the client and server can send and receive messages to each other, and the connection will always exist as long as the client or server does not initiate disconnection. Each time one party sends a message, the other party will reply with an ack signal.



Install Processing

In this tutorial, we use Processing to build a simple TCP/IP communication platform.

If you've not installed Processing, you can download it by clicking <https://processing.org/download/>. You can choose an appropriate version to download according to your PC system.

Processing p5.js Processing.py Processing for Android Processing for Pi Processing Foundation

Processing

Download Processing. Processing is available for Linux, Mac OS X, and Windows. Select your choice to download the software below.

3.5.4 (17 January 2020)

[Windows 64-bit](#) [Linux 64-bit](#) [Mac OS X](#)

[Windows 32-bit](#)

» [Github](#) [Report Bugs](#) [Wiki](#) [Supported Platforms](#)

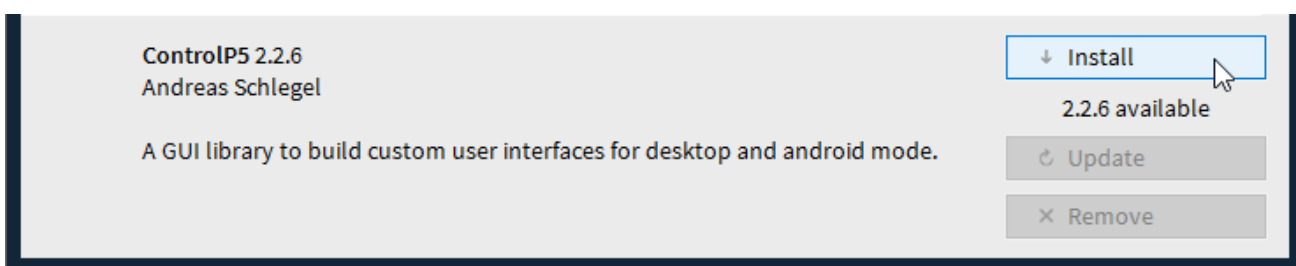
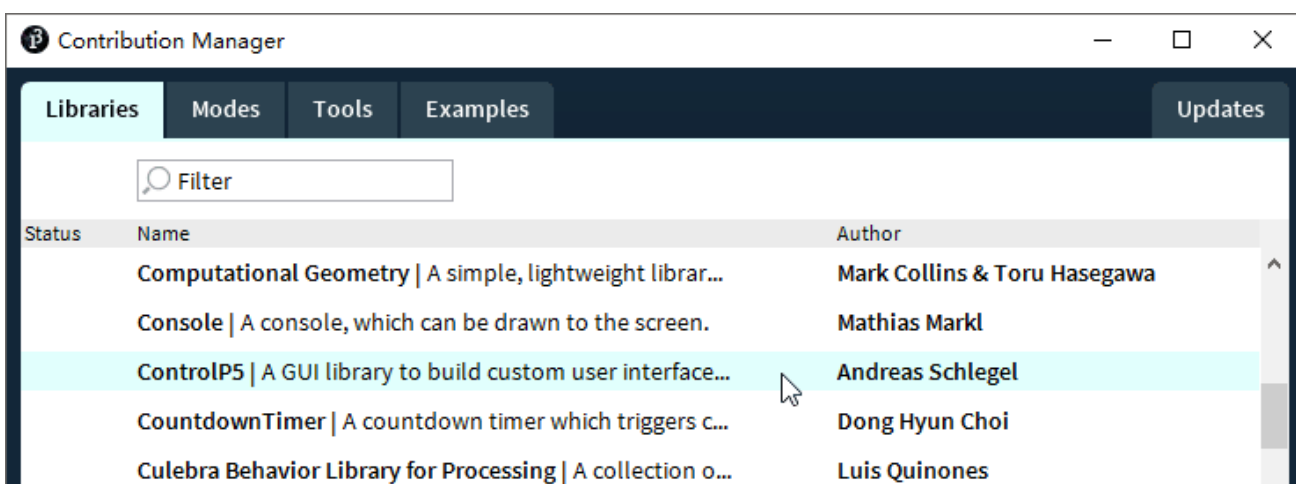
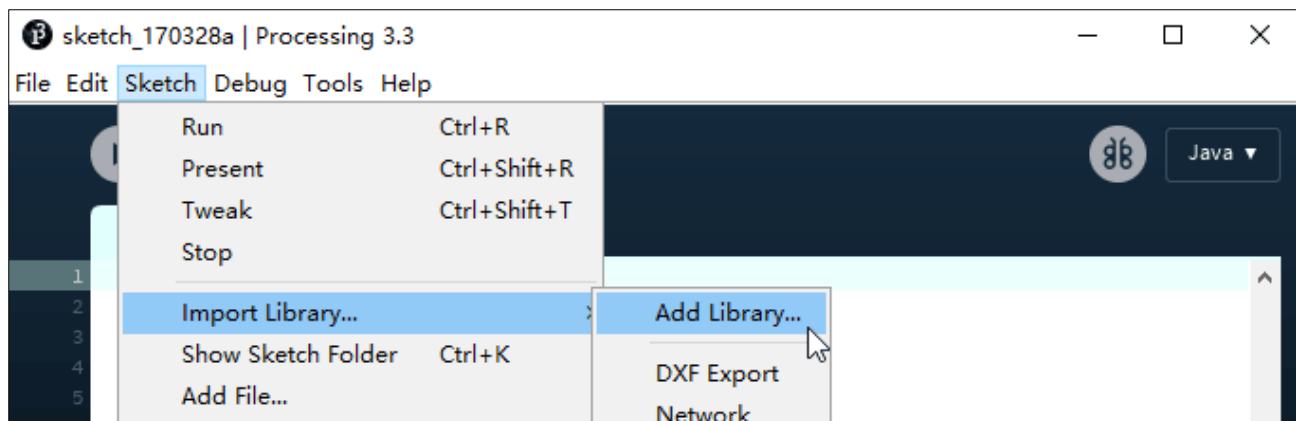
Read about the [changes in 3.0](#). The [list of revisions](#) covers the differences between releases in detail.

Unzip the downloaded file to your computer. Click "processing.exe" as the figure below to run this software.

	core	2020/1/17 12:16
	java	2020/1/17 12:17
	lib	2020/1/17 12:16
	modes	2020/1/17 12:16
	tools	2020/1/17 12:16
	processing.exe	2020/1/17 12:16
	processing-java.exe	2020/1/17 12:16
	revisions.txt	2020/1/17 12:16

Use Server mode for communication

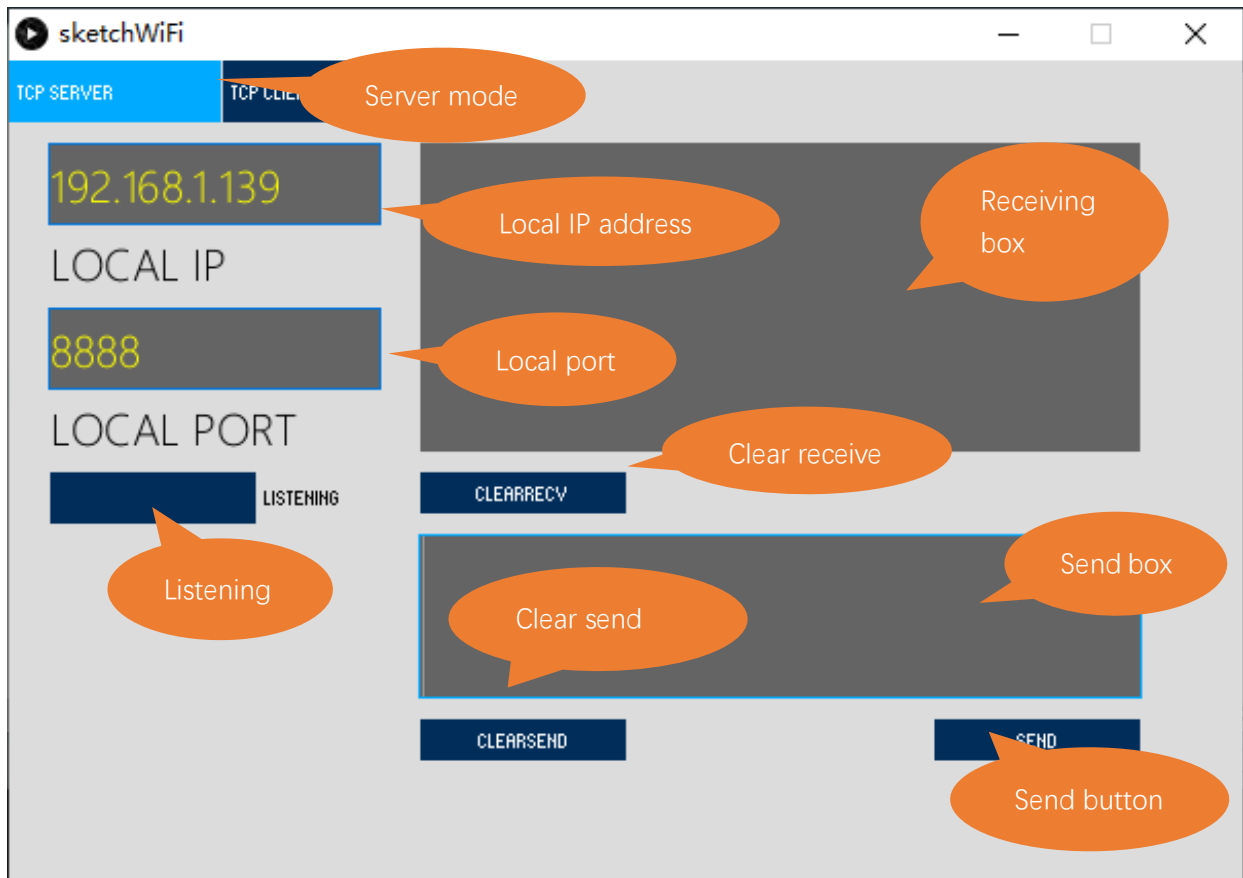
Install ControlP5.



Open the "Freenove_ESP32_S3_WROVER_Board\Sketches\Sketches\Sketch_06.1_WiFiClient\sketchWiFi\sketchWiFi.pde", and click "Run".

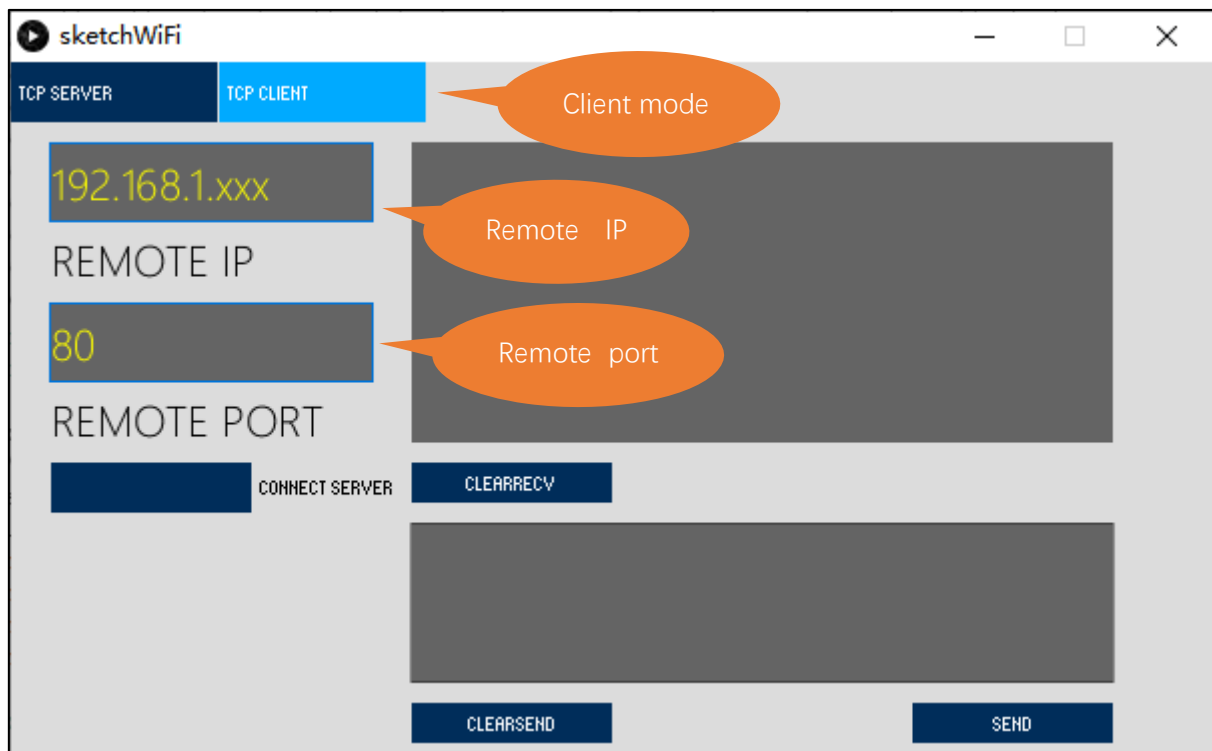


The new pop-up interface is as follows. If ESP32-S3 is used as client, select TCP SERVER mode for sketchWiFi.



When sketchWiFi selects TCP SERVER mode, ESP32-S3 Sketch needs to be changed according to sketchWiFi's displaying of LOCAL IP or LOCAL PORT.

If ESP32-S3 serves as server, select TCP CLIENT mode for sketchWiFi.



When sketchWiFi selects TCP CLIENT mode, the LOCAL IP and LOCAL PORT of sketchWiFi need to be changed according to the IP address and port number printed by the serial monitor.

Mode selection: select **Server mode/Client mode**.

IP address: In server mode, this option does not need to be filled in, and the computer will automatically obtain the IP address.

In client mode, fill in the remote IP address to be connected.

Port number: In server mode, fill in a port number for client devices to make an access connection.

In client mode, fill in port number given by the Server devices to make an access connection.

Start button: In server mode, push the button, then the computer will serve as server and open a port number for client to make access connection. During this period, the computer will keep monitoring.

In client mode, before pushing the button, please make sure the server is on, remote IP address and remote port number is correct; push the button, and the computer will make access connection to the remote port number of the remote IP as a client.

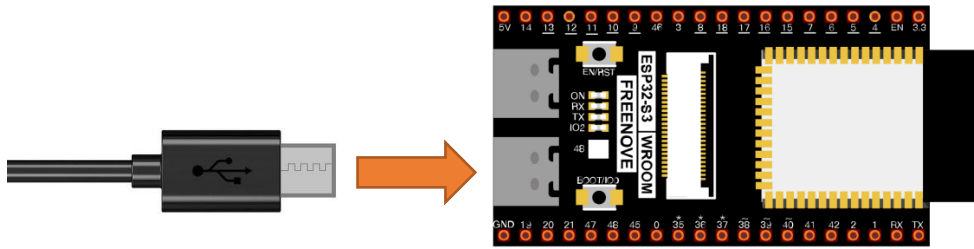
clear receive: clear out the content in the receiving text box

clear send: clear out the content in the sending text box

Sending button: push the sending button, the computer will send the content in the text box to others.

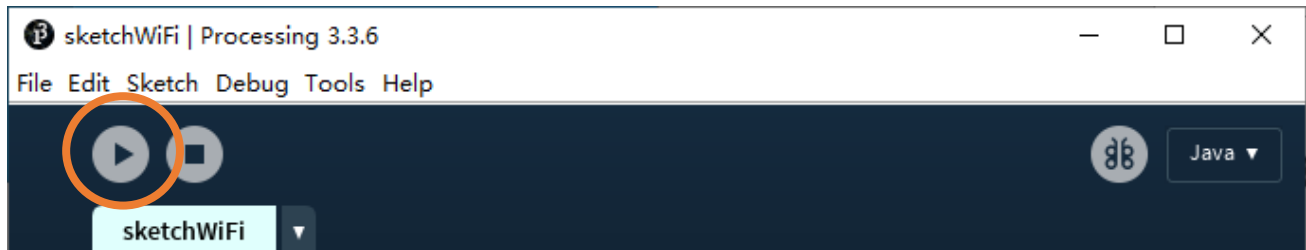
Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.

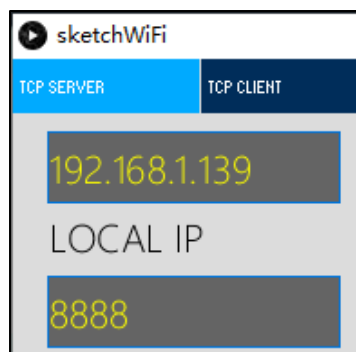


Sketch

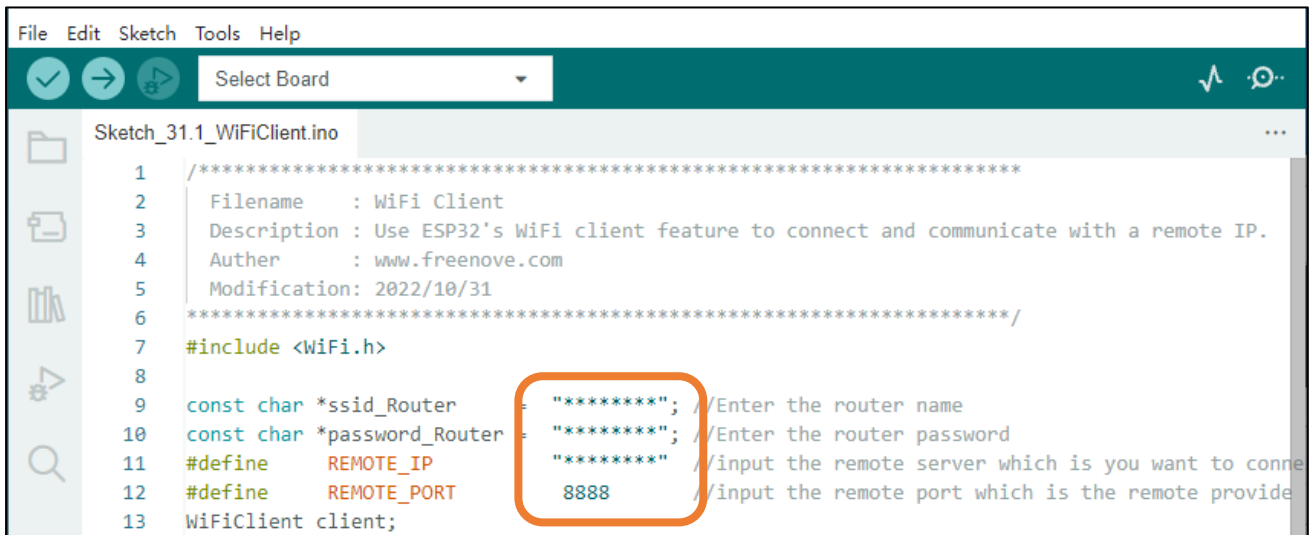
Before running the Sketch, please open "sketchWiFi.pde." first, and click "Run".



The newly pop up window will use the computer's IP address by default and open a data monitor port.



Next, open Sketch_06.1_WiFiClient.ino. Before running it, please change the following information based on "LOCAL IP" and "LOCAL PORT" in the figure above.

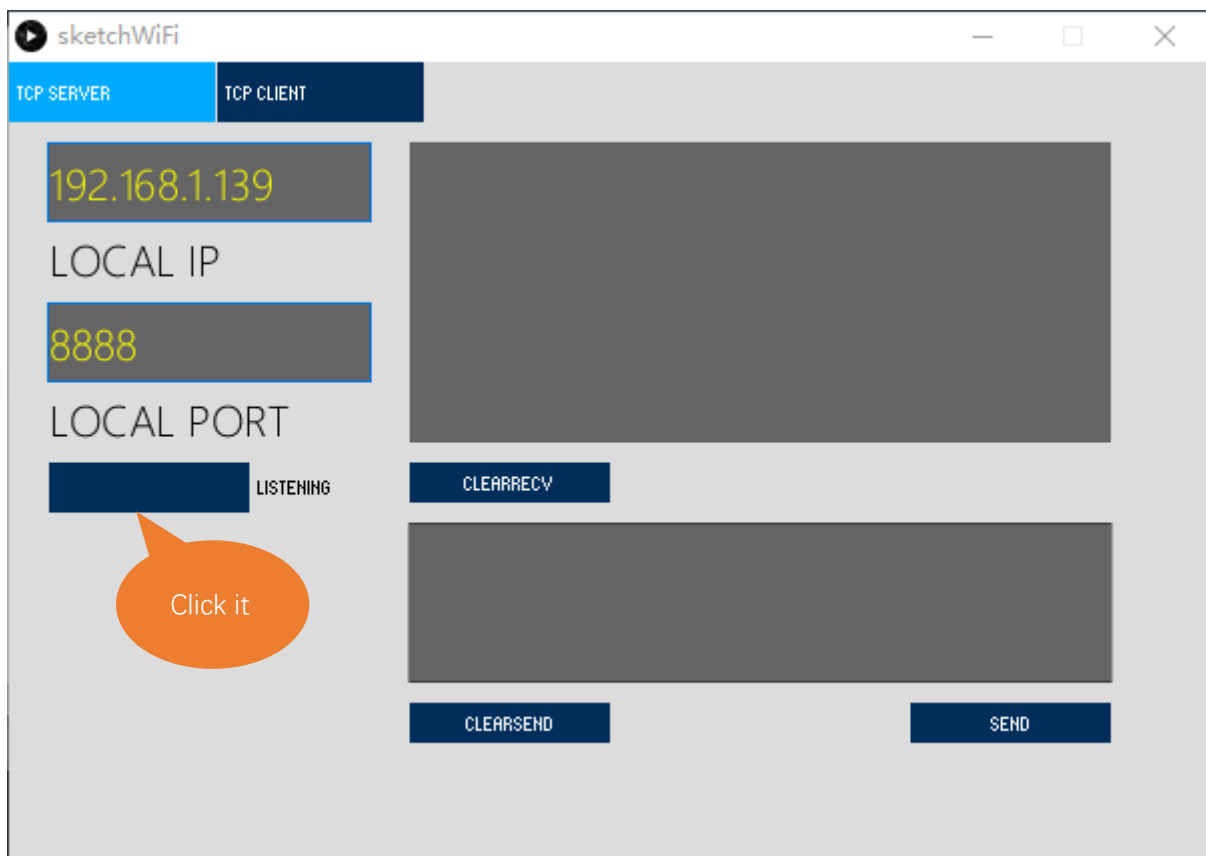


```

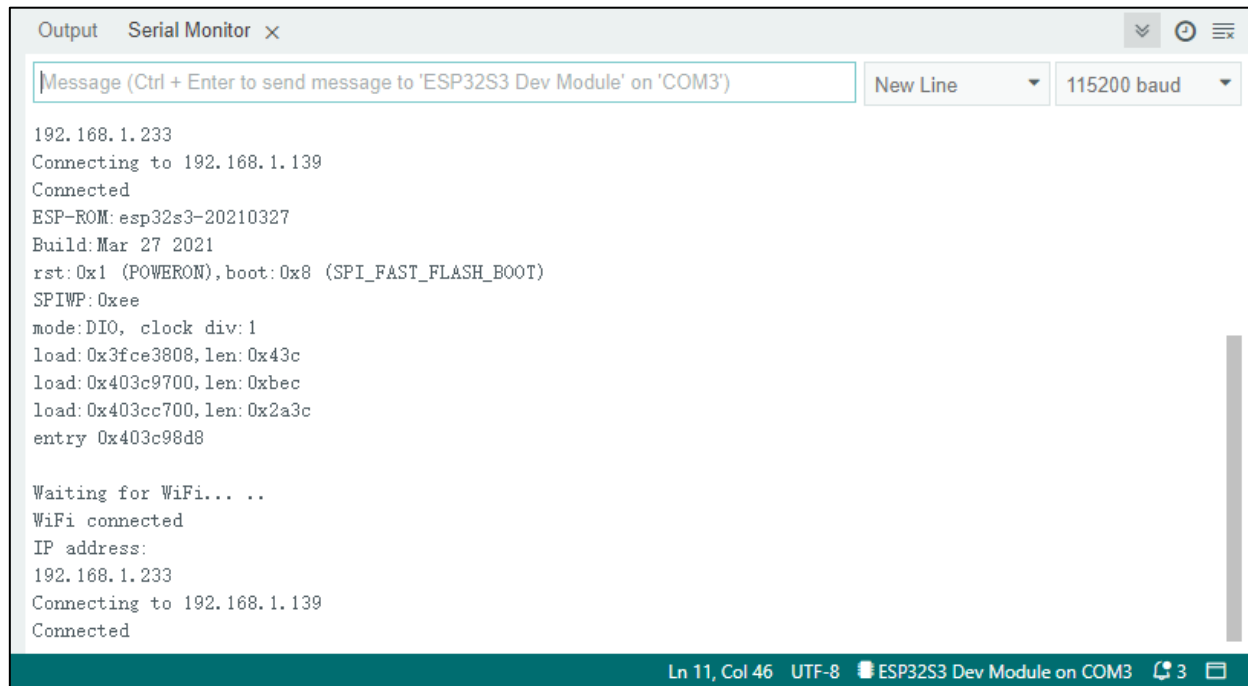
1  /*****
2  Filename   : WiFi Client
3  Description : Use ESP32's WiFi client feature to connect and communicate with a remote IP.
4  Author    : www.freenove.com
5  Modification: 2022/10/31
6  *****/
7  #include <WiFi.h>
8
9  const char *ssid_Router = "*****"; //Enter the router name
10 const char *password_Router = "*****"; //Enter the router password
11 #define REMOTE_IP "*****" //input the remote server which is you want to connect
12 #define REMOTE_PORT 8888 //input the remote port which is the remote provider
13 WiFiClient client;
  
```

REMOTE_IP needs to be filled in according to the interface of sketchWiFi.pde. Taking this tutorial as an example, its REMOTE_IP is "192.168.1.133". Generally, by default, the ports do not need to change its value.

Click LISTENING, turn on TCP SERVER's data listening function and wait for ESP32-S3 to connect.



Compile and upload code to ESP32-S3 WROOM, open the serial monitor and set the baud rate to 115200. ESP32-S3 connects router, obtains IP address and sends access request to server IP address on the same LAN till the connection is successful. When connect successfully, ESP32-S3 can send messages to server.



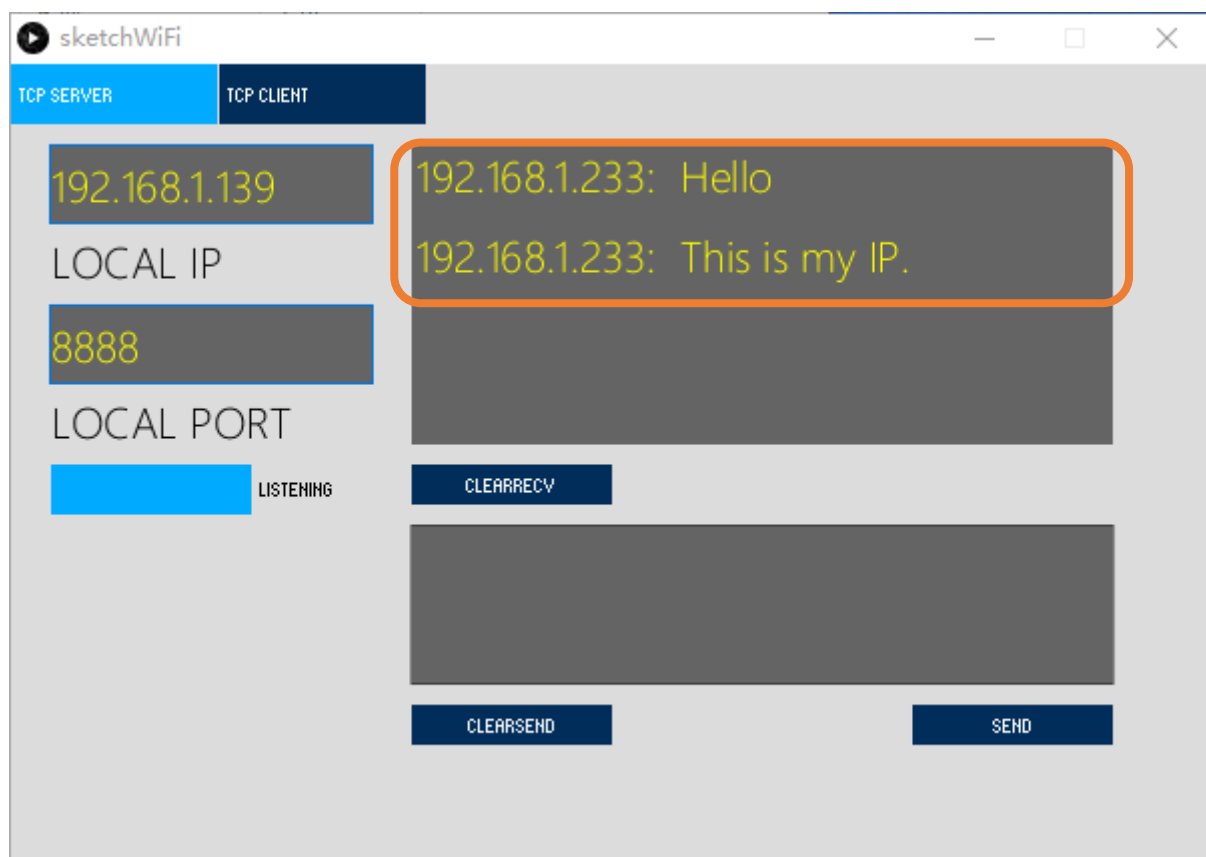
```
Output Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3') New Line 115200 baud

192.168.1.233
Connecting to 192.168.1.139
Connected
ESP-ROM: esp32s3-20210327
Build: Mar 27 2021
rst: 0x1 (POWERON), boot: 0x8 (SPI_FAST_FLASH_BOOT)
SPIWP: 0xee
mode: DIO, clock div: 1
load: 0x3fce3808, len: 0x43c
load: 0x403c9700, len: 0xbec
load: 0x403cc700, len: 0x2a3c
entry 0x403c98d8

Waiting for WiFi...
WiFi connected
IP address:
192.168.1.233
Connecting to 192.168.1.139
Connected

Ln 11, Col 46 UTF-8 ESP32S3 Dev Module on COM3 3
```

ESP32-S3 connects with TCP SERVER, and TCP SERVER receives messages from ESP32-S3, as shown in the figure below.



Sketch_06.1_As_Client

The following is the program code:

```
1  #include <WiFi.h>
2
3  const char *ssid_Router    = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  #define    REMOTE_IP        "*****" //input the remote server which is you want to connect
6  #define    REMOTE_PORT      8888     //input the remote port which is the remote provide
7  WiFiClient client;
8
9  void setup() {
10     Serial.begin(115200);
11     delay(10);
12
13     WiFi.begin(ssid_Router, password_Router);
14     Serial.print("\nWaiting for WiFi... ");
15     while (WiFi.status() != WL_CONNECTED) {
16         Serial.print(".");
17         delay(500);
18     }
19     Serial.println("");
20     Serial.println("WiFi connected");
21     Serial.println("IP address: ");
22     Serial.println(WiFi.localIP());
23     delay(500);
24
25     Serial.print("Connecting to ");
26     Serial.println(REMOTE_IP);
27
28     while (!client.connect(REMOTE_IP, REMOTE_PORT)) {
29         Serial.println("Connection failed.");
30         Serial.println("Waiting a moment before retrying...");
31     }
32     Serial.println("Connected");
33     client.print("Hello\n");
34     client.print("This is my IP.\n");
35 }
36
37 void loop() {
38     if (client.available() > 0) {
39         delay(20);
40         //read back one line from the server
41         String line = client.readString();
42         Serial.println(REMOTE_IP + String(":") + line);
```

```

42     }
43     if (Serial.available() > 0) {
44         delay(20);
45         String line = Serial.readString();
46         client.print(line);
47     }
48     if (client.connected () == 0) {
49         client.stop();
50         WiFi.disconnect();
51     }
52 }

```

Add WiFi function header file.

```

1  #include <WiFi.h>

```

Enter the actual router name, password, remote server IP address, and port number.

```

3  const char *ssid_Router      = "*****"; //Enter the router name
4  const char *password_Router = "*****"; //Enter the router password
5  #define      REMOTE_IP        "*****" //input the remote server which is you want to connect
6  #define      REMOTE_PORT      8888     //input the remote port which is the remote provide

```

Apply for the method class of WiFiClient.

```

7  WiFiClient client;

```

Connect specified WiFi until it is successful. If the name and password of WiFi are correct but it still fails to connect, please push the reset key.

```

13  WiFi.begin(ssid_Router, password_Router);
14  Serial.print("\nWaiting for WiFi... ");
15  while (WiFi.status() != WL_CONNECTED) {
16      Serial.print(".");
17      delay(500);
18  }

```

Send connection request to remote server until connect successfully. When connect successfully, print out the connecting prompt on the serial monitor and send messages to remote server.

```

28  while (!client.connect(REMOTE_IP, REMOTE_PORT)) { //Connect to Server
29      Serial.println("Connection failed.");
30      Serial.println("Waiting a moment before retrying...");
31  }
32  Serial.println("Connected");
33  client.print("Hello\n");

```

When ESP32-S3 receive messages from servers, it will print them out via serial port; Users can also send messages to servers from serial port.

```

37  if (client.available() > 0) {
38      delay(20);
39      //read back one line from the server
40      String line = client.readString();
41      Serial.println(REMOTE_IP + String(":") + line);
42  }

```

```

43     if (Serial.available() > 0) {
44         delay(20);
45         String line = Serial.readString();
46         client.print(line);
47     }

```

If the server is disconnected, turn off WiFi of ESP32-S3.

```

48     if (client.connected () == false) {
49         client.stop();
50         WiFi.disconnect();
51     }

```

Reference

Class Client

Every time when using Client, you need to include header file "WiFi.h."

connect(ip, port, timeout)/connect(*host, port, timeout): establish a TCP connection.

ip, *host: ip address of target server

port: port number of target server

timeout: connection timeout

connected(): judge whether client is connecting. If return value is 1, then connect successfully; If return value is 0, then fail to connect.

stop(): stop tcp connection

print(): send data to server connecting to client

available(): return to the number of bytes readable in receive buffer, if no, return to 0 or -1.

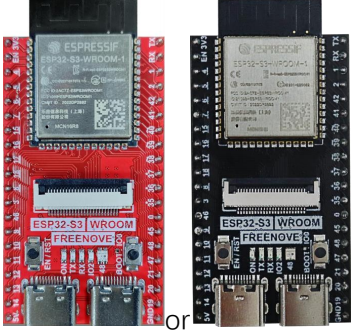
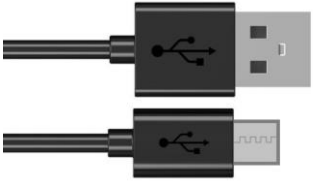
read(): read one byte of data in receive buffer

readString(): read string in receive buffer

Project 6.2 As Server

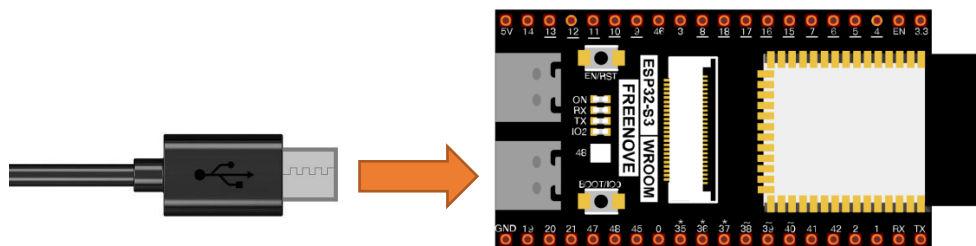
In this section, ESP32-S3 is used as a server to wait for the connection and communication of client on the same LAN.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
---	--

Circuit

Connect Freenove ESP32-S3 to the computer using a USB cable.



Sketch

Before running Sketch, please modify the contents of the box below first.

Sketch_06.2_As_Server

```

8  #include <WiFi.h>
9
10 #define port 80
11 const char *ssid_Router      "*****"; //input your wifi name
12 const char *password_Router  "*****"; //input your wifi passwords
13 WiFiServer server(port);
14
15 void setup()
16 {
17     Serial.begin(115200);
18     Serial.printf("\nConnecting to ");
19     Serial.println(ssid_Router);
20     WiFi.disconnect();
21     WiFi.begin(ssid_Router, password_Router);

```

Compile and upload code to ESP32-S3 WROOM board, open the serial monitor and set the baud rate to 115200. Turn on server mode for ESP32-S3, waiting for the connection of other devices on the same LAN. Once a device connects to server successfully, they can send messages to each other.

If the ESP32-S3 fails to connect to router, press the reset button as shown below and wait for ESP32-S3 to run again.

The screenshot shows the Serial Monitor window for COM6. The output displays the following sequence of events:

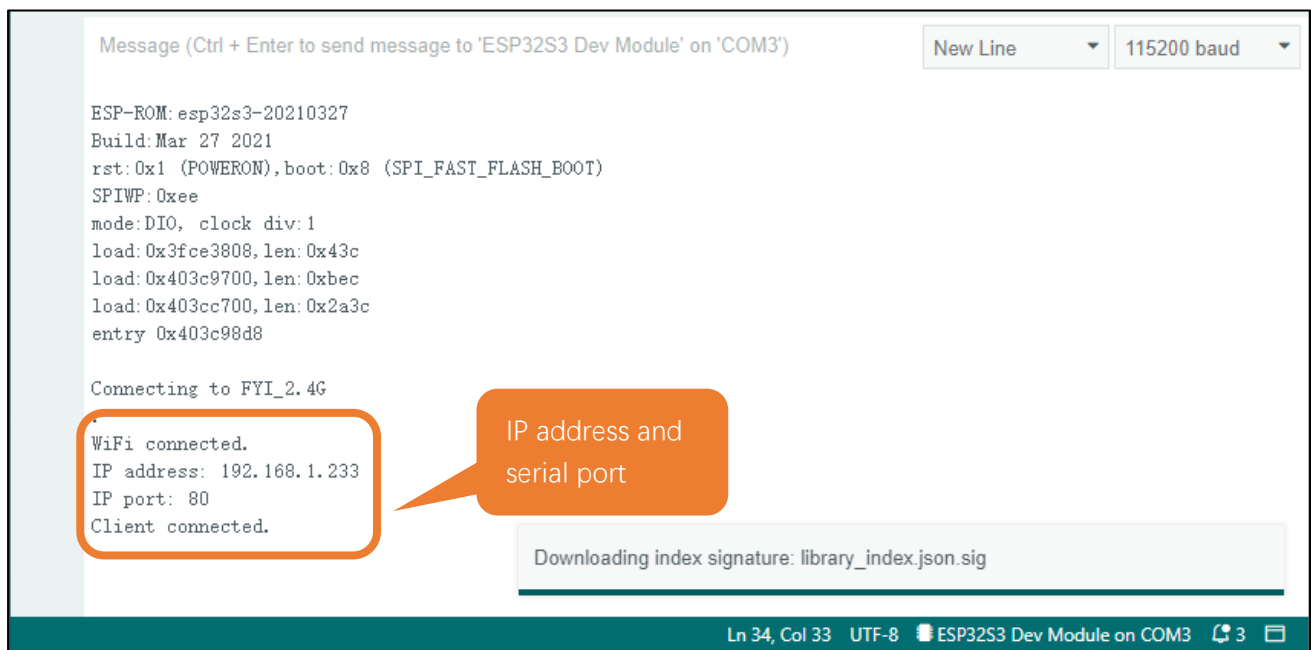
```

09:09:32.062 -> e 'Ju$0$02$LO$0$&Q7$H$0$:0$0B$0$Cn$Q_RQ*0U$I$0$:0TB$0$I$0MT_0*MH_$0Uj
09:09:32.062 -> c$K$0$0$0b0ZA$WP'De$,k$E$0$0$0$,0E$0$:0$0,dE$0$:0$0i$A_$.0$0x$0b$0$0rv'$A0$0$0r$'$A0$0$0iDQ$0$0
09:09:32.093 -> $0$0$ 0$008$0a$
09:09:33.766 -> Connecting to FYI_2.4G
09:09:34.950 -> .....

```

The bottom of the window shows the configuration settings: ☐ Autoscroll, ☐ Show timestamp, Newline, 115200 baud, and Clear output.

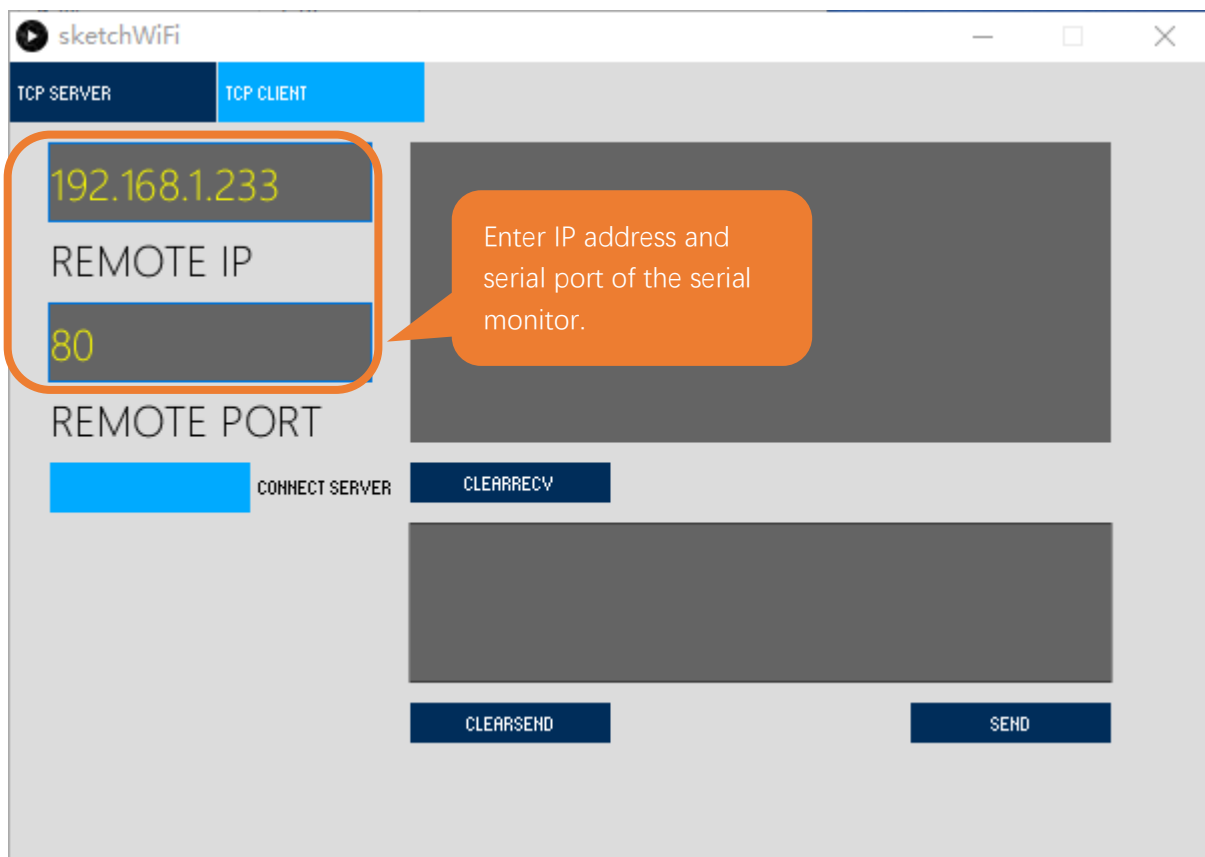
Serial Monitor



Processing:

Open the "Freenove_ESP32_S3_WROVER_Board\Sketches\Sketches\Sketch_06.2_WiFiServer\sketchWiFi\sketchWiFi.pde".

Based on the messages printed by the serial monitor, enter correct IP address and serial port in Processing to establish connection and make communication.



The following is the program code:

```

1  #include <WiFi.h>
2
3  #define port 80
4  const char *ssid_Router      = "*****"; //input your wifi name
5  const char *password_Router = "*****"; //input your wifi passwords
6  WiFiServer server(port);
7
8  void setup()
9  {
10     Serial.begin(115200);
11     Serial.printf("\nConnecting to ");
12     Serial.println(ssid_Router);
13     WiFi.disconnect();
14     WiFi.begin(ssid_Router, password_Router);
15     delay(1000);
16     while (WiFi.status() != WL_CONNECTED) {
17         delay(500);
18         Serial.print(".");
19     }
20     Serial.println("");
21     Serial.println("WiFi connected.");
22     Serial.print("IP address: ");
23     Serial.println(WiFi.localIP());
24     Serial.printf("IP port: %d\n",port);
25     server.begin(port);
26     WiFi.setAutoReconnect(true);
27 }
28
29 void loop() {
30     WiFiClient client = server.accept();           // listen for incoming clients
31     if (client) {                                   // if you get a client
32         Serial.println("Client connected.");
33         while (client.connected()) {                // loop while the client's connected
34             if (client.available()) {                // if there's bytes to read from the
client
35                 Serial.println(client.readStringUntil('\n')); // print it out the serial monitor
36                 while(client.read()>0);              // clear the wifi receive area cache
37             }
38             if(Serial.available()){                  // if there's bytes to read from the
serial monitor
39                 client.print(Serial.readStringUntil('\n')); // print it out the client.
40                 while(Serial.read()>0);              // clear the wifi receive area cache
41             }

```

```

42     }
43     client.stop();                // stop the client connecting.
44     Serial.println("Client Disconnected.");
45 }
46 }

```

Apply for method class of WiFiServer.

```

6  WiFiServer server(port);        //Apply for a Server object whose port number is 80

```

Connect specified WiFi until it is successful. If the name and password of WiFi are correct but it still fails to connect, please push the reset key.

```

13  WiFi.disconnect();
14  WiFi.begin(ssid_Router, password_Router);
15  delay(1000);
16  while (WiFi.status() != WL_CONNECTED) {
17      delay(500);
18      Serial.print(".");
19  }
20  Serial.println("");
21  Serial.println("WiFi connected.");

```

Print out the IP address and port number of ESP32-S3.

```

22  Serial.print("IP address: ");
23  Serial.println(WiFi.localIP());    //print out IP address of ESP32-S3
24  Serial.printf("IP port: %d\n",port); //Print out ESP32-S3's port number

```

Turn on server mode of ESP32-S3, turn on automatic reconnection.

```

25  server.begin();                  //Turn ON ESP32-S3 as Server mode
26  WiFi.setAutoReconnect(true);

```

When ESP32-S3 receive messages from servers, it will print them out via serial port; Users can also send messages to servers from serial port.

```

34  if (client.available()) {        // if there's bytes to read from the
client
35      Serial.println(client.readStringUntil('\n')); // print it out the serial monitor
36      while(client.read()>0);        // clear the wifi receive area cache
37  }
38  if(Serial.available()){          // if there's bytes to read from the
serial monitor
39      client.print(Serial.readStringUntil('\n')); // print it out the client.
40      while(Serial.read()>0);        // clear the wifi receive area cache
41  }

```

Reference

Class Server

Every time use Server functionality, we need to include header file "WiFi.h".

WiFiServer(uint16_t port=80, uint8_t max_clients=4): create a TCP Server.

port: ports of Server; range from 0 to 65535 with the default number as 80.

max_clients: maximum number of clients with default number as 4.

begin(port): start the TCP Server.

port: ports of Server; range from 0 to 65535 with the default number as 0.

setNoDelay(bool nodelay): whether to turn off the delay sending functionality.

nodelay: true stands for forbidden Nagle algorithm.

close(): close tcp connection.

stop(): stop tcp connection.

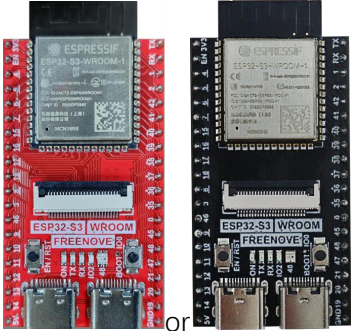
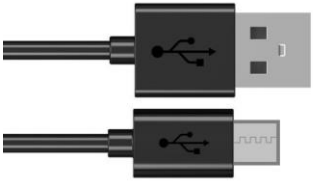
Chapter 7 Camera Web Server

In this section, we'll use ESP32-S3's video function as an example to study.

Project 7.1 Camera Web Server

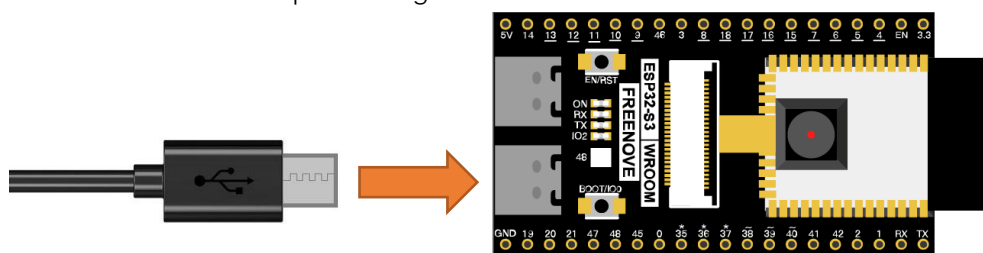
Connect ESP32-S3 using USB and check its IP address through serial monitor. Use web page to access IP address to obtain video and image data.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
--	---

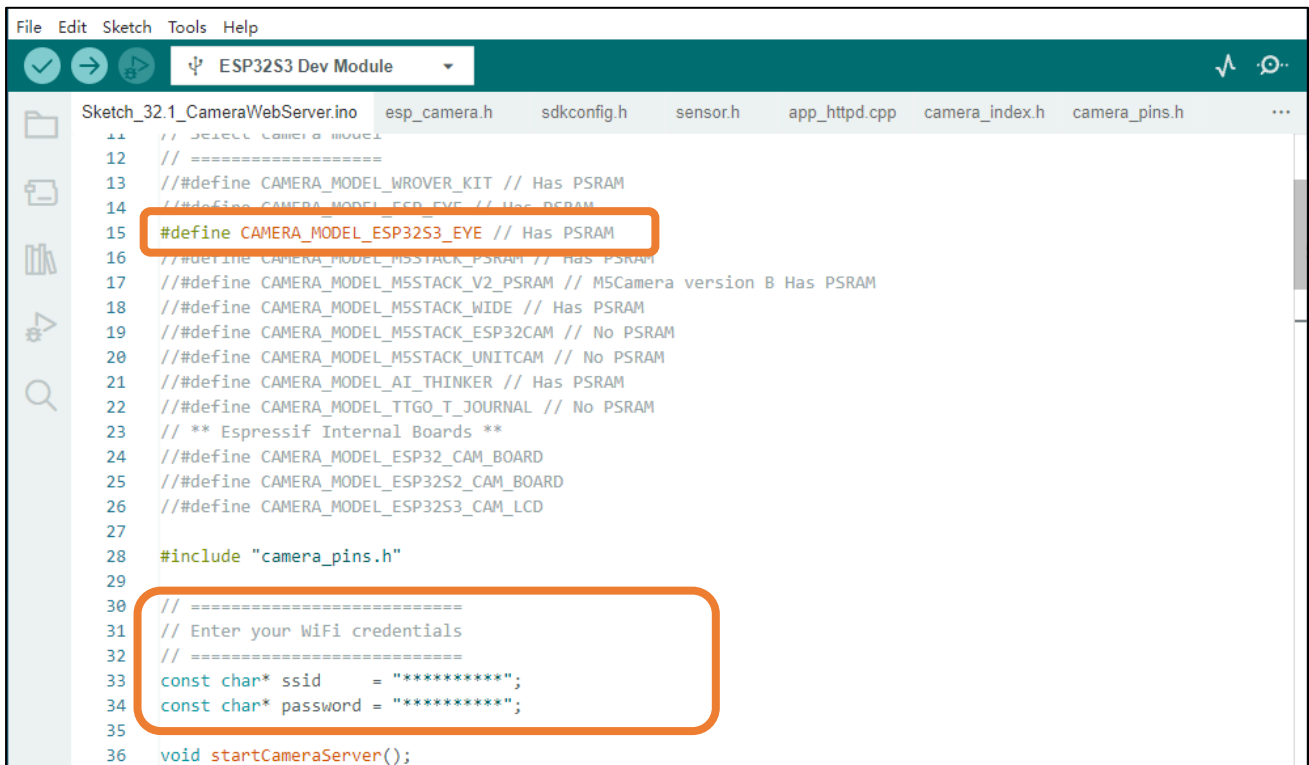
Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Sketch_07.1_As_CameraWebServer



```

11 // =====
12 // =====
13 // #define CAMERA_MODEL_WROVER_KIT // Has PSRAM
14 // #define CAMERA_MODEL_ESP_EYE // Has PSRAM
15 #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
16 // #define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM
17 // #define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has PSRAM
18 // #define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM
19 // #define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM
20 // #define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM
21 // #define CAMERA_MODEL_AI_THINKER // Has PSRAM
22 // #define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM
23 // ** Espressif Internal Boards **
24 // #define CAMERA_MODEL_ESP32_CAM_BOARD
25 // #define CAMERA_MODEL_ESP32S2_CAM_BOARD
26 // #define CAMERA_MODEL_ESP32S3_CAM_LCD
27
28 #include "camera_pins.h"
29
30 // =====
31 // Enter your WiFi credentials
32 // =====
33 const char* ssid = "*****";
34 const char* password = "*****";
35
36 void startCameraServer();
  
```

Before running the program, please modify your router's name and password in the box shown in the illustration above to make sure that your Sketch can compile and work successfully.

Compile and upload codes to ESP32-S3, open the serial monitor and set the baud rate to 115200, and the serial monitor will print out a network link address.



```

Output Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3') New Line 115200 baud

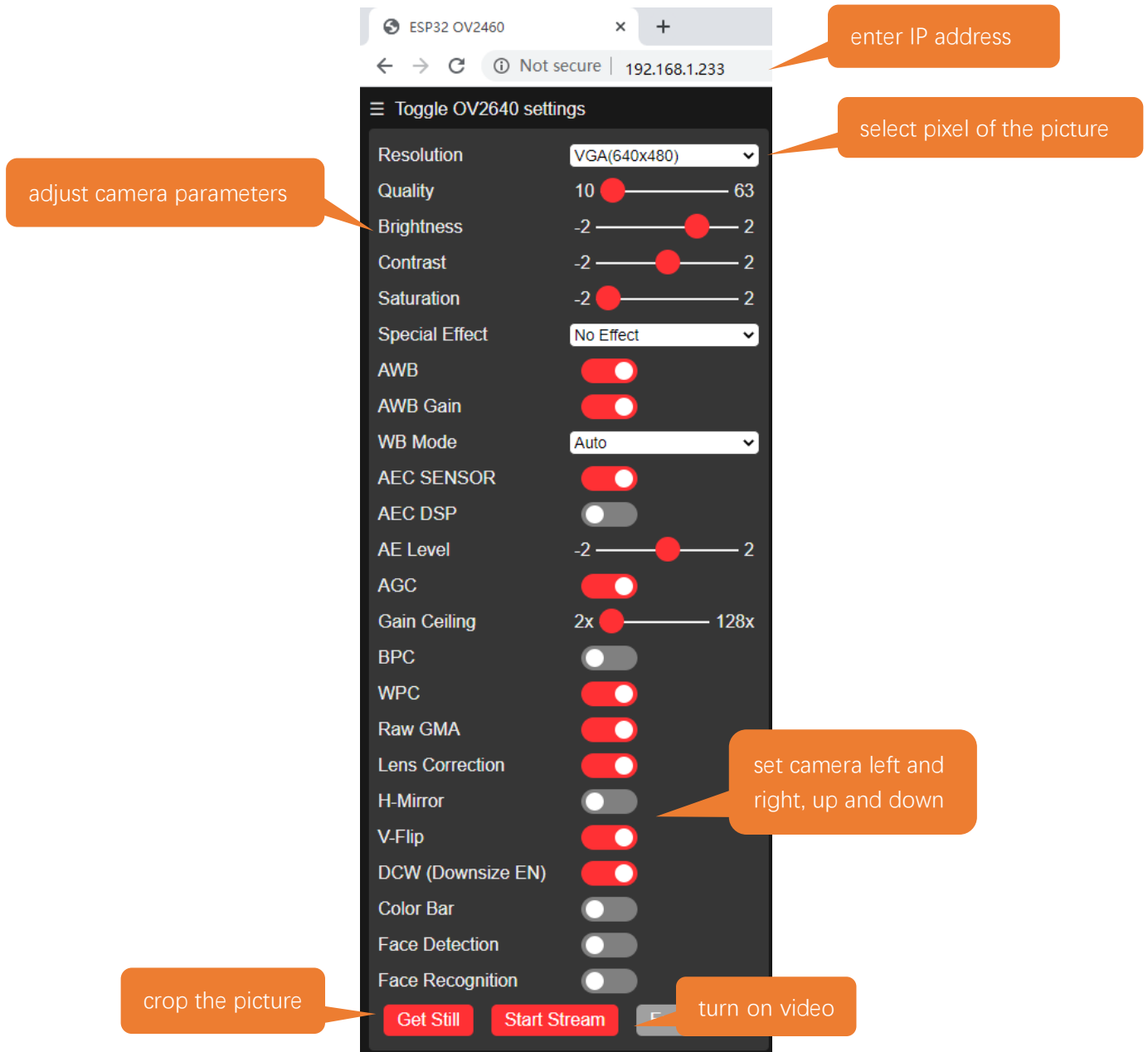
ESP-ROM: esp32s3-20210327
Build: Mar 27 2021
rst:0x1 (POWERON),boot:0x8 (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fce3808, len:0x43c
load:0x403c9700, len:0xbec
load:0x403cc700, len:0x2a3c
entry 0x403c98d8

..
WiFi connected
Camera Ready! Use 'http://192.168.1.233' to connect
  
```

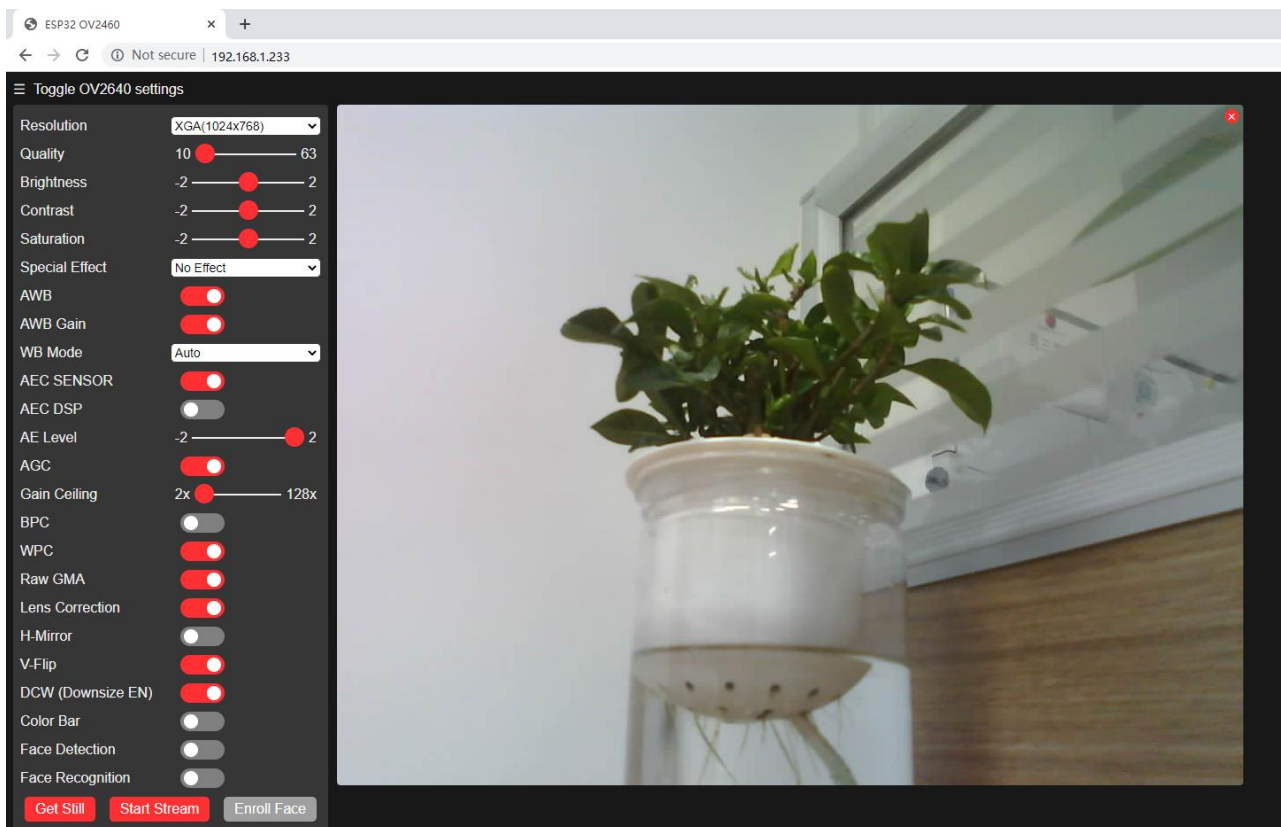
If your ESP32-S3 has been in the process of connecting to router, but the information above has not been printed out, please re-check whether the router name and password have been entered correctly and press the reset key on ESP32-S3 WROOM to wait for a successful connection prompt.

Open a web browser, enter the IP address printed by the serial monitor in the address bar, and access it.

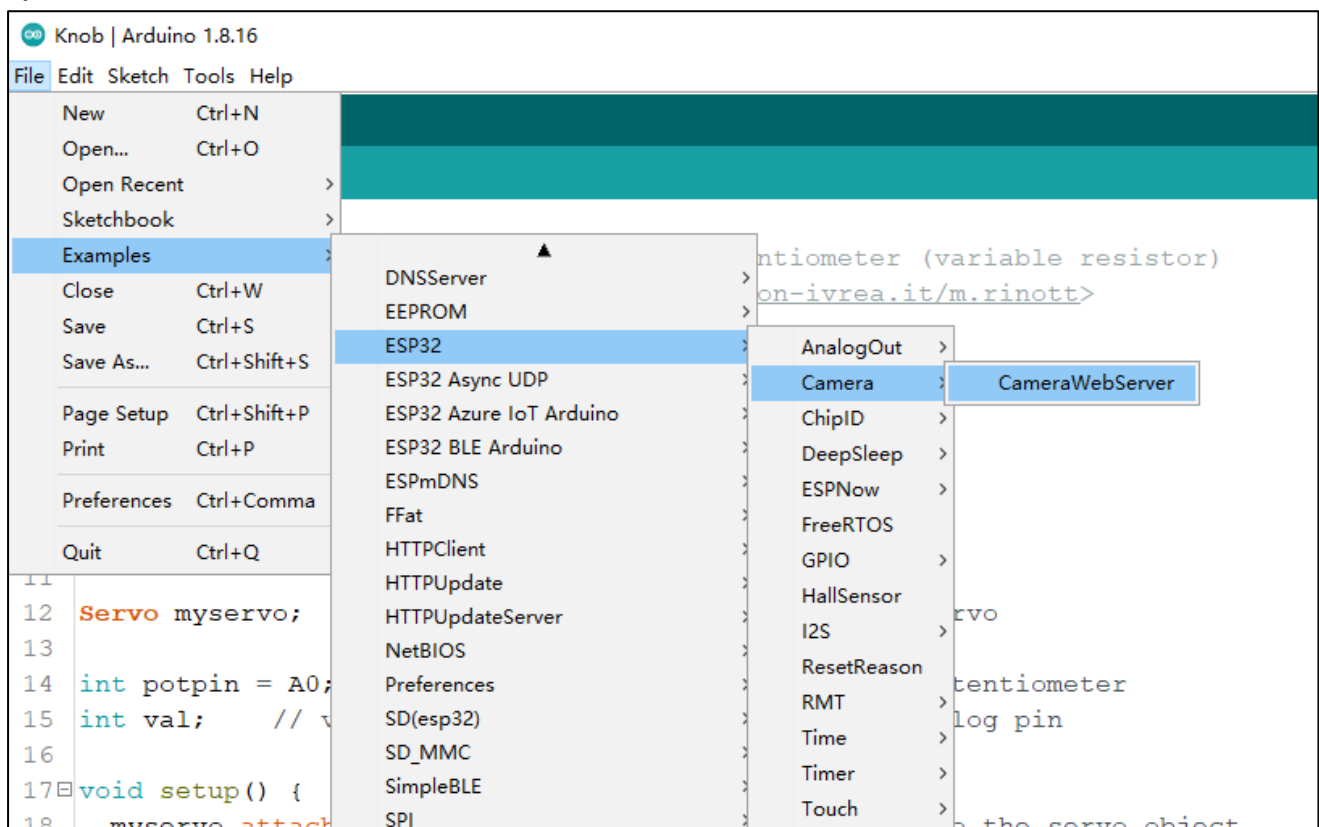
Taking the Google browser as an example, here's what the browser prints out after successful access to ESP32-S3's IP.



Click on Start Stream. The effect is shown in the image below.



Note: If sketch compilation fails due to ESP32-S3 support package, follow the steps of the image to open the CameraWebServer. This sketch is the same as described in the tutorial above.



The following is the main program code. You need include other code files in the same folder when write your own code.

```
1  #include "esp_camera.h"
2  #include <WiFi.h>
3  #include "board_config.h"
4  // =====
5  // Select camera model
6  // =====
7  // #define CAMERA_MODEL_WROVER_KIT // Has PSRAM
8  // #define CAMERA_MODEL_ESP_EYE // Has PSRAM
9  #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
10 // #define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM
11 // #define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has PSRAM
12 // #define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM
13 // #define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM
14 // #define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM
15 // #define CAMERA_MODEL_AI_THINKER // Has PSRAM
16 // #define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM
17 // ** Espressif Internal Boards **
18 // #define CAMERA_MODEL_ESP32_CAM_BOARD
19 // #define CAMERA_MODEL_ESP32S2_CAM_BOARD
20 // #define CAMERA_MODEL_ESP32S3_CAM_LCD
21
22 #include "camera_pins.h"
23
24 // =====
25 // Enter your WiFi credentials
26 // =====
27 const char* ssid = "*****";
28 const char* password = "*****";
29 camera_config_t config;
30
31 void startCameraServer();
32 void camera_init();
33
34 void setup() {
35     Serial.begin(115200);
36     Serial.setDebugOutput(true);
37     Serial.println();
38
39     camera_init();
40
41     WiFi.begin(ssid, password);
42     WiFi.setSleep(false);
```

```
43
44 while (WiFi.status() != WL_CONNECTED) {
45     delay(500);
46     Serial.print(".");
47 }
48 while (WiFi.STA.hasIP() != true) {
49     Serial.print(".");
50     delay(500);
51 }
52 Serial.println("");
53 Serial.println("WiFi connected");
54
55 startCameraServer();
56
57 Serial.print("Camera Ready! Use 'http://");
58 Serial.print(WiFi.localIP());
59 Serial.println("' to connect");
60 }
61
62 void loop() {
63     // Do nothing. Everything is done in another task by the web server
64     delay(10000);
65 }
66
67 void camera_init() {
68     config.ledc_channel = LEDC_CHANNEL_0;
69     config.ledc_timer = LEDC_TIMER_0;
70     config.pin_d0 = Y2_GPIO_NUM;
71     config.pin_d1 = Y3_GPIO_NUM;
72     config.pin_d2 = Y4_GPIO_NUM;
73     config.pin_d3 = Y5_GPIO_NUM;
74     config.pin_d4 = Y6_GPIO_NUM;
75     config.pin_d5 = Y7_GPIO_NUM;
76     config.pin_d6 = Y8_GPIO_NUM;
77     config.pin_d7 = Y9_GPIO_NUM;
78     config.pin_xclk = XCLK_GPIO_NUM;
79     config.pin_pclk = PCLK_GPIO_NUM;
80     config.pin_vsync = VSYNC_GPIO_NUM;
81     config.pin_href = HREF_GPIO_NUM;
82     config.pin_sccb_sda = SIOD_GPIO_NUM;
83     config.pin_sccb_scl = SIOC_GPIO_NUM;
84     config.pin_pwdn = PWDN_GPIO_NUM;
85     config.pin_reset = RESET_GPIO_NUM;
86     config.xclk_freq_hz = 10000000;
```

```
87 config.frame_size = FRAMESIZE_QVGA;
88 config.pixel_format = PIXFORMAT_JPEG; // for streaming
89 config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
90 config.fb_location = CAMERA_FB_IN_PSRAM;
91 config.jpeg_quality = 10;
92 config.fb_count = 1;
93
94 // camera init
95 esp_err_t err = esp_camera_init(&config);
96 if (err != ESP_OK) {
97     if(err==ESP_ERR_NOT_SUPPORTED) {
98         config.pixel_format = PIXFORMAT_RGB565;
99         esp_err_t err = esp_camera_init(&config);
100         if (err != ESP_OK) {
101             Serial.printf("Camera init failed with error 0x%x", err);
102             return;
103         }
104     }
105 }
106
107 sensor_t * s = esp_camera_sensor_get();
108 // drop down frame size for higher initial frame rate
109 uint16_t pid = s->id.PID;
110 if(pid == OV2640_PID) {
111     s->set_hmirror(s, 1);
112     s->set_vflip(s, 1);
113 }
114 else if(pid == OV3660_PID) {
115     s->set_hmirror(s, 1);
116     s->set_vflip(s, 0);
117 }
118 else if(pid == GC2145_PID) {
119     s->set_hmirror(s, 0);
120     delay(500);
121     s->set_vflip(s, 0);
122 }
123 else if(pid == GC0308_PID) {
124     s->set_hmirror(s, 0);
125     delay(500);
126     s->set_vflip(s, 0);
127 }
128 else {
129     s->set_hmirror(s, 1);
130     s->set_vflip(s, 0);
```

```

131 }
132 s->set_brightness(s, 1); // Slightly increase brightness
133 s->set_saturation(s, 0); // Reduce saturation
134 s->set_ae_level(s, -3); // Set exposure compensation level
135 }

```

Add procedure files and API interface files related to ESP32-S3 camera.

```

1 #include "esp_camera.h"
2 #include <WiFi.h>
...
9 #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
...
22 #include "camera_pins.h"

```

Enter the name and password of the router

```

27 const char *ssid = "*****"; //input your wifi name
28 const char *password = "*****"; //input your wifi passwords

```

Configure parameters including interface pins of the camera. Note: It is generally not recommended to change them.

```

67 void camera_init() {
68     config.ledc_channel = LEDC_CHANNEL_0;
69     config.ledc_timer = LEDC_TIMER_0;
70     config.pin_d0 = Y2_GPIO_NUM;
71     config.pin_d1 = Y3_GPIO_NUM;
72     config.pin_d2 = Y4_GPIO_NUM;
73     config.pin_d3 = Y5_GPIO_NUM;
74     config.pin_d4 = Y6_GPIO_NUM;
75     config.pin_d5 = Y7_GPIO_NUM;
76     config.pin_d6 = Y8_GPIO_NUM;
77     config.pin_d7 = Y9_GPIO_NUM;
78     config.pin_xclk = XCLK_GPIO_NUM;
79     config.pin_pclk = PCLK_GPIO_NUM;
80     config.pin_vsync = VSYNC_GPIO_NUM;
81     config.pin_href = HREF_GPIO_NUM;
82     config.pin_sccb_sda = SIOD_GPIO_NUM;
83     config.pin_sccb_scl = SIOC_GPIO_NUM;
84     config.pin_pwdn = PWDN_GPIO_NUM;
85     config.pin_reset = RESET_GPIO_NUM;
86     config.xclk_freq_hz = 10000000;
87     config.frame_size = FRAMESIZE_QVGA;
88     config.pixel_format = PIXFORMAT_JPEG; // for streaming
89     config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
90     config.fb_location = CAMERA_FB_IN_PSRAM;
91     config.jpeg_quality = 10;
92     config.fb_count = 1;
93 }

```

```
94 // camera init
95 esp_err_t err = esp_camera_init(&config);
96 if (err != ESP_OK) {
97     if(err==ESP_ERR_NOT_SUPPORTED) {
98         config.pixel_format = PIXFORMAT_RGB565;
99         esp_err_t err = esp_camera_init(&config);
100         if (err != ESP_OK) {
101             Serial.printf("Camera init failed with error 0x%x", err);
102             return;
103         }
104     }
105 }
106
107 sensor_t * s = esp_camera_sensor_get();
108 // drop down frame size for higher initial frame rate
109 uint16_t pid = s->id.PID;
110 if(pid == OV2640_PID) {
111     s->set_hmirror(s, 1);
112     s->set_vflip(s, 1);
113 }
114 else if(pid == OV3660_PID) {
115     s->set_hmirror(s, 1);
116     s->set_vflip(s, 0);
117 }
118 else if(pid == GC2145_PID) {
119     s->set_hmirror(s, 0);
120     delay(500);
121     s->set_vflip(s, 0);
122 }
123 else if(pid == GC0308_PID) {
124     s->set_hmirror(s, 0);
125     delay(500);
126     s->set_vflip(s, 0);
127 }
128 else{
129     s->set_hmirror(s, 1);
130     s->set_vflip(s, 0);
131 }
132 s->set_brightness(s, 1); // Slightly increase brightness
133 s->set_saturation(s, 0); // Reduce saturation
134 s->set_ae_level(s, -3); // Set exposure compensation level
135 }
```

ESP32-S3 connects to the router and prints a successful connection prompt. If it has not been successfully connected, press the reset key on the ESP32-S3 WROOM.

```

41  WiFi.begin(ssid, password);
42  WiFi.setSleep(false);
43
44  while (WiFi.status() != WL_CONNECTED) {
45      delay(500);
46      Serial.print(".");
47  }
48  while (WiFi.STA.hasIP() != true) {
49      delay(500);
50      Serial.print(".");
51  }
52  Serial.println("");
53  Serial.println("WiFi connected");

```

Open the video streams server function of the camera and print its IP address via serial port.

```

55  startCameraServer();
56
57  Serial.print("Camera Ready! Use 'http://");
58  Serial.print(WiFi.localIP());
59  Serial.println("' to connect");

```

Configure the display image information of the camera.

The `set_vflip()` function sets whether the image is flipped 180°, with 0 for no flip and 1 for flip 180°.

The `set_brightness()` function sets the brightness of the image, with values ranging from -2 to 2.

The `set_saturation()` function sets the color saturation of the image, with values ranging from -2 to 2.

```

36  sensor_t *s = esp_camera_sensor_get();
37  s->set_vflip(s, 1);           //flip it back
38  s->set_brightness(s, 1);      //up the blightness just a bit
39  s->set_saturation(s, -1);     //lower the saturation

```

Modify the resolution and sharpness of the images captured by the camera. The sharpness ranges from 10 to 63, and the smaller the number, the sharper the picture. The larger the number, the blurrier the picture. Please refer to the table below.

```

26  config.frame_size = FRAME_SIZE_VGA;
27  config.jpeg_quality = 10;

```

Reference

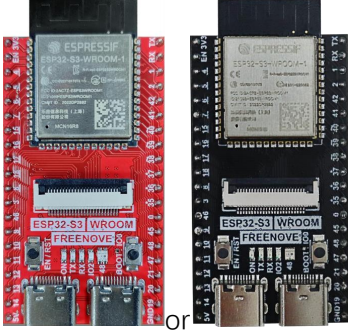
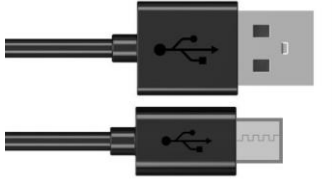
Image resolution	Sharpness	Image resolution	Sharpness
FRAME_SIZE_96x96	96x96	FRAME_SIZE_HVGA	480x320
FRAME_SIZE_QQVGA	160x120	FRAME_SIZE_VGA	640x480
FRAME_SIZE_QCIF	176x144	FRAME_SIZE_SVGA	800x600
FRAME_SIZE_HQVGA	240x176	FRAME_SIZE_XGA	1024x768
FRAME_SIZE_240x240	240x240	FRAME_SIZE_HD	1280x720
FRAME_SIZE_QVGA	320x240	FRAME_SIZE_SXGA	1280x1024
FRAME_SIZE_CIF	400x296	FRAME_SIZE_UXGA	1600x1200

We recommend that the resolution not exceed VGA(640x480).

Project 7.2 Video Web Server

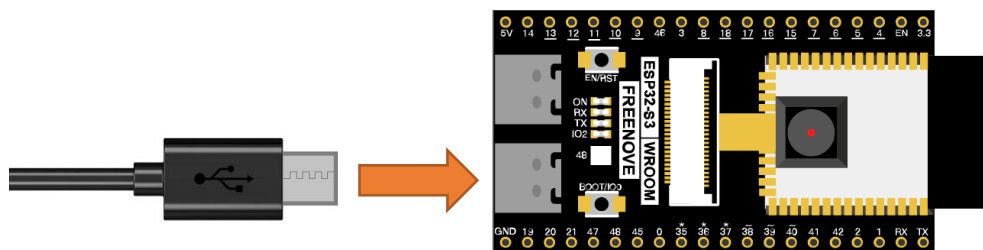
Connect to ESP32-S3 using USB and view its IP address through a serial monitor. Access IP addresses through web pages to obtain real-time video data.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 	SDcard x1 
---	--	--

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

Sketch_07.2_As_VideoWebServer

```

File Edit Sketch Tools Help
ESP32S3 Dev Module
Sketch_32.2_As_VideoWebServer.ino app_httpd.cpp camera_pins.h
1  /*****
2  Filename   : Video Web Server
3  Description : The camera images captured by the ESP32S3 are displayed on the web page.
4  Author    : www.freenove.com
5  Modification: 2022/11/01
6  *****/
7  #include "esp_camera.h"
8  #include <WiFi.h>
9
10 // Select camera model
11 #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
12
13 #include "camera_pins.h"
14
15 const char* ssid   = "*****"; //input your wifi name
16 const char* password = "*****"; //input your wifi passwords
17
18 void startCameraServer();
19
20 void setup() {
21   Serial.begin(115200);
22   Serial.setDebugOutput(true);
23   Serial.println();
24
25   camera_config_t config;
26   config.ledc_channel3 = LEDC_CHANNEL_0;

```

Before running the program, please modify your router's name and password in the box shown in the illustration above to make sure that your Sketch can compile and work successfully.

Compile and upload codes to ESP32-S3, open the serial monitor and set the baud rate to 115200, and the serial monitor will print out a network link address.

```

SD_MMC Card Type: SDSC
SD_MMC Card Size: 961MB
Total space: 958MB
Used space: 15MB
Removing Dir: /video
rmdir failed
Creating Dir: /video
Dir created

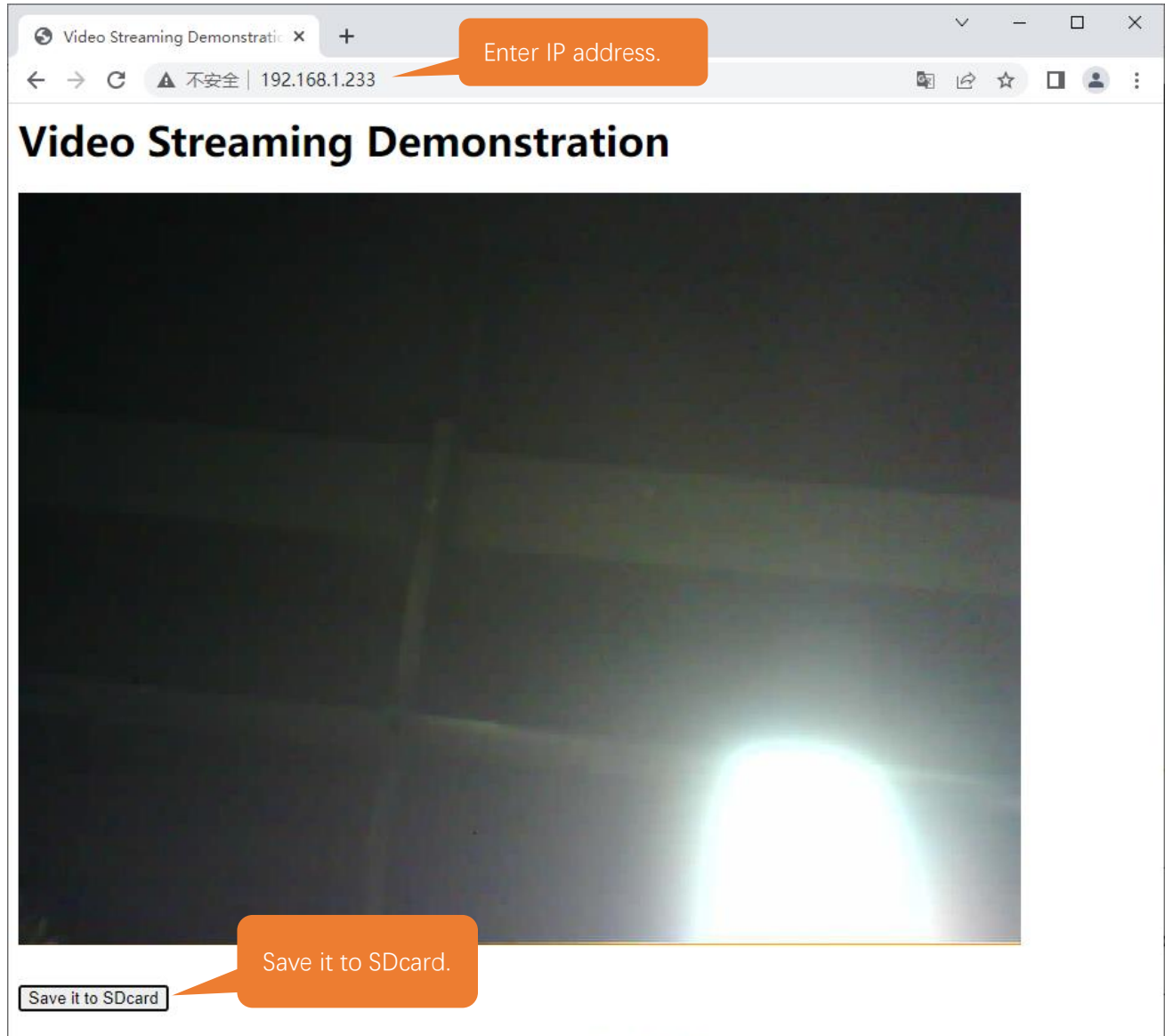
WiFi connected
[ 1543] [I] [app_httpd.cpp:305] startCameraServer(): [] Starting web server on port: '80'
[ 1545] [I] [app_httpd.cpp:315] startCameraServer(): [] Starting stream server on port: '81'
Camera Ready! Use 'http://192.168.1.233' to connect

```

If your ESP32-S3 has been in the process of connecting to router, but the information above has not been printed out, please re-check whether the router name and password have been entered correctly and press the reset key on ESP32-S3 WROOM to wait for a successful connection prompt.

Open a web browser, enter the IP address printed by the serial monitor in the address bar, and access it. Taking the Google browser as an example, here's what the browser prints out after successful access to ESP32-S3's IP.

The effect is shown in the image below.



The following is the main program code. You need include other code files in the same folder when write your own code.

```
1  #include "esp_camera.h"
2  #include <WiFi.h>
3  #include "sd_read_write.h"
4
5  // Select camera model
6  #define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
7
```

```
8  #include "camera_pins.h"
9
10 const char* ssid      = "*****";  //input your wifi name
11 const char* password = "*****";  //input your wifi passwords
12 camera_config_t config;
13
14 void cameraInit(void);
15 void startCameraServer();
16
17 void setup() {
18     Serial.begin(115200);
19     Serial.setDebugOutput(true);
20     Serial.println();
21
22     cameraInit();
23     sdmmcInit();
24     removeDir(SD_MMC, "/video");
25     createDir(SD_MMC, "/video");
26
27     WiFi.begin(ssid, password);
28
29     while (WiFi.status() != WL_CONNECTED) {
30         delay(500);
31         Serial.print(".");
32     }
33     Serial.println("");
34     Serial.println("WiFi connected");
35
36     startCameraServer();
37
38     Serial.print("Camera Ready! Use 'http://");
39     Serial.print(WiFi.localIP());
40     Serial.println("' to connect");
41 }
42
43 void loop() {
44     // put your main code here, to run repeatedly:
45     delay(10000);
46 }
47
48 void cameraInit(void) {
49     config.ledc_channel = LEDC_CHANNEL_0;
50     config.ledc_timer = LEDC_TIMER_0;
51     config.pin_d0 = Y2_GPIO_NUM;
```

```
52 config.pin_d1 = Y3_GPIO_NUM;
53 config.pin_d2 = Y4_GPIO_NUM;
54 config.pin_d3 = Y5_GPIO_NUM;
55 config.pin_d4 = Y6_GPIO_NUM;
56 config.pin_d5 = Y7_GPIO_NUM;
57 config.pin_d6 = Y8_GPIO_NUM;
58 config.pin_d7 = Y9_GPIO_NUM;
59 config.pin_xclk = XCLK_GPIO_NUM;
60 config.pin_pclk = PCLK_GPIO_NUM;
61 config.pin_vsync = VSYNC_GPIO_NUM;
62 config.pin_href = HREF_GPIO_NUM;
63 config.pin_sccb_sda = SIOD_GPIO_NUM;
64 config.pin_sccb_scl = SIOC_GPIO_NUM;
65 config.pin_pwdn = PWDN_GPIO_NUM;
66 config.pin_reset = RESET_GPIO_NUM;
67 config.xclk_freq_hz = 10000000;
68 config.frame_size = FRAMESIZE_QVGA;
69 config.pixel_format = PIXFORMAT_JPEG; // for streaming
70 config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
71 config.fb_location = CAMERA_FB_IN_PSRAM;
72 config.jpeg_quality = 10;
73 config.fb_count = 1;
74
75 // camera init
76 esp_err_t err = esp_camera_init(&config);
77 if (err != ESP_OK) {
78     if(err==ESP_ERR_NOT_SUPPORTED) {
79         config.pixel_format = PIXFORMAT_RGB565;
80         esp_err_t err = esp_camera_init(&config);
81         if (err != ESP_OK) {
82             Serial.printf("Camera init failed with error 0x%x", err);
83             return;
84         }
85     }
86 }
87
88 sensor_t * s = esp_camera_sensor_get();
89 // drop down frame size for higher initial frame rate
90 uint16_t pid = s->id.PID;
91 if(pid == OV2640_PID) {
92     s->set_hmirror(s, 1);
93     s->set_vflip(s, 1);
94 }
95 else if(pid == OV3660_PID) {
```

```
96     s->set_hmirror(s, 1);
97     s->set_vflip(s, 0);
98 }
99 else if(pid == GC2145_PID){
100     s->set_hmirror(s, 0);
101     delay(500);
102     s->set_vflip(s, 0);
103 }
104 else if(pid == GC0308_PID){
105     s->set_hmirror(s, 0);
106     delay(500);
107     s->set_vflip(s, 0);
108 }
109 else{
110     s->set_hmirror(s, 1);
111     s->set_vflip(s, 0);
112 }
113 s->set_brightness(s, 1); // Slightly increase brightness
114 s->set_saturation(s, 0); // Reduce saturation
115 s->set_ae_level(s, -3); // Set exposure compensation level
116 }
```

Configure parameters including interface pins of the camera. Note: It is generally not recommended to change them.

```
49 config.ledc_channel = LEDC_CHANNEL_0;
50 config.ledc_timer = LEDC_TIMER_0;
51 config.pin_d0 = Y2_GPIO_NUM;
52 config.pin_d1 = Y3_GPIO_NUM;
53 config.pin_d2 = Y4_GPIO_NUM;
54 config.pin_d3 = Y5_GPIO_NUM;
55 config.pin_d4 = Y6_GPIO_NUM;
56 config.pin_d5 = Y7_GPIO_NUM;
57 config.pin_d6 = Y8_GPIO_NUM;
58 config.pin_d7 = Y9_GPIO_NUM;
59 config.pin_xclk = XCLK_GPIO_NUM;
60 config.pin_pclk = PCLK_GPIO_NUM;
61 config.pin_vsync = VSYNC_GPIO_NUM;
62 config.pin_href = HREF_GPIO_NUM;
63 config.pin_sccb_sda = SIOD_GPIO_NUM;
64 config.pin_sccb_scl = SIOC_GPIO_NUM;
65 config.pin_pwdn = PWDN_GPIO_NUM;
66 config.pin_reset = RESET_GPIO_NUM;
67 config.xclk_freq_hz = 20000000;
68 config.frame_size = FRAMESIZE_UXGA;
```

```

69  config.pixel_format = PIXFORMAT_JPEG; // for streaming
70  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
71  config.fb_location = CAMERA_FB_IN_PSRAM;
72  config.jpeg_quality = 12;
73  config.fb_count = 1;

```

The camera's picture can be automatically adjusted according to the model of the camera driver. Here, you can also configure the picture to be upside-down or left-right reversed.

```

88  sensor_t * s = esp_camera_sensor_get();
89  // drop down frame size for higher initial frame rate
90  uint16_t pid = s->id.PID;
91  if(pid == OV2640_PID) {
92      s->set_hmirror(s, 1);
93      s->set_vflip(s, 1);
94  }
95  else if(pid == OV3660_PID) {
96      s->set_hmirror(s, 1);
97      s->set_vflip(s, 0);
98  }
99  else if(pid == GC2145_PID) {
100     s->set_hmirror(s, 0);
101     delay(500);
102     s->set_vflip(s, 0);
103 }
104 else if(pid == GC0308_PID) {
105     s->set_hmirror(s, 0);
106     delay(500);
107     s->set_vflip(s, 0);
108 }
109 else {
110     s->set_hmirror(s, 1);
111     s->set_vflip(s, 0);
112 }
113 s->set_brightness(s, 1); // Slightly increase brightness
114 s->set_saturation(s, 0); // Reduce saturation
115 s->set_ae_level(s, -3); // Set exposure compensation level

```

ESP32-S3 connects to the router and prints a successful connection prompt. If it has not been successfully connected, press the reset key on the ESP32-S3 WROOM.

```

27  WiFi.begin(ssid, password);
28
29  while (WiFi.status() != WL_CONNECTED) {
30      delay(500);
31      Serial.print(".");
32  }

```

Open the video streams server function of the camera and print its IP address via serial port.

```

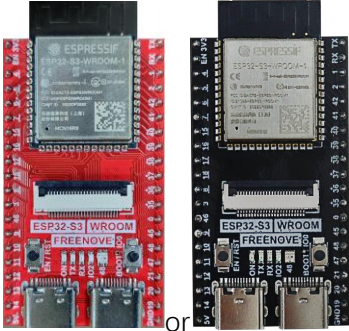
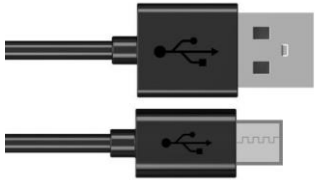
36 startCameraServer();
37
38 Serial.print("Camera Ready! Use 'http://");
39 Serial.print(WiFi.localIP());
40 Serial.println("' to connect");

```

Project 7.3 Camera and SDcard

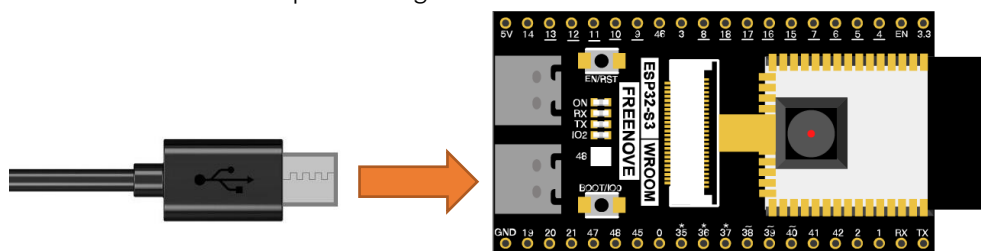
In this chapter, we continue to use the camera and SD card. We will use the onboard button as the shutter. When the button is pressed, the ESP32-S3 takes a photo and stores the photo in the SD folder.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 	SDcard x1 
--	---	---

Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

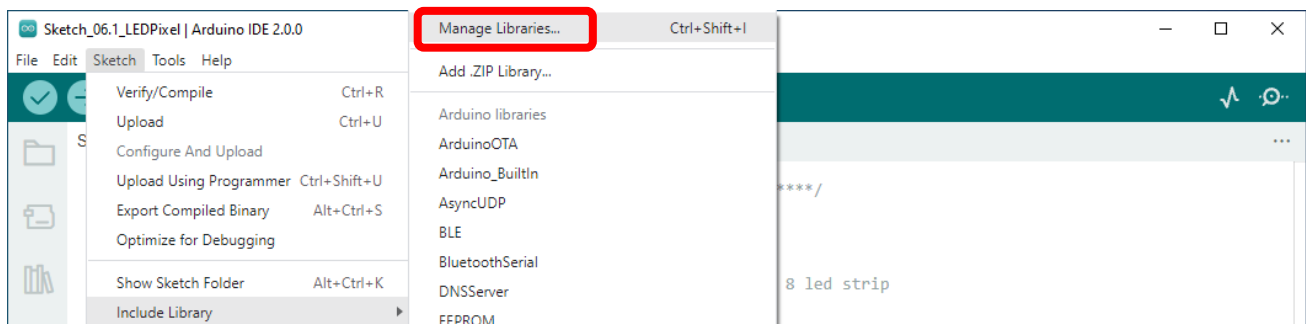
This code uses a library named "Freenove_WS2812_Lib_for_ESP32", if you have not installed it, please do so first.

Library is an important feature of the open source world, and we know that Arduino is an open source platform that everyone can contribute to. Libraries are generally licensed under the LGPL, which means you can use them for free to apply to your creations.

How to install the library

There are two ways to add libraries.

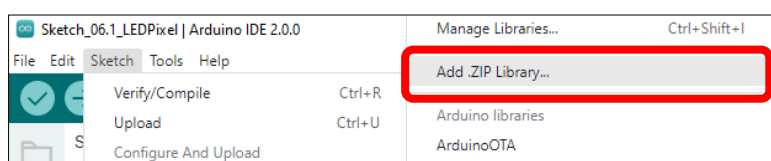
The first way, open the Arduino IDE, click Sketch → Include Library → Manager Libraries.



In the pop-up window, Library Manager, search for the name of the Library, "Freenove WS2812 Lib for ESP32". Then click Install.



The second way, open Arduino IDE, click Sketch→Include Library→Add .ZIP Library, In the pop-up window, find the file named ".Libraries/Freenove_WS2812_Lib_for_ESP32.Zip" which locates in this directory, and click OPEN.



Sketch_07.3_Camera_SDCard

```

File Edit Sketch Tools Help
ESP32S3 Dev Module

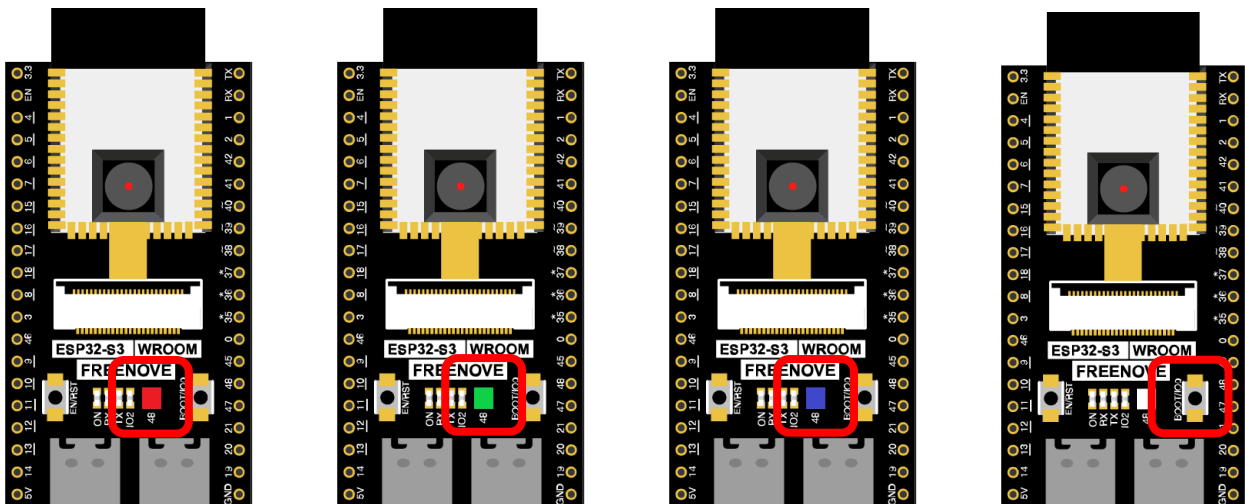
Sketch_32.3_Camera_SDCard.ino camera_pins.h sd_read_write.cpp sd_read_write.h ws2812.cpp ws2812.h

1 /*****
2  Filename   : Camera and SDCard
3  Description : Use the onboard buttons to take photo and save them to an SD card.
4  Author    : www.freenove.com
5  Modification: 2022/11/02
6  *****/
7 #include "esp_camera.h"
8 #define CAMERA_MODEL_ESP32S3_EYE
9 #include "camera_pins.h"
10 #include "ws2812.h"
11 #include "sd_read_write.h"
12
13 #define BUTTON_PIN 0
14
15 void setup() {
16     Serial.begin(115200);
17     Serial.setDebugOutput(false);
18     Serial.println();
19     pinMode(BUTTON_PIN, INPUT_PULLUP);
20     ws2812Init();
21     sdmmcInit();
22     //removeDir(SD_MMC, "/camera");
23     createDir(SD_MMC, "/camera");
24     listDir(SD_MMC, "/camera", 0);

```

Compile and upload the code to the ESP32-S3.

If your camera is not installed properly, causing the camera to fail to initialize, or you have not inserted the SD card into the ESP32-S3 in advance, the on-board colored lights will turn on red as a reminder. If all is well, the onboard colored light will light up green. When the onboard BOOT button is pressed, the ESP32-S3 will capture the current camera image and save it in the "Camera" folder of the SD card. At the same time, the onboard LED lights up blue, and returns to green after taking a photo.



As shown in the image below, after uploading the code to the ESP32-S3, the ESP32-S3 will automatically create a folder named "camera" in the SD card. Every time the BOOT button is pressed, the on-board colored light turns on blue, and ESP32-S3 collects a photo information and stores it in the "camera" folder. Press the button once to take a photo.

When we press the RST button to reset the ESP32-S3, we can see that there are some photo files in the SD card folder. These photos you can read directly through the card reader.

```
Output Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3')
New Line 115200 baud

SD_MMC Card Type: SDSC
SD_MMC Card Size: 961MB
Total space: 958MB
Used space: 14MB
Creating Dir: /camera
Dir created
Listing directory: /camera
Camera configuration complete!
Saved file to path: /camera/0.jpg
ESP-ROM: esp32s3-20210327
Build: Mar 27 2021
rst: 0x1 (POWERON), boot: 0x8 (SPI_FAST_FLASH_BOOT)
SPIWP: 0xee
mode: DIO, clock div: 1
load: 0x3fce3808, len: 0x43c
load: 0x403c9700, len: 0xbec
load: 0x403cc700, len: 0x2a3c
entry 0x403c98d8

SD_MMC Card Type: SDSC
SD_MMC Card Size: 961MB
Total space: 958MB
Used space: 14MB
Creating Dir: /camera
Dir created
Listing directory: /camera
FILE: 0.jpg SIZE: 25390
Camera configuration complete!

Ln 24. Col 31 UTF-8 ESP32S3 Dev Module on COM3
```

The following is the main program code. You need include other code files in the same folder when write your own code.

[illegible]

```
43     }
44     else {
45         uint8_t *jpg_buf = NULL;
46         size_t jpg_len = 0;
47
48         if (frame2jpg(fb, 80, &jpg_buf, &jpg_len)) {
49             writejpg(SD_MMC, path.c_str(), jpg_buf, jpg_len);
50             free(jpg_buf);
51         } else {
52             Serial.println("JPEG compression failed!");
53         }
54     }
55 }
56 esp_camera_fb_return(fb);
57 }
58 else {
59     Serial.println("Camera capture failed.");
60 }
61 ws2812SetColor(2);
62 }
63 }
64 }
65
66 int cameraSetup(void) {
67     camera_config_t config;
68     config.ledc_channel = LEDC_CHANNEL_0;
69     config.ledc_timer = LEDC_TIMER_0;
70     config.pin_d0 = Y2_GPIO_NUM;
71     config.pin_d1 = Y3_GPIO_NUM;
72     config.pin_d2 = Y4_GPIO_NUM;
73     config.pin_d3 = Y5_GPIO_NUM;
74     config.pin_d4 = Y6_GPIO_NUM;
75     config.pin_d5 = Y7_GPIO_NUM;
76     config.pin_d6 = Y8_GPIO_NUM;
77     config.pin_d7 = Y9_GPIO_NUM;
78     config.pin_xclk = XCLK_GPIO_NUM;
79     config.pin_pclk = PCLK_GPIO_NUM;
80     config.pin_vsync = VSYNC_GPIO_NUM;
81     config.pin_href = HREF_GPIO_NUM;
82     config.pin_sccb_sda = SIOD_GPIO_NUM;
83     config.pin_sccb_scl = SIOC_GPIO_NUM;
84     config.pin_pwdn = PWDN_GPIO_NUM;
85     config.pin_reset = RESET_GPIO_NUM;
86     config.xclk_freq_hz = 10000000;
```

```
87 config.frame_size = FRAMESIZE_QVGA;
88 config.pixel_format = PIXFORMAT_JPEG; // for streaming
89 config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
90 config.fb_location = CAMERA_FB_IN_PSRAM;
91 config.jpeg_quality = 10;
92 config.fb_count = 1;
93
94 // camera init
95 esp_err_t err = esp_camera_init(&config);
96 if (err != ESP_OK) {
97     if(err==ESP_ERR_NOT_SUPPORTED) {
98         config.pixel_format = PIXFORMAT_RGB565;
99         esp_err_t err = esp_camera_init(&config);
100         if (err != ESP_OK) {
101             Serial.printf("Camera init failed with error 0x%x", err);
102             return 0;
103         }
104     }
105 }
106
107 sensor_t * s = esp_camera_sensor_get();
108 // drop down frame size for higher initial frame rate
109 uint16_t pid = s->id.PID;
110 if(pid == OV2640_PID) {
111     s->set_hmirror(s, 1);
112     s->set_vflip(s, 1);
113 }
114 else if(pid == OV3660_PID) {
115     s->set_hmirror(s, 1);
116     s->set_vflip(s, 0);
117 }
118 else if(pid == GC2145_PID) {
119     s->set_hmirror(s, 0);
120     delay(500);
121     s->set_vflip(s, 0);
122 }
123 else if(pid == GC0308_PID) {
124     s->set_hmirror(s, 0);
125     delay(500);
126     s->set_vflip(s, 0);
127 }
128 else {
129     s->set_hmirror(s, 1);
130     s->set_vflip(s, 0);
```

```

131 }
132 s->set_brightness(s, 1); // Slightly increase brightness
133 s->set_saturation(s, 0); // Reduce saturation
134 s->set_ae_level(s, -3); // Set exposure compensation level
135 return 1;
136 }

```

Configure camera parameters, including camera interface pins and other information. Altering them is generally not recommended. Returns 1 if the camera is initialized successfully, and returns 0 if it fails.

```

66 int cameraSetup(void) {
67     camera_config_t config;
68     config.ledc_channel = LEDC_CHANNEL_0;
69     config.ledc_timer = LEDC_TIMER_0;
70     config.pin_d0 = Y2_GPIO_NUM;
71     config.pin_d1 = Y3_GPIO_NUM;
72     config.pin_d2 = Y4_GPIO_NUM;
73     config.pin_d3 = Y5_GPIO_NUM;
74     config.pin_d4 = Y6_GPIO_NUM;
75     config.pin_d5 = Y7_GPIO_NUM;
76     config.pin_d6 = Y8_GPIO_NUM;
77     config.pin_d7 = Y9_GPIO_NUM;
78     config.pin_xclk = XCLK_GPIO_NUM;
79     config.pin_pclk = PCLK_GPIO_NUM;
80     config.pin_vsync = VSYNC_GPIO_NUM;
81     config.pin_href = HREF_GPIO_NUM;
82     config.pin_sccb_sda = SIOD_GPIO_NUM;
83     config.pin_sccb_scl = SIOC_GPIO_NUM;
84     config.pin_pwn = PWDN_GPIO_NUM;
85     config.pin_reset = RESET_GPIO_NUM;
86     config.xclk_freq_hz = 10000000;
87     config.frame_size = FRAMESIZE_QVGA;
88     config.pixel_format = PIXFORMAT_JPEG; // for streaming
89     config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
90     config.fb_location = CAMERA_FB_IN_PSRAM;
91     config.jpeg_quality = 10;
92     config.fb_count = 1;
93
94     // camera init
95     esp_err_t err = esp_camera_init(&config);
96     if (err != ESP_OK) {
97         if(err==ESP_ERR_NOT_SUPPORTED) {
98             config.pixel_format = PIXFORMAT_RGB565;
99             esp_err_t err = esp_camera_init(&config);
100             if (err != ESP_OK) {
101                 Serial.printf("Camera init failed with error 0x%x", err);

```

```

102         return 0;
103     }
104 }
105 }
106
107 sensor_t * s = esp_camera_sensor_get();
108 // drop down frame size for higher initial frame rate
109 uint16_t pid = s->id.PID;
110 if(pid == OV2640_PID) {
111     s->set_hmirror(s, 1);
112     s->set_vflip(s, 1);
113 }
114 else if(pid == OV3660_PID) {
115     s->set_hmirror(s, 1);
116     s->set_vflip(s, 0);
117 }
118 else if(pid == GC2145_PID) {
119     s->set_hmirror(s, 0);
120     delay(500);
121     s->set_vflip(s, 0);
122 }
123 else if(pid == GC0308_PID) {
124     s->set_hmirror(s, 0);
125     delay(500);
126     s->set_vflip(s, 0);
127 }
128 else {
129     s->set_hmirror(s, 1);
130     s->set_vflip(s, 0);
131 }
132 s->set_brightness(s, 1); // Slightly increase brightness
133 s->set_saturation(s, 0); // Reduce saturation
134 s->set_ae_level(s, -3); // Set exposure compensation level
135 return 1;
136 }

```

Initialize the serial port, buttons, lights and SD card.

```

10 Serial.begin(115200);
11 Serial.setDebugOutput(false);
12 Serial.println();
13 pinMode(BUTTON_PIN, INPUT_PULLUP);
14 ws2812Init();
15 sdmmcInit();

```

Call `ws2812SetColor()` to set the color of the LED. When the parameter is 0, the LED is turned off, when the parameter is 1, the red light is displayed, when the parameter is 2, the green light is displayed, and when the parameter is 3, the blue light is displayed.

```
20 ws2812SetColor(2);
```

First, obtain the camera data. Then, read the file number from the camera folder in the SD card. Based on this, create a new file. If the camera data is not in JPEG format, convert it to JPEG format first, then write the camera data into it. If the camera data cannot be obtained, directly print the prompt message.

```
34 camera_fb_t * fb = esp_camera_fb_get();
35 if (fb != NULL) {
36     int photo_index = readFileNum(SD_MMC, "/camera");
37     if(photo_index != -1) {
38         String path = "/camera/" + String(photo_index) + ".jpg";
39
40         if (fb->format == PIXFORMAT_JPEG) {
41             writejpg(SD_MMC, path.c_str(), fb->buf, fb->len);
42             Serial.println("Direct JPEG save.");
43         }
44         else {
45             uint8_t *jpg_buf = NULL;
46             size_t jpg_len = 0;
47
48             if (frame2jpg(fb, 80, &jpg_buf, &jpg_len)) {
49                 writejpg(SD_MMC, path.c_str(), jpg_buf, jpg_len);
50                 free(jpg_buf);
51             } else {
52                 Serial.println("JPEG compression failed!");
53             }
54         }
55     }
56     esp_camera_fb_return(fb);
57 }
58 else {
59     Serial.println("Camera capture failed.");
60 }
```

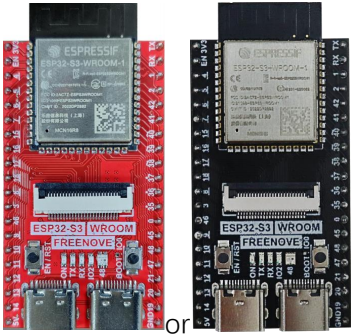
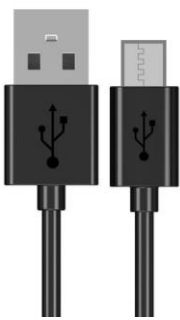
Chapter 8 Camera Tcp Server

In the previous section, we used web page to display the video data captured by ESP32-S3, and in this section, we will use a mobile phone to display it.

Project 8.1 Camera Tcp Server

Connect ESP32-S3 using USB and check its IP address through serial monitor. Use a mobile phone to obtain video and image data.

Component List

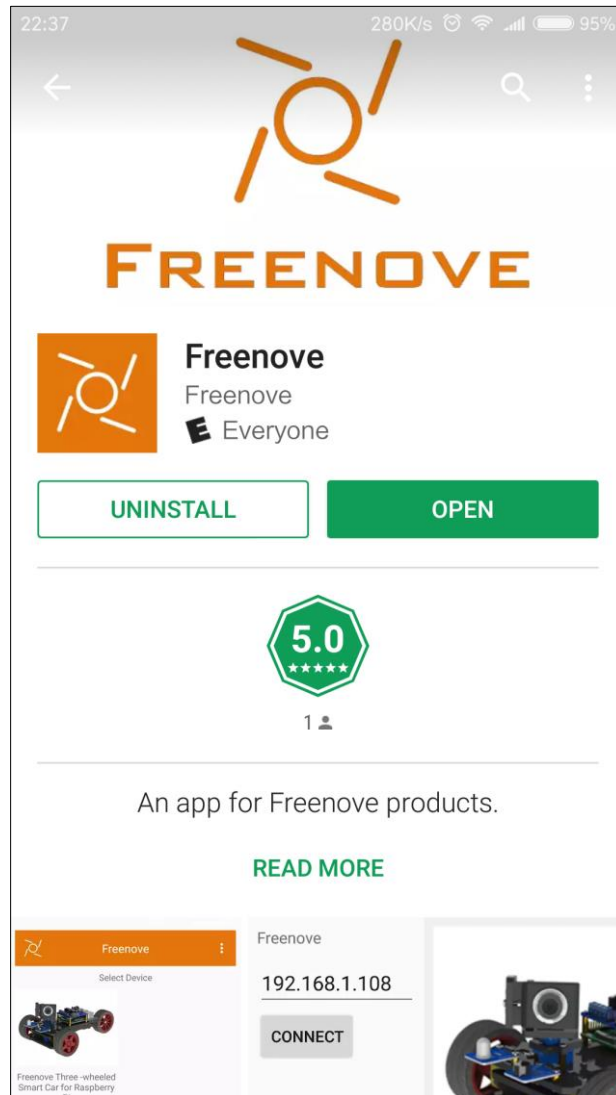
ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
--	--

Install Freenove app

There are three ways to install app, you can choose any one.

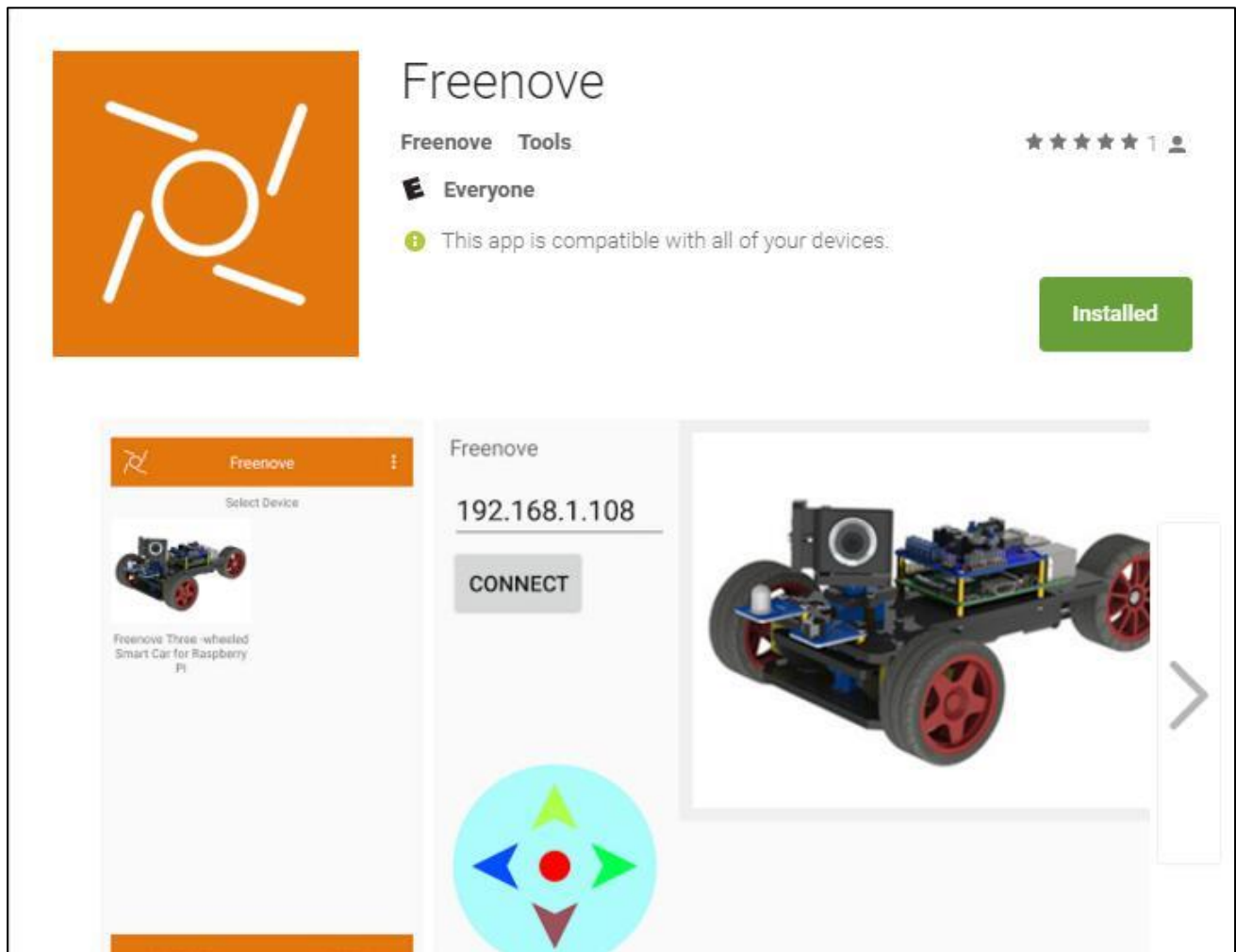
Method 1

Use Google play to search “Freenove”, download and install.



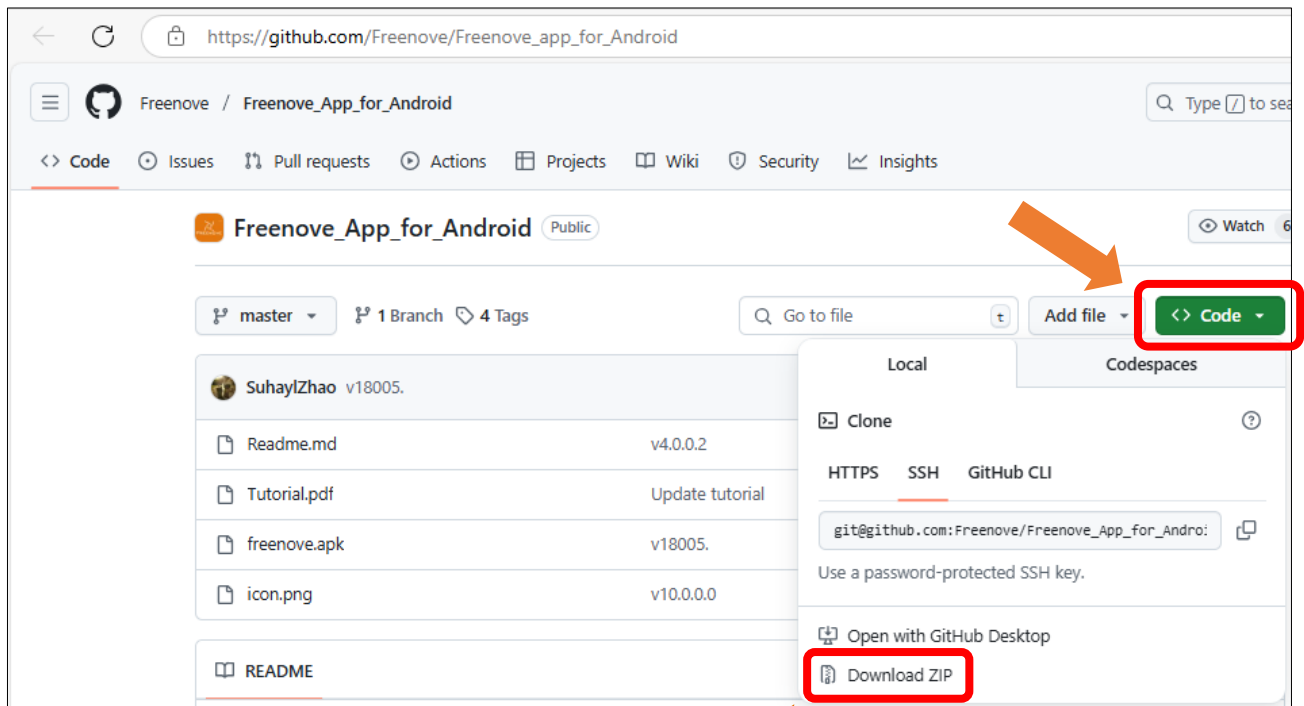
Method 2

Visit <https://play.google.com/store/apps/details?id=com.freenove.suhayl.Freenove>, and click install.



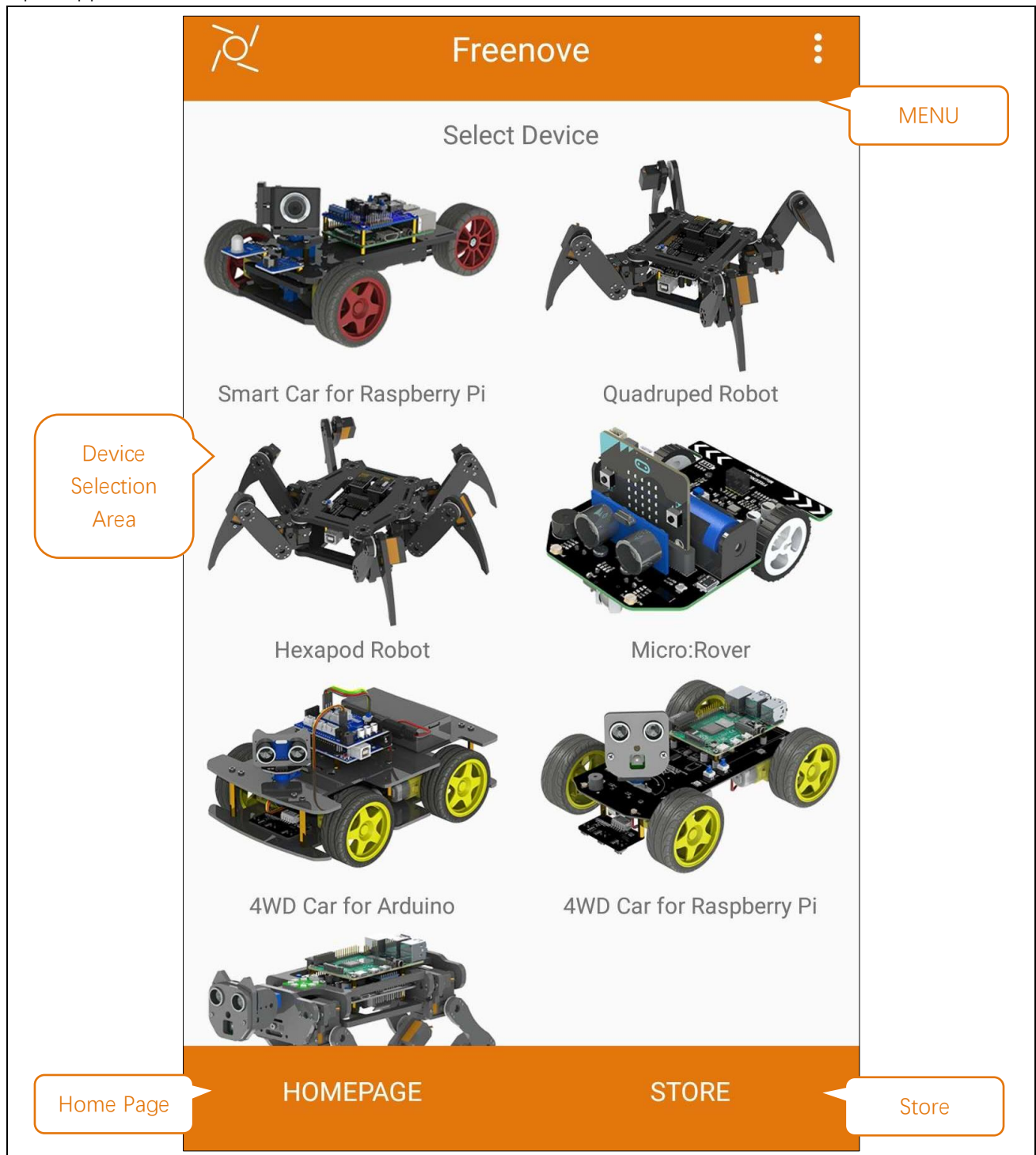
Method 3

Visit https://github.com/Freenove/Freenove_app_for_Android, download the files in this library, and install freenove.apk to your Android phone manually.



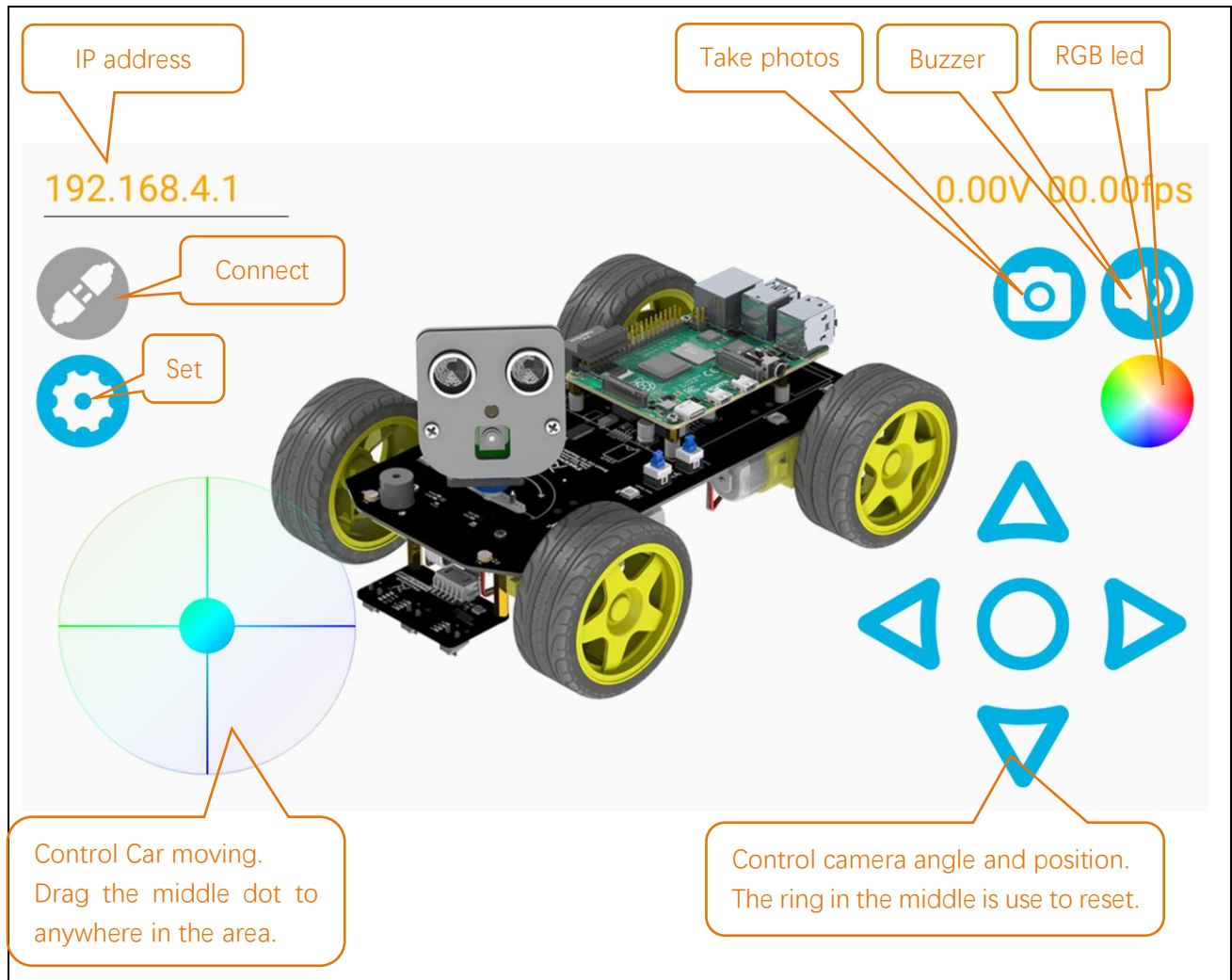
Menu

Open application "Freenove", as shown below:



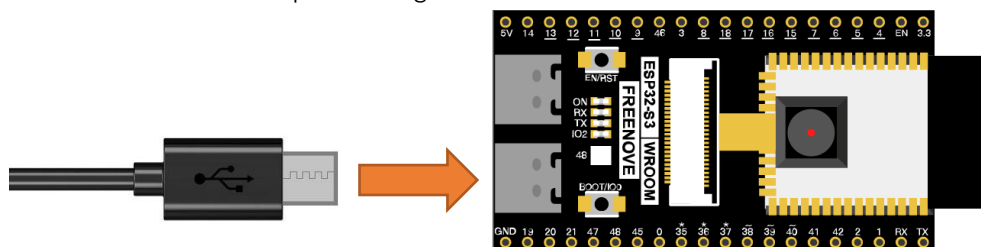
Freenove 4WD Car for Raspberry Pi

In this chapter, we use Freenove 4WD Car for Raspberry Pi, so it is necessary to understand the interface of this mode.



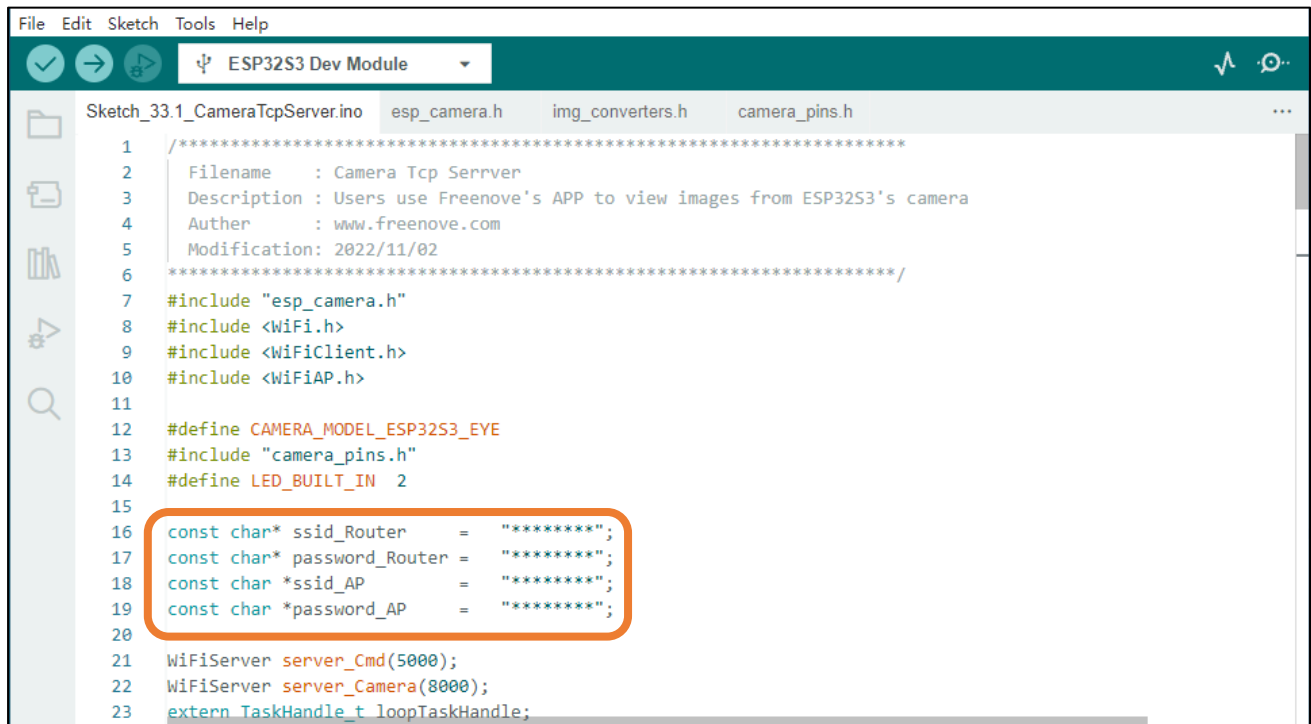
Circuit

Connect Freenove ESP32-S3 to the computer using the USB cable.



Sketch

After making sure the Tools is configured correctly, don't run Sketch. Due to WiFi, we need to modify Sketch a little bit based on physical situation.



```

1  /*****
2  Filename   : Camera Tcp Serrver
3  Description : Users use Freenove's APP to view images from ESP32S3's camera
4  Auther    : www.freenove.com
5  Modification: 2022/11/02
6  *****/
7  #include "esp_camera.h"
8  #include <WiFi.h>
9  #include <WiFiClient.h>
10 #include <WiFiAP.h>
11
12 #define CAMERA_MODEL_ESP32S3_EYE
13 #include "camera_pins.h"
14 #define LED_BUILT_IN 2
15
16 const char* ssid_Router = "*****";
17 const char* password_Router = "*****";
18 const char *ssid_AP = "*****";
19 const char *password_AP = "*****";
20
21 WiFiServer server_Cmd(5000);
22 WiFiServer server_Camera(8000);
23 extern TaskHandle_t loopTaskHandle;
  
```

In the box in the figure above, `ssid_Router` and `password_Router` are the user's Router name and password, which need to be modified according to the actual name and password. `ssid_AP` and `password_AP` are name and password of a AP created by ESP32-S3, and they are freely set by the user. When all settings are correct, compile and upload the code to ESP32-S3, turn on the serial port monitor, and set the baud rate to 115200. The serial monitor will print out two IP addresses.



```

Output  Serial Monitor x
Message (Ctrl + Enter to send message to 'ESP32S3 Dev Module' on 'COM3')
New Line 115200 baud

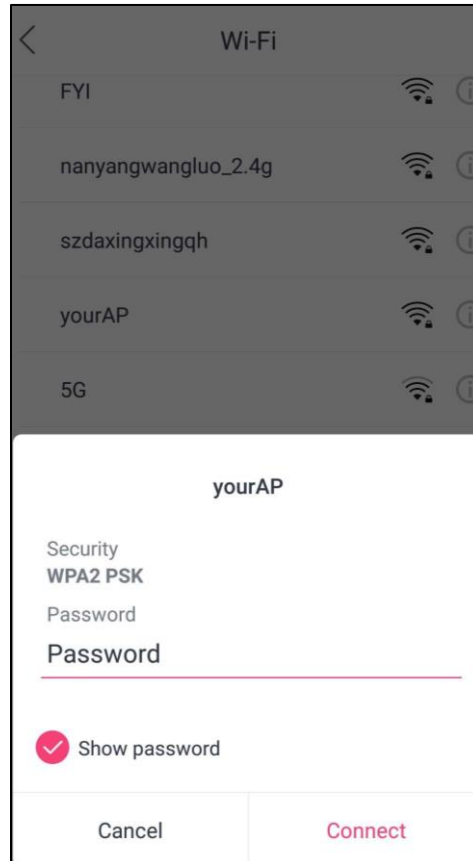
entry 0x403c98d8

Camera configuration complete!
AP IP address: 192.168.4.1
Connecting FYI_2.4G..
WiFi connected
Camera Ready! Use '192.168.4.1 or 192.168.1.233' to connect in Freenove app.
Task Cmd_Server is starting ...
  
```

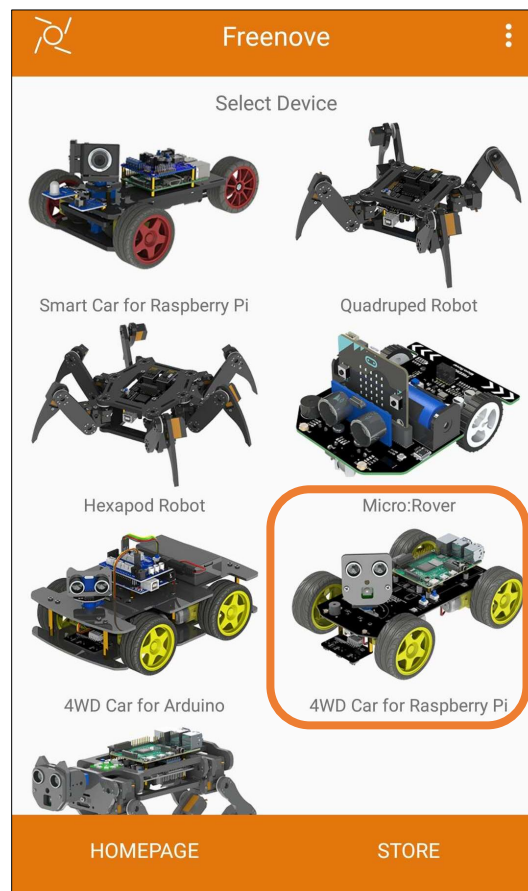
There are two methods for you to check camera data of ESP32-S3 via mobile phone APP.

Method 1:

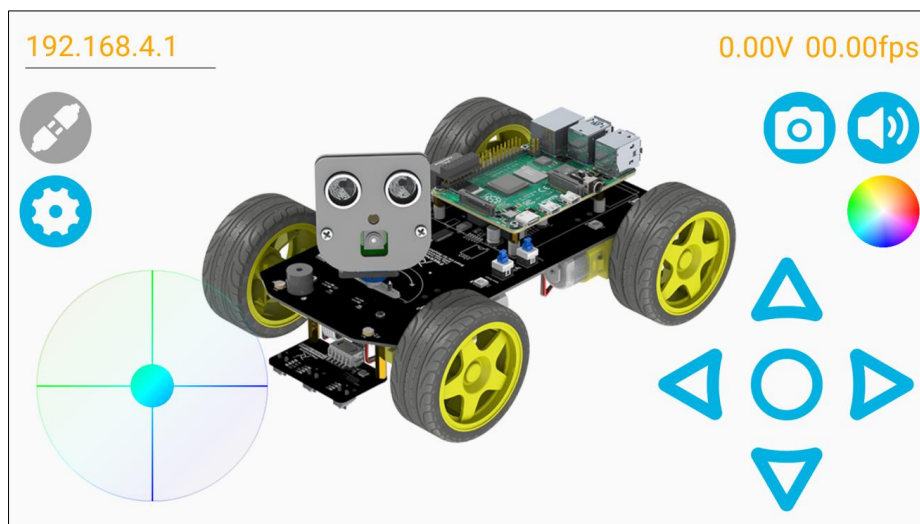
Using your phone's WiFi function, select the WiFi name represented by ssid_AP in Sketch and enter the password "password_AP" to connect.



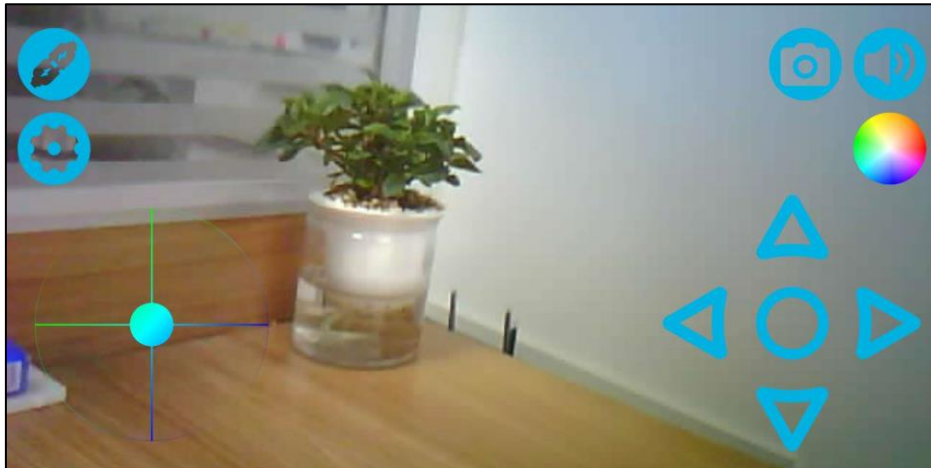
Next, open Freenove app and select 4WD Car for Raspberry Pi mode.



Enter the IP address printed by serial port in the new interface, which generally is "192.168.4.1"



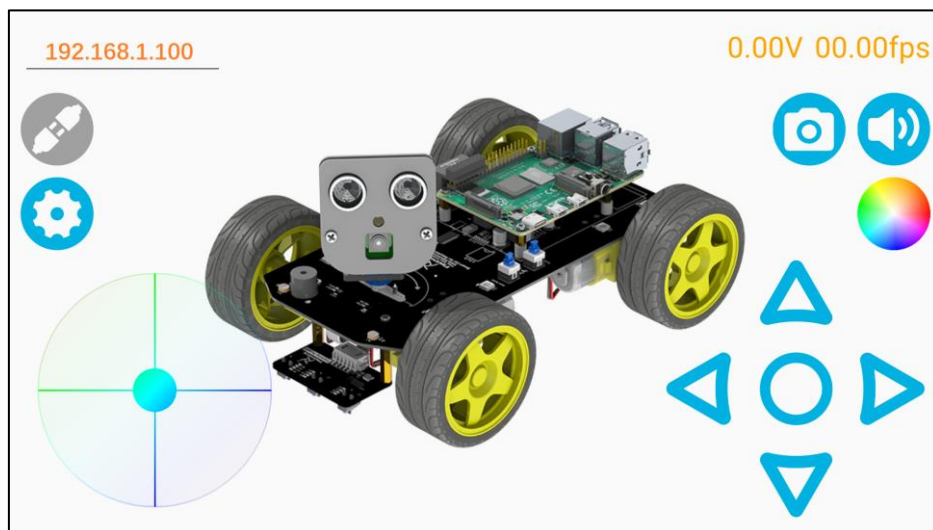
Click "Connect".



Method 2:

Using your phone's WiFi function, select the router named ssid_Router and enter the password "ssid_password" to connect. And then open Freenove app and select 4WD Car for Raspberry Pi mode. The operation is similar to Method 1.

Enter the IP address printed by serial port in the new interface, which generally is not "192.168.4.1" but another one. The IP address in this example is "192.168.1.100". After entering the IP address, click "Connect".



The following is the main program code. You need include other code files in the same folder when write your own code.

Sketch_08.1_Camera_Tcp_Server

```

1  #include "esp_camera.h"
2  #include <WiFi.h>
3  #include <WiFiClient.h>
4  #include <WiFiAP.h>
5
6  #define CAMERA_MODEL_ESP32S3_EYE

```

```

7  #include "camera_pins.h"
8  #define LED_BUILT_IN 2
9
10 const char* ssid_Router    = "*****";
11 const char* password_Router = "*****";
12 const char *ssid_AP        = "*****";
13 const char *password_AP     = "*****";
14
15 WiFiServer server_Cmd(5000);
16 WiFiServer server_Camera(8000);
17 extern TaskHandle_t loopTaskHandle;
18 camera_config_t config;
19 void camera_init();
20
21 void setup() {
22     Serial.begin(115200);
23     Serial.setDebugOutput(false);
24     Serial.println();
25     pinMode(LED_BUILT_IN, OUTPUT);
26     camera_init();
27     WiFi.softAP(ssid_AP, password_AP);
28     IPAddress myIP = WiFi.softAPIP();
29     Serial.print("AP IP address: ");
30     Serial.println(myIP);
31     server_Camera.begin(8000);
32     server_Cmd.begin(5000);
33     //////////////////////////////////////
34     WiFi.begin(ssid_Router, password_Router);
35     Serial.print("Connecting ");
36     Serial.print(ssid_Router);
37     while (WiFi.status() != WL_CONNECTED) {
38         delay(500);
39         Serial.print(".");
40     }
41     while (WiFi.STA.hasIP() != true) {
42         Serial.print(".");
43         delay(500);
44     }
45     Serial.println("");
46     Serial.println("WiFi connected");
47     //////////////////////////////////////
48     Serial.print("Camera Ready! Use '");
49     Serial.print(WiFi.softAPIP());
50     Serial.print("' or ");

```

```

51 Serial.print(WiFi.localIP());
52 Serial.println(" to connect in Freenove app.");
53
54 disableCore0WDT();
55 xTaskCreateUniversal(loopTask_Cmd, "loopTask_Cmd", 8192, NULL, 1, &loopTaskHandle, 0);
//loopTask_Cmd uses core 0.
56 xTaskCreateUniversal(loopTask_Blink, "loopTask_Blink", 8192, NULL, 1, &loopTaskHandle,
0); //loopTask_Blink uses core 0.
57 }
58 //task loop uses core 1.
59 void loop() {
60   WiFiClient client = server_Camera.accept();           // listen for incoming clients
61   if (client) {                                           // if you get a client,
62     Serial.println("Camera Server connected to a client.");
63     String currentLine = "";                               // make a String to hold incoming data from the client
64     while (client.connected()) { // loop while the client's connected
65       camera_fb_t * fb = NULL;
66       size_t jpg_buf_len = 0;
67       uint8_t *jpg_buf = NULL;
68       while (client.connected()) {
69         fb = esp_camera_fb_get();
70         if (!fb) {
71           Serial.println("Camera capture failed");
72         }
73         else{
74           if (fb->format != PIXFORMAT_JPEG) {
75             bool jpeg_converted = frame2jpg(fb, 80, &jpg_buf, &jpg_buf_len);
76             esp_camera_fb_return(fb);
77             fb = NULL;
78             if (!jpeg_converted) {
79               Serial.println("JPEG compression failed");
80               continue;
81             }
82           }
83           else{
84             jpg_buf_len = fb->len;
85             jpg_buf = fb->buf;
86           }
87           uint8_t slen[4];
88           slen[0] = jpg_buf_len >> 0;
89           slen[1] = jpg_buf_len >> 8;
90           slen[2] = jpg_buf_len >> 16;
91           slen[3] = jpg_buf_len >> 24;
92           client.write(slen, 4);

```

```

93         client.write(jpg_buf, jpg_buf_len);
94     }
95     if (fb) {
96         esp_camera_fb_return(fb);
97         fb = NULL;
98         jpg_buf = NULL;
99     }
100     else if (jpg_buf) {
101         free(jpg_buf);
102         jpg_buf = NULL;
103     }
104 }
105 // close the connection:
106 client.stop();
107 Serial.println("Camera Client Disconnected.");
108 }
109 }
110
111 void loopTask_Cmd(void *pvParameters) {
112     Serial.println("Task Cmd_Server is starting ... ");
113     while (1) {
114         WiFiClient client = server_Cmd.accept(); // listen for incoming clients
115         if (client) { // if you get a client,
116             Serial.println("Command Server connected to a client.");
117             String currentLine = ""; // make a String to hold incoming data from the client
118             while (client.connected()) { // loop while the client's connected
119                 if (client.available()) { // if there's bytes to read from the client,
120                     char c = client.read(); // read a byte, then
121                     client.write(c);
122                     Serial.write(c); // print it out the serial monitor
123                     if (c == '\n') { // if the byte is a newline character
124                         currentLine = "";
125                     }
126                     else {
127                         currentLine += c; // add it to the end of the currentLine
128                     }
129                 }
130             }
131             // close the connection:
132             client.stop();
133             Serial.println("Command Client Disconnected.");
134         }
135     }

```

```
136 }
137 void loopTask_Blink(void *pvParameters) {
138     Serial.println("Task Blink is starting ... ");
139     while (1) {
140         digitalWrite(LED_BUILT_IN, !digitalRead(LED_BUILT_IN));
141         delay(1000);
142     }
143 }
144
145 void camera_init() {
146     config.ledc_channel = LEDC_CHANNEL_0;
147     config.ledc_timer = LEDC_TIMER_0;
148     config.pin_d0 = Y2_GPIO_NUM;
149     config.pin_d1 = Y3_GPIO_NUM;
150     config.pin_d2 = Y4_GPIO_NUM;
151     config.pin_d3 = Y5_GPIO_NUM;
152     config.pin_d4 = Y6_GPIO_NUM;
153     config.pin_d5 = Y7_GPIO_NUM;
154     config.pin_d6 = Y8_GPIO_NUM;
155     config.pin_d7 = Y9_GPIO_NUM;
156     config.pin_xclk = XCLK_GPIO_NUM;
157     config.pin_pclk = PCLK_GPIO_NUM;
158     config.pin_vsync = VSYNC_GPIO_NUM;
159     config.pin_href = HREF_GPIO_NUM;
160     config.pin_sccb_sda = SIOD_GPIO_NUM;
161     config.pin_sccb_scl = SIOC_GPIO_NUM;
162     config.pin_pwdn = PWDN_GPIO_NUM;
163     config.pin_reset = RESET_GPIO_NUM;
164     config.xclk_freq_hz = 10000000;
165     config.frame_size = FRAMESIZE_QVGA;
166     config.pixel_format = PIXFORMAT_JPEG; // for streaming
167     config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
168     config.fb_location = CAMERA_FB_IN_PSRAM;
169     config.jpeg_quality = 10;
170     config.fb_count = 1;
171
172     // camera init
173     esp_err_t err = esp_camera_init(&config);
174     if (err != ESP_OK) {
175         if(err==ESP_ERR_NOT_SUPPORTED) {
176             config.pixel_format = PIXFORMAT_RGB565;
177             esp_err_t err = esp_camera_init(&config);
178             if (err != ESP_OK) {
179                 Serial.printf("Camera init failed with error 0x%x", err);
```

```
180         return;
181     }
182 }
183 }
184
185 sensor_t * s = esp_camera_sensor_get();
186 // drop down frame size for higher initial frame rate
187 uint16_t pid = s->id.PID;
188 if(pid == OV2640_PID) {
189     s->set_hmirror(s, 1);
190     s->set_vflip(s, 1);
191 }
192 else if(pid == OV3660_PID) {
193     s->set_hmirror(s, 1);
194     s->set_vflip(s, 0);
195 }
196 else if(pid == GC2145_PID) {
197     s->set_hmirror(s, 0);
198     delay(500);
199     s->set_vflip(s, 0);
200 }
201 else if(pid == GC0308_PID) {
202     s->set_hmirror(s, 0);
203     delay(500);
204     s->set_vflip(s, 0);
205 }
206 else {
207     s->set_hmirror(s, 1);
208     s->set_vflip(s, 0);
209 }
210 s->set_brightness(s, 1); // Slightly increase brightness
211 s->set_saturation(s, 0); // Reduce saturation
212 s->set_ae_level(s, -3); // Set exposure compensation level
213 }
```

Include header files that drive camera and WiFi.

```

1  #include "esp_camera.h"
2  #include <WiFi.h>
3  #include <WiFiClient.h>
4  #include <WiFiAP.h>
5
6  #define CAMERA_MODEL_ESP32S3_EYE
7  #include "camera_pins.h"
8  #define LED_BUILT_IN 2

```

Set name and password for router that ESP32-S3 needs to connect to. And set ESP32-S3 to open two servers, whose port are 8000 and 5000 respectively.

```

10 const char *ssid_Router    = "*****";
11 const char *password_Router = "*****";
12 const char *ssid_AP        = "*****";
13 const char *password_AP     = "*****";

```

Enable ESP32-S3's server function and set two monitor ports as 5000 and 8000. In general, the two port numbers do not require modifications.

```

15 WiFiServer server_Cmd(5000);
16 WiFiServer server_Camera(8000);
17 extern TaskHandle_t loopTaskHandle;

```

Initialize serial port, set baud rate to 115200; open the debug and output function of the serial.

```

20 Serial.begin(115200);
21 Serial.setDebugOutput(true);
22 Serial.println();

```

Loop function will constantly send camera data obtained to mobile phone APP.

```

65 camera_fb_t * fb = NULL;
66 size_t jpg_buf_len = 0;
67 uint8_t *jpg_buf = NULL;
68 while (client.connected()) {
69     fb = esp_camera_fb_get();
70     if (!fb) {
71         Serial.println("Camera capture failed");
72     }
73     else {
74         if (fb->format != PIXFORMAT_JPEG) {
75             bool jpeg_converted = frame2jpg(fb, 80, &jpg_buf, &jpg_buf_len);
76             esp_camera_fb_return(fb);
77             fb = NULL;
78             if (!jpeg_converted) {
79                 Serial.println("JPEG compression failed");
80                 continue;
81             }
82         }
83         else {

```

```

84         jpg_buf_len = fb->len;
85         jpg_buf = fb->buf;
86     }
87     uint8_t slen[4];
88     slen[0] = jpg_buf_len >> 0;
89     slen[1] = jpg_buf_len >> 8;
90     slen[2] = jpg_buf_len >> 16;
91     slen[3] = jpg_buf_len >> 24;
92     client.write(slen, 4);
93     client.write(jpg_buf, jpg_buf_len);
94 }
95 if (fb) {
96     esp_camera_fb_return(fb);
97     fb = NULL;
98     jpg_buf = NULL;
99 }
100 else if (jpg_buf) {
101     free(jpg_buf);
102     jpg_buf = NULL;
103 }

```

The loopTask_Cmd() function sends the received instruction back to the phone app and prints it out through a serial port.

```

111 void loopTask_Cmd(void *pvParameters) {
112     Serial.println("Task Cmd_Server is starting ... ");
113     while (1) {
114         WiFiClient client = server_Cmd.available(); // listen for incoming clients
115         if (client) { // if you get a client,
116             Serial.println("Command Server connected to a client."); // print a message out the
serial port
117             String currentLine = ""; // make a String to hold incoming data from the client
118             while (client.connected()) { // loop while the client's connected
119                 if (client.available()) { // if there's bytes to read from the client,
120                     char c = client.read(); // read a byte, then
121                     client.write(c);
122                     Serial.write(c); // print it out the serial monitor
123                     if (c == '\n') { // if the byte is a newline character
124                         currentLine = "";
125                     }
126                     else {
127                         currentLine += c; // add it to the end of the currentLine
128                     }
129                 }

```

```

130     }
131     // close the connection:
132     client.stop();
133     Serial.println("Command Client Disconnected.");
134 }
135 }
136 }

```

loopTask_Blink()function will control the blinking of LED. When you see LED blinking, it indicates that ESP32-S3 has been configured and starts working.

```

137 void loopTask_Blink(void *pvParameters) {
138     Serial.println("Task Blink is starting ... ");
139     while (1) {
140         digitalWrite(LED_BUILT_IN, !digitalRead(LED_BUILT_IN));
141         delay(1000);
142     }
143 }

```

If you do not have a router near you, or if you are outdoors, you can annotate the following code, and then compile and upload it to ESP32-S3. And you can display the video images on your phone by Method 1.

```

33     //////////////////////////////////////
34     WiFi.begin(ssid_Router, password_Router);
35     Serial.print("Connecting ");
36     Serial.print(ssid_Router);
37     while (WiFi.isConnected() != true) {
38         delay(500);
39         Serial.print(".");
40     }
41     while (WiFi.STA.hasIP() != true) {
42         Serial.print(".");
43         delay(500);
44     }
45     Serial.println("");
46     Serial.println("WiFi connected");
47     //////////////////////////////////////

```

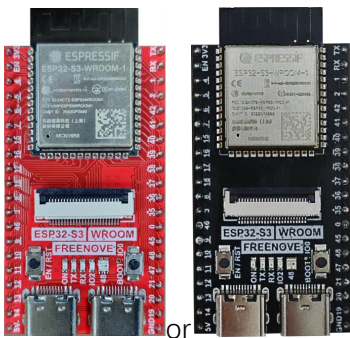
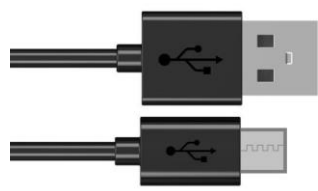
Chapter 9 USB

In this chapter, we will learn some simple examples of USB ports.

Project 9.1 USB Serial Example

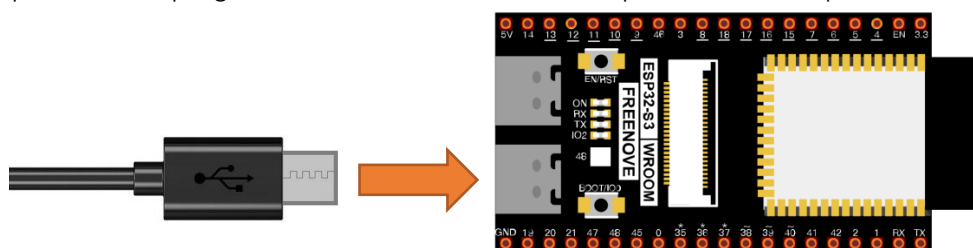
In this project, we have created a USB-to-serial bridge, allowing communication with the ESP32's hardware serial port via the USB interface.

Component List

ESP32-S3(N16R8)/ESP32-S3(N8R8) 	USB cable x1 
--	---

Power

ESP32-S3 WROOM needs 5v power supply. In this tutorial, we need connect ESP32-S3 WROOM to computer via USB cable to power it and program it. We can also use other 5v power source to power it.



In the following projects, we only use USB cable to power ESP32-S3 WROOM by default.

In the whole tutorial, we don't use T extension to power ESP32-S3 WROOM. So 5V and 3.3V (includeing EXT 3.3V) on the extension board are provided by ESP32-S3 WROOM.

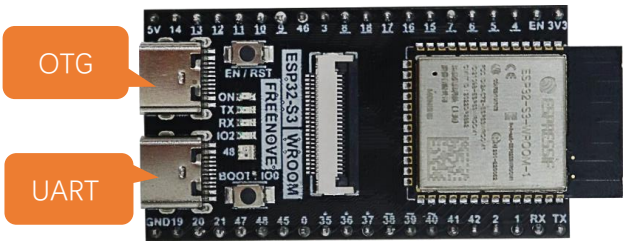
We can also use DC jack of extension board to power ESP32-S3 WROOM. In this way, 5v and EXT 3.3v on extension board are provided by external power resource.

Component knowledge

Difference between USB-OTG and USB-UART Interfaces

As shown in the figure below, our development board includes two USB interfaces: USB-OTG and USB-UART. Among them, the USB-UART interface is connected to the on-board USB-to-TTL circuit, which is then connected to the ESP32-S3 module's serial port pins (TX: GPIO43 and RX: GPIO44).

The USB-OTG interface is directly connected to the ESP32-S3 module's USB pins (USB_D-: GPIO19 and USB_D+: GPIO20).

USB-OTG	The USB-OTG interface allows code uploads and can simulate peripherals such as a mouse, keyboard, computer controller, or gamepad based on the provided code.	
USB-UART	The USB-UART interface is primarily used for code upload and serial communication. It is more stable and reliable than USB-OTG.	

Enabling USB-OTG

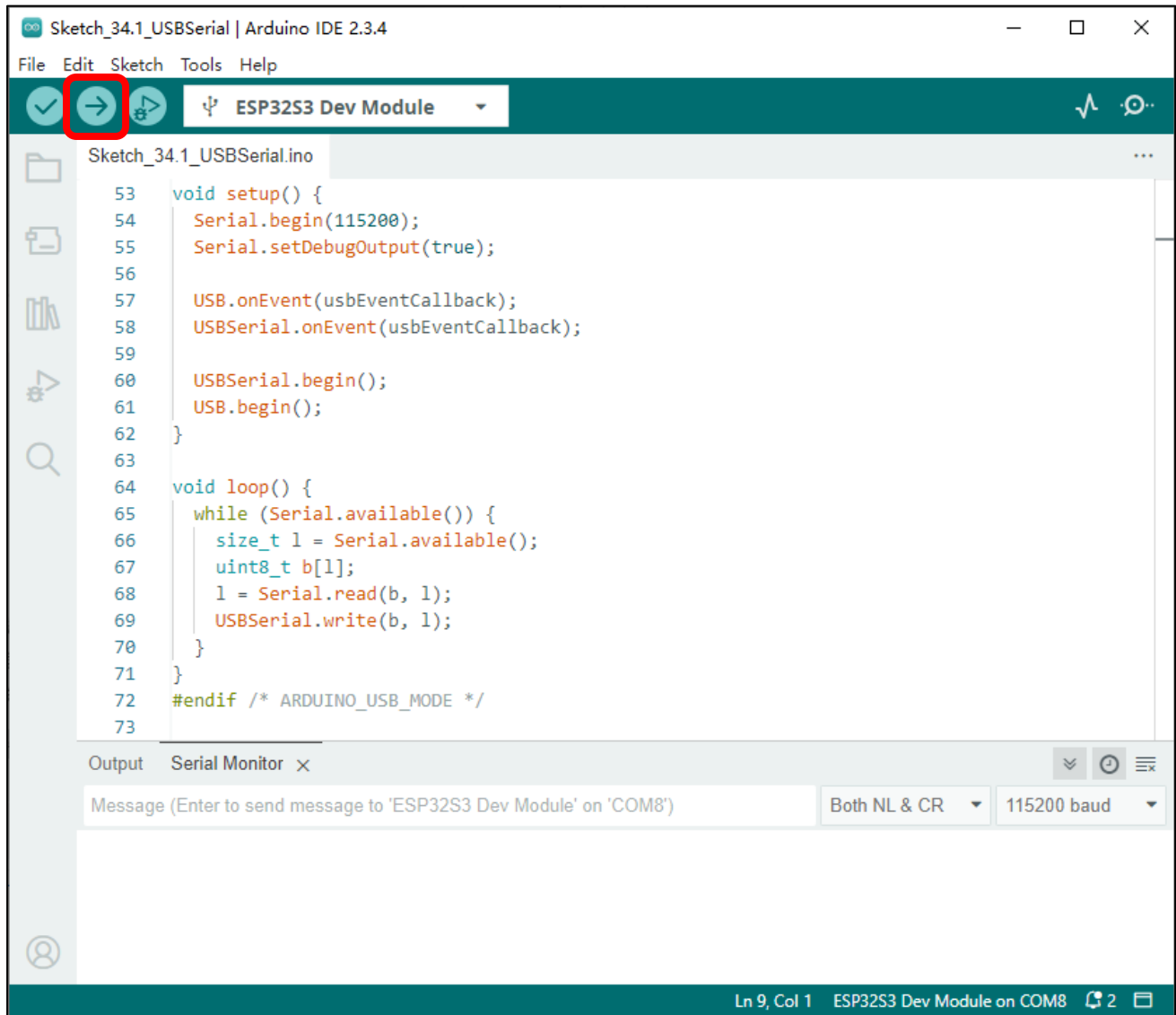
Please note: To use the USB function of the ESP32-S3, you must enable the configuration boxed below. Otherwise, the code will not take effect.

Auto Format	Ctrl+T
Archive Sketch	
Manage Libraries...	Ctrl+Shift+I
Serial Monitor	Ctrl+Shift+M
Serial Plotter	
Firmware Updater	
Upload SSL Root Certificates	
Board: "ESP32S3 Dev Module"	▶
Port: "COM13"	▶
Reload Board Data	
Get Board Info	
USB CDC On Boot: "Disabled"	▶
CPU Frequency: "240MHz (WiFi)"	▶
Core Debug Level: "None"	▶
USB DFU On Boot: "Disabled"	▶
Erase All Flash Before Sketch Upload: "Disabled"	▶
Events Run On: "Core 1"	▶
Flash Mode: "QIO 80MHz"	▶
Flash Size: "4MB (32Mb)"	▶
JTAG Adapter: "Disabled"	▶
Arduino Runs On: "Core 1"	▶
USB Firmware MSC On Boot: "Disabled"	▶
Partition Scheme: "Default 4MB with spiiffs (1.2MB APP/1.5MB SPIFFS)"	▶
PSRAM: "Disabled"	▶
Upload Mode: "UART0 / Hardware CDC"	▶
Upload Speed: "921600"	▶
USB Mode: "USB-OTG (TinyUSB)"	▶
Zigbee Mode: "Disabled"	▶
Programmer	▶
Burn Bootloader	

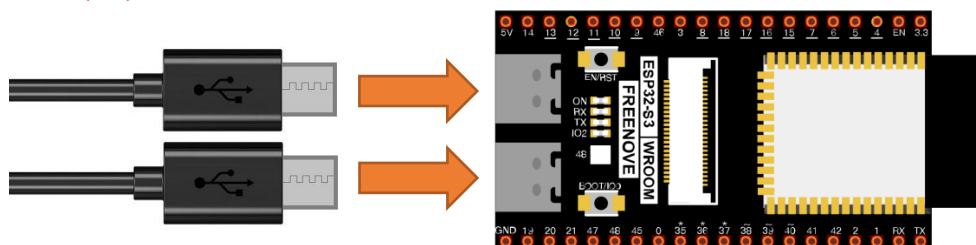
Sketch

Sketch_09.1_USBSerial

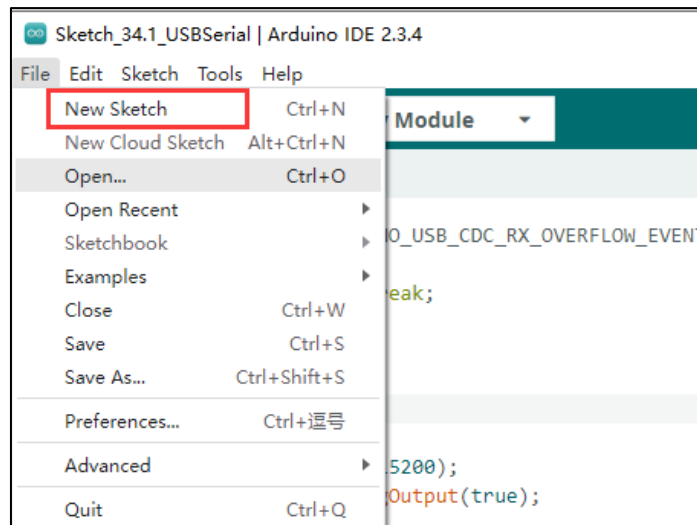
Click the icon to compile and upload the sketch to the ESP32S3.



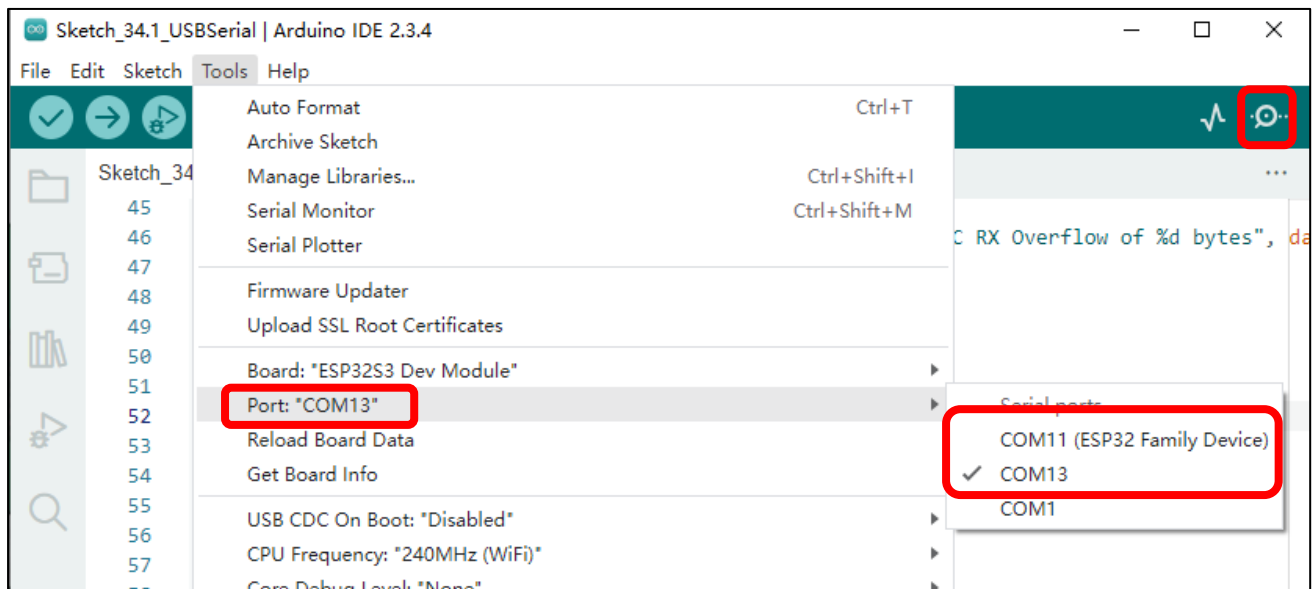
Note: You will need to prepare an additional USB cable to connect both USB interfaces simultaneously.



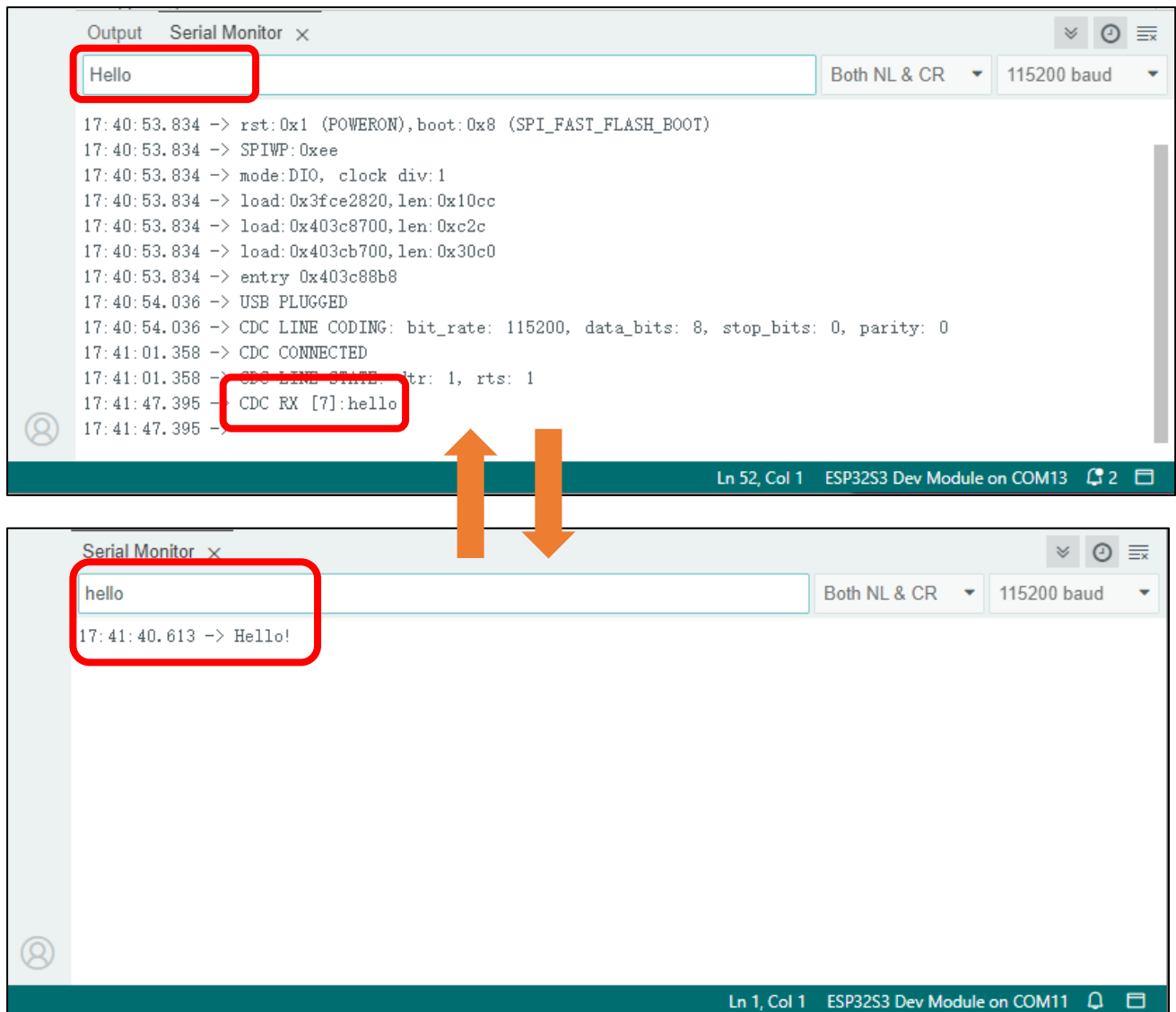
You can open a new interface by clicking File -> New Sketch.



Select the COM port numbers corresponding to USB-UART and USB-OTG respectively in the two interfaces, open the serial monitors, and set the baud rate to 115200.



You can input any content in the serial monitor of the USB-UART port and press Enter. The ESP32-S3 will detect the data input from the USB-UART, package it, and send it to the USB-OTG. Similarly, when the ESP32-S3 detects data input from the USB-OTG, it will package the data and forward it to the USB-UART.



The communication between USB-OTG and USB-UART is similar to that described in the Serial Communication section.

The following is the program code:

```

1  #ifndef ARDUINO_USB_MODE
2  #error This ESP32 SoC has no Native USB interface
3  #elif ARDUINO_USB_MODE == 1
4  #warning This sketch should be used when USB is in OTG mode
5  void setup() {}
6  void loop() {}
7  #else
8  #include "USB.h"
9

```

```

10  #if !ARDUINO_USB_CDC_ON_BOOT
11  USBCDC USBSerial;
12  #endif
13
14  static void usbEventCallback(void *arg, esp_event_base_t event_base, int32_t event_id, void
    *event_data) {
15      if (event_base == ARDUINO_USB_EVENTS) {
16          arduino_usb_event_data_t *data = (arduino_usb_event_data_t *)event_data;
17          switch (event_id) {
18              case ARDUINO_USB_STARTED_EVENT: Serial.println("USB PLUGGED"); break;
19              case ARDUINO_USB_STOPPED_EVENT: Serial.println("USB UNPLUGGED"); break;
20              case ARDUINO_USB_SUSPEND_EVENT: Serial.printf("USB SUSPENDED: remote_wakeup_en: %u\n",
data->suspend.remote_wakeup_en); break;
21              case ARDUINO_USB_RESUME_EVENT: Serial.println("USB RESUMED"); break;
22
23              default: break;
24          }
25      } else if (event_base == ARDUINO_USB_CDC_EVENTS) {
26          arduino_usb_cdc_event_data_t *data = (arduino_usb_cdc_event_data_t *)event_data;
27          switch (event_id) {
28              case ARDUINO_USB_CDC_CONNECTED_EVENT: Serial.println("CDC CONNECTED"); break;
29              case ARDUINO_USB_CDC_DISCONNECTED_EVENT: Serial.println("CDC DISCONNECTED"); break;
30              case ARDUINO_USB_CDC_LINE_STATE_EVENT: Serial.printf("CDC LINE STATE: dtr: %u,
rts: %u\n", data->line_state.dtr, data->line_state.rts); break;
31              case ARDUINO_USB_CDC_LINE_CODING_EVENT:
32                  Serial.printf(
33                      "CDC LINE CODING: bit_rate: %lu, data_bits: %u, stop_bits: %u, parity: %u\n",
data->line_coding.bit_rate, data->line_coding.data_bits,
34                      data->line_coding.stop_bits, data->line_coding.parity
35                  );
36                  break;
37              case ARDUINO_USB_CDC_RX_EVENT:
38                  Serial.printf("CDC RX [%u]:", data->rx.len);
39                  {
40                      uint8_t buf[data->rx.len];
41                      size_t len = USBSerial.read(buf, data->rx.len);
42                      Serial.write(buf, len);
43                  }
44                  Serial.println();
45                  break;
46              case ARDUINO_USB_CDC_RX_OVERFLOW_EVENT: Serial.printf("CDC RX Overflow of %d bytes",
data->rx_overflow.dropped_bytes); break;
47
48              default: break;

```

```

49     }
50 }
51 }
52
53 void setup() {
54     Serial.begin(115200);
55     Serial.setDebugOutput(true);
56
57     USB.onEvent(usbEventCallback);
58     USBSerial.onEvent(usbEventCallback);
59
60     USBSerial.begin();
61     USB.begin();
62 }
63
64 void loop() {
65     while (Serial.available()) {
66         size_t l = Serial.available();
67         uint8_t b[l];
68         l = Serial.read(b, l);
69         USBSerial.write(b, l);
70     }
71 }
72 #endif /* ARDUINO_USB_MODE */

```

Check the configuration information. If the configuration is incorrect, the code will not run properly. Please pay attention to the prompt messages during compilation.

```

#ifndef ARDUINO_USB_MODE
#error This ESP32 SoC has no Native USB interface
#elif ARDUINO_USB_MODE == 1
#warning This sketch should be used when USB is in OTG mode
void setup() {}
void loop() {}
#else
#include "USB.h"

#if !ARDUINO_USB_CDC_ON_BOOT
USBCDC USBSerial;
#endif

```

The USB callback function is primarily used to handle USB events, CDC events, data reception events, and so on.

```

static void usbEventCallback(void *arg, esp_event_base_t event_base, int32_t event_id, void
*event_data) {
    if (event_base == ARDUINO_USB_EVENTS) {
        arduino_usb_event_data_t *data = (arduino_usb_event_data_t *)event_data;

```

```

switch (event_id) {
    case ARDUINO_USB_STARTED_EVENT: Serial.println("USB PLUGGED"); break;
    case ARDUINO_USB_STOPPED_EVENT: Serial.println("USB UNPLUGGED"); break;
    case ARDUINO_USB_SUSPEND_EVENT: Serial.printf("USB SUSPENDED: remote_wakeup_en: %u\n",
data->suspend.remote_wakeup_en); break;
    case ARDUINO_USB_RESUME_EVENT: Serial.println("USB RESUMED"); break;

    default: break;
}
} else if (event_base == ARDUINO_USB_CDC_EVENTS) {
    arduino_usb_cdc_event_data_t *data = (arduino_usb_cdc_event_data_t *)event_data;
    switch (event_id) {
        case ARDUINO_USB_CDC_CONNECTED_EVENT: Serial.println("CDC CONNECTED"); break;
        case ARDUINO_USB_CDC_DISCONNECTED_EVENT: Serial.println("CDC DISCONNECTED"); break;
        case ARDUINO_USB_CDC_LINE_STATE_EVENT: Serial.printf("CDC LINE STATE: dtr: %u,
rts: %u\n", data->line_state.dtr, data->line_state.rts); break;
        case ARDUINO_USB_CDC_LINE_CODING_EVENT:
            Serial.printf(
                "CDC LINE CODING: bit_rate: %lu, data_bits: %u, stop_bits: %u, parity: %u\n",
data->line_coding.bit_rate, data->line_coding.data_bits,
                data->line_coding.stop_bits, data->line_coding.parity
            );
            break;
        case ARDUINO_USB_CDC_RX_EVENT:
            Serial.printf("CDC RX [%u]:", data->rx.len);
            {
                uint8_t buf[data->rx.len];
                size_t len = USBSerial.read(buf, data->rx.len);
                Serial.write(buf, len);
            }
            Serial.println();
            break;
        case ARDUINO_USB_CDC_RX_OVERFLOW_EVENT: Serial.printf("CDC RX Overflow of %d bytes",
data->rx_overflow.dropped_bytes); break;

        default: break;
    }
}
}
}

```

Initialize the serial port, configure callback functions for both USB and USBSerial, and then initialize USBSerial and USB. Note that USB.begin() should be placed last during initialization.

```

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
}

```

```

USB.onEvent(usbEventCallback);
USBSerial.onEvent(usbEventCallback);

USBSerial.begin();
USB.begin();
}

```

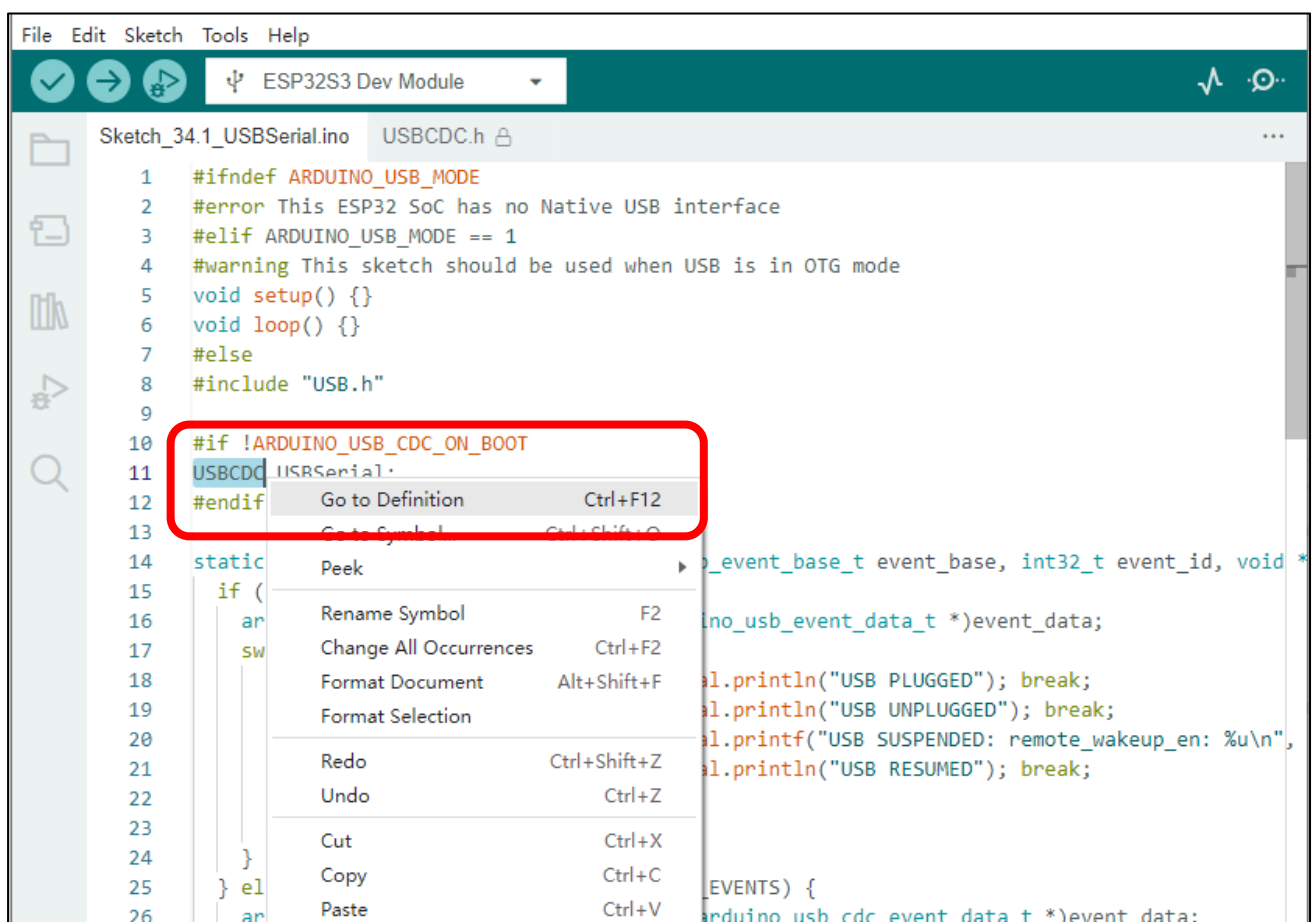
If data is received on the USB-UART interface, obtain the length of the data in the buffer, create an array, read the data from the buffer into the array, and then send the data out via the USB-OTG interface.

```

void loop() {
  while (Serial.available()) {
    size_t l = Serial.available();
    uint8_t b[l];
    l = Serial.read(b, l);
    USBSerial.write(b, l);
  }
}

```

If you wish to learn more about USBSerial, you can select it in the code, right-click, and choose "Go to Definition".



What's next?

Thanks for your reading. This tutorial is all over here. If you find any mistakes, omissions or you have other ideas and questions about contents of this tutorial or the kit and etc., please feel free to contact us:

support@freenove.com

We will check and correct it as soon as possible.

If you want learn more about ESP32-S3, you view our ultimate tutorial:

https://github.com/Freenove/Freenove_Ultimate_Starter_Kit_for_ESP32_S3/archive/master.zip

If you want to learn more about Arduino, Raspberry Pi, smart cars, robots and other interesting products in science and technology, please continue to focus on our website. We will continue to launch cost-effective, innovative and exciting products.

<http://www.freenove.com/>

End of the Tutorial

Thank you again for choosing Freenove products.